

Verification Continuum™

Verdi® Transaction and Protocol Debug User Guide

Version W-2024.09, September 2024



Copyright and Proprietary Information Notice

© 2024 Synopsys, Inc. This Synopsys software and all associated documentation are proprietary to Synopsys, Inc. and may only be used pursuant to the terms and conditions of a written license agreement with Synopsys, Inc. All other use, reproduction, modification, or distribution of the Synopsys software or the associated documentation is strictly prohibited.

Destination Control Statement

All technical data contained in this publication is subject to the export control laws of the United States of America. Disclosure to nationals of other countries contrary to United States law is prohibited. It is the reader's responsibility to determine the applicable regulations and to comply with them.

Disclaimer

SYNOPSYS, INC., AND ITS LICENSORS MAKE NO WARRANTY OF ANY KIND, EXPRESS OR IMPLIED, WITH REGARD TO THIS MATERIAL, INCLUDING, BUT NOT LIMITED TO, THE IMPLIED WARRANTIES OF MERCHANTABILITY AND FITNESS FOR A PARTICULAR PURPOSE.

Trademarks

Synopsys and certain Synopsys product names are trademarks of Synopsys, as set forth at <https://www.synopsys.com/company/legal/trademarks-brands.html>. All other product or company names may be trademarks of their respective owners.

Free and Open-Source Licensing Notices

If applicable, Free and Open-Source Software (FOSS) licensing notices are available in the product installation.

Third-Party Links

Any links to third-party websites included in this document are for your convenience only. Synopsys does not endorse and is not responsible for such websites and their practices, including privacy practices, availability, and content.

www.synopsys.com

Contents

Customer Support	8
Synopsys Statement on Inclusivity and Diversity	8
1. Introducing Verdi Transaction and Protocol Debug	10
2. Using Data Types in Transaction Debug	12
Transactions	12
Streams	13
Attributes	13
Tags	13
Transaction Relations	14
3. Dumping Transactions into FSDB	15
Automatically Recording UVM Transactions and Messages	15
Compiling and Linking a Design With VCS and FSDB Dumper	17
Compiling and Linking a UVM Example With Other Simulators	21
Recording Transactions for Non-Sequencer Components	25
Supporting VCS Three-Step Flow	26
Transaction begin_time	27
Dumping Data Using SystemVerilog Wrapper Functions (API)	27
Stream Tasks	29
Transaction Tasks	31
Transaction/Relation Tasks	33
Miscellaneous Tasks	33
Simple API Example	34
Rules	35
Coding Style for API	36
Automatically Dumping Transactions From Synopsys VIP	37
Using VC APPS Protocol Analyzer to Dump Transactions From Custom VIP	37
4. Using Transaction and Protocol Analyzer	39

Contents

Invoking Transaction Debug Model	40
Invoking Transaction and Protocol Analyzer	41
Importing an FSDB to the Transaction and Protocol Analyzer Window	42
Transaction and Protocol Analyzer Window Overview	42
Hierarchy Tree Pane	44
Stream Pane	51
Transaction Pane	53
Preference Options	62
Transaction and Protocol Analyzer Operations	64
Displaying Attribute Name and Value as Additional Text in Transaction Pane ..	65
Highlighting by Relation	66
Adding Streams by Relation From the Selected Transaction	67
Search Capability in Hier. Tree Tab	68
Search or Filter Capability in Details Tab	69
Search Capability for Transactions	69
Configuring Display	70
Filter/Colorize the Transactions	71
Creating Virtual Streams	73
Copying Stream Name From Protocol Analyzer Window	80
Interlinking Between Transactions and Signals	81
Loading UVM Interface Signals From the Selected Transaction	82
Loading Protocol Interface Signals to Selected nWave Application	85
Loading Transactions to Selected Transaction and Protocol Analyzer Application	87
Synchronization With nWave	90
Points to Note	93
Connecting nWave and Transaction and Protocol Analyzer	93
5. Using Cache and Memory Protocol Analyzer (Memory Mode)	97
Cache and Memory Protocol Analyzer Overview	99
Invoking Cache and Memory Protocol Analyzer	99
Cache and Memory Protocol Analyzer Window Overview	100
Memory Array Pane	100
Details Pane	103
Synchronization With Verdi Transaction and Protocol Analyzer	104
Synchronization With Transaction Table View	105

6. Using Transaction Table View	107
Opening Transaction Table View	107
Adding Items to Table View	108
Exporting the Table Content	109
7. Using Transaction Relation Navigator	110
Invoking the Transaction Relation Navigator Window	110
Transaction Relation Navigator Window	111
Analyzing Transaction Relationships in Transaction Relation Navigator	113
Importing Transactions into Transaction Relation Navigator	113
Relationship Types	114
Viewing Operations	114
8. Performing Interactive Simulation Debug	118
Linking Transaction and Protocol Analyzer With Testbench Debug Browsers	123
Reverse Debug Capability in Transaction and Protocol Analyzer	124
9. Practice	127
Compiling and Simulating Design With UVM Environment	127
Viewing Transactions in Transaction and Protocol Analyzer Window	128
Viewing Transactions in the Transaction Table View Window	132
Viewing Transactions in Transaction Relation Navigator Window	136
Installing VIP Examples	139
Configuring VIP to Generate Transaction FSDB File	139
Viewing VIP Transactions in Transaction and Protocol Analyzer Window	139
Viewing VIP Transactions in Transaction Table View Window	141
Viewing VIP Transactions in Transaction Relation Navigator Window	143
A. Appendix A: Using Transaction and Protocol Analyzer (Protocol Debug Mode)	144
Invoking Protocol Debug	144
Invoking Transaction and Protocol Analyzer	146

Contents

Prerequisites	146
Transaction and Protocol Analyzer Window Overview	147
Protocol Analyzer Toolbar or Menu Bar Overview	150
Tree pane	155
Detail Pane	162
B. Appendix B: Automatically Record SCV Transactions	166
Use Model	166
Limitations	167
C. Appendix C: Dumping Transaction Data Using SC-API	169
SC-API Descriptions	170
Execution Flow	174
Example Using VCS	174
Example Using IUS	175
D. Appendix D: Automatically Recording OVM Transactions and Messages ...	176
Compiling and Linking a Design	176
Support for UVM Aware Debug	177
Recording Transactions for Non-Sequencer Components	178
E. Appendix E: Using the Old Transaction Table View	180
Importing FSDB to the Transaction Table View Window	180
Transaction Table View Window	181
Reading and Analyzing Transactions in Transaction Table View	183
Importing Streams into Transaction Table View	183
Column (Attribute) Operations	189
Row (Transaction) Operations	191
Support for Transaction Comparison	195
Showing Transaction Relation Navigator	202
Auto-Switching During Sync Selection	203
F. Appendix F: Log Conversion Using the log2transdb Utility	205

Contents

Log Messages Supported by Log Analyzer	206
Logging and Debugging	206

Preface

The Verdi Transaction and Protocol Debug User Guide provides Transaction and Protocol Debug solution in the Verdi Automated Debug Platform.

Customer Support

For any online access to the self-help resources, you can refer to the documentation and searchable knowledge base available in SolvNetPlus.

To obtain support for your Verdi product, choose one of the following:

- Open a case through SolvNetPlus.

Go to <https://solvnetplus.synopsys.com/s/contactsupport> and provide the requested information, including:

- Product L1 as Verdi
- Case Type

Fill in the remaining fields according to your environment and issue.

- Send an e-mail message to verdi_support@synopsys.com.

Include product name (L1), sub-product name/technology (L2), and product version in your e-mail, so it can be routed correctly.

Your e-mail will be acknowledged by automatic reply and assigned a Case number along with Case reference ID in the subject (ref:____:ref).

For any further communication on this Case via e-mail, send e-mail to verdi_support@synopsys.com and ensure to have the same Case ref ID in the subject header or else it will open duplicate cases.

Note:

In general, we need to be able to reproduce the problem in order to fix it, so a simple model demonstrating the error is the most effective way for us to identify the bug. If that is not possible, then provide a detailed explanation of the problem along with complete error and corresponding code, if any/permissible.

Synopsys Statement on Inclusivity and Diversity

Synopsys is committed to creating an inclusive environment where every employee, customer, and partner feels welcomed. We are reviewing and removing exclusionary language from our products and supporting customer-facing collateral. Our effort also

Contents

includes internal initiatives to remove biased language from our engineering and working environment, including terms that are embedded in our software and IPs. At the same time, we are working to ensure that our web content and software applications are usable to people of varying abilities. You may still find examples of non-inclusive language in our software or documentation as our IPs implement industry-standard specifications that are currently under review to remove exclusionary language.

1

Introducing Verdi Transaction and Protocol Debug

The Transaction and Protocol Debug solution in the *Verdi® Automated Debug Platform* is an add-on option that is designed from the ground-up for capture, display, debug, and analysis of high level abstraction data, such as, UVM sequences, UVM messages, VIP protocol information at different layers or any other information that users want to access at high-level. In general, all these data at different abstraction levels are called transactions.

This document is intended to illustrate the capabilities as well as provide guidelines on how to get started for early users.

The Verdi Transaction and Protocol Debug mechanism includes two key aspects:

- **The data:** Transaction data (can be any level of abstraction) needs to be dumped into an FSDB file. For UVM, Verdi provides transaction recorder and message catcher to automatically record UVM sequences and UVM messages. Synopsys VIPs can also be enabled to automatically dump the protocol level transaction data into an FSDB file. SystemVerilog tasks, SystemC APIs, and VC APPs NPIs are also provided by Verdi for advanced users to use in their custom VIP or other verification environments.
- **The applications:** Several application windows are provided by Verdi to help debug and analyze the transaction data recorded in an FSDB file. Each of these applications come with a rich set of features geared towards analyzing and debugging at this level, such as, filtering, sorting, and highlighting.

The *Transaction and Protocol Analyzer* provides a temporal view to help navigate the transactions (from UVM, VIP, and so on) across time and testbench hierarchies.

The *Transaction Table View* provides a spreadsheet look and feel to help analyze the details of the transaction attributes.

The *Transaction Relation View* provides a cockpit to help trace from one transaction to its related transactions.

The *Cache and Memory Protocol Analyzer* is designed to specifically analyze cache and memory issues at protocol level.

The *Performance Analyzer* is a special tool to analyze the protocol performance at SoC level. For details on Performance Analyzer, see *Verdi Performance Analyzer User Guide*.

The remainder of this document provides further details on the data (transactions), UVM automatic recording flow, how to record this data to the FSDB file, and how to use the applications.

The old FSDB file is not compatible in the Transaction Debug mechanism of the Verdi platform with versions prior to 2013.01 and the Verdi platform with versions prior to version 2013.04 cannot read the transaction-based FSDB.

2

Using Data Types in Transaction Debug

This chapter summarizes the Verdi Transaction Debug data type categories and their definitions, including *Transactions*, *Streams*, *Attributes*, *Transaction Relations* and *Tags* in the following sections:

- [Transactions](#)
- [Streams](#)
- [Attributes](#)
- [Tags](#)
- [Transaction Relations](#)

Transactions

A *Transaction* is an entry of data input by user code (for example, an UVM message) and it contains attributes of different data types. Transactions can be linked by a relation. For maximum flexibility, transactions can be of different types including but not limited to the traditional transaction type. The predefined transaction types (for built-in visualization) are as follows:

- **Message:** Regular transactions for `printf` like logging purposes.
- **Action:** A 0-time transaction for recording some Actions (for example read, write, interrupt). Several Action transactions can be formed into a Group transaction by adding the `belong_to` relation between the Action transaction and the Group transaction.
- **Transaction:** A transaction with a begin time and an end time. A transaction type can be 0-time, that is, begin time is equal to end time. The Transaction type and Action type are relative concepts, based on data abstraction levels.
- **Group:** A group contains several member transactions (usually of Action type) that together form a Transaction. The begin time of the Group transaction adopts the begin time of the first member transaction and the end time adopts the end time of the last member child.

Different types of transactions can be shown with different symbols (shapes) in the supported visualization, debug, and analysis applications.

Streams

A *Stream* is used to organize (or record as a group in the FSDB file) a temporal sequence of transactions grouped by some criteria, for example, transactions originating from the same UVM component can be put into one stream. Different streams cannot share the same transaction. Some streams have hierarchies (for example, following the UVM component hierarchy), some do not (that is, UVM phases, objections). Streams may form different hierarchies (UVM component hierarchy and sequence hierarchy). A stream can be expanded into rows with the same type of transactions in each row. A *Virtual Stream* that is a group of transactions organized based on user-defined criteria can also be created – this is discussed later in the document.

Attributes

An *Attribute* is a characteristic or a property of a transaction. Attributes are divided into two categories: *Dense Attribute* and *Sparse Attribute*.

- **Dense Attribute** (the predefined attribute): If all transactions in a stream have an attribute of the same name and data type, the attribute is called a dense attribute. Dense attributes need to be predefined when creating a stream. Although predefined, dense attributes still need to be added for each transaction.
- **Sparse Attribute**: If only a few (not all) transactions have an attribute of specific name and data type, or the attribute cannot be predefined at the stream creation stage, the attribute is called a sparse attribute. Sparse attributes do not need to be predefined, and can be directly added to a transaction.

Tags

A *Tag* is a special attribute with no name and has been optimized for searching. The identifier that is attached to a transaction is for identification or other informational purposes. A tag is a non-hierarchical keyword or term assigned to a transaction. This kind of metadata helps to describe an item and allows it to be found again by browsing or searching. Multiple tags can be associated with a transaction. A tag ties a transaction to the keyword and it could be used to imply some relationship between transactions or some attributes or any implied meaning interpreted by the creators or by the viewers.

Transaction Relations

A *Relation* is a link between transactions. Relationships can be defined between transactions using the `$fsdbTrans_add_relation(string rel_name, trans_id transaction1, trans_id transaction2)` task, where relation (`rel_name`) can be:

- **belong_to**: In the `belong_to` relation, *transaction2* belongs to *transaction1*. *transaction1* can only be a Group type and *transaction2* cannot be a Transaction or Group type. *transaction2* can only belong to *transaction1*. `belong_to` is a grouping relationship and can model a container (for example, using pause messages to model a Transaction or a Transaction Group) and may contain several action transactions.
- **annotate_to**: In the `annotate_to` relation, annotates *transaction1* to *transaction2*. `annotate_to` is the annotative relationship, *transaction1* is the annotator and *transaction2* is the annotatee. *transaction1* can only be a message type (for example, objections annotate to phases in UVM).
- **parent_child**: In the `parent_child` relation, *transaction1* is the parent and *transaction2* is the child. `parent_child` is the protocol stacking/layering relationship and can model the parent/child relationship of UVM sequences (transactions). This relationship only applies to Transaction type. Parent/child can be in different streams.

Note that for UVM recorded transactions, the child range must be within the range of the parent.

- **pred_succ**: In the `pred_succ` relation, *transaction1* is the predecessor and *transaction2* is the successor. `pred_succ` is the temporal ordering/triggering relationship and can model a Transaction trace across different test units (for example UVM components). The time of the predecessor must be earlier than the successor.
- **any_string**: User defined relationship.

Note:

Setting a run-time relationship is not supported.

3

Dumping Transactions into FSDB

This chapter provides details on the various methods to dump the transaction data from the Synopsys VIPs in the following sections:

- [Automatically Recording UVM Transactions and Messages](#)
- [Dumping Data Using SystemVerilog Wrapper Functions \(API\)](#)
- [Automatically Dumping Transactions From Synopsys VIP](#)
- [Using VC APPS Protocol Analyzer to Dump Transactions From Custom VIP](#)

Note:

- For details on automatically recording OVM transactions and messages, see [Appendix D: Automatically Recording OVM Transactions and Messages](#).
- For details on dumping transaction data using SC-API, see [Appendix C: Dumping Transaction Data Using SC-API](#).
- For details on automatically recording SCV transactions, see [Appendix B: Automatically Record SCV Transactions](#).

Automatically Recording UVM Transactions and Messages

For the UVM users, the Verdi platform provides an automatic recording mechanism to record UVM transactions and messages into an FSDB file with the data types described in [Using Data Types in Transaction Debug](#) for transaction debugging and analysis in the Verdi platform. The following UVM libraries have been set up to record all the data as described in [Using Data Types in Transaction Debug](#).

Table 1 UVM Libraries and the Location

UVM Library	Location
UVM 1.1d	\$VERDI_HOME/etc/uvm
UVM 1.2	\$VERDI_HOME/etc/uvm-1.2

Table 1 UVM Libraries and the Location (Continued)

UVM-ieee	\$VERDI_HOME/etc/uvm-ieee
UVM-ieee-2020	\$VERDI_HOME/etc/uvm-ieee-2020
UVM-ieee-2020-2.0	\$VERDI_HOME/etc/uvm-ieee-2020-2.0

The following sub-sections discuss how you can compile and link these libraries with VCS and other simulators:

- [Compiling and Linking a Design With VCS and FSDB Dumper](#)
- [Compiling and Linking a UVM Example With Other Simulators](#)
- [Recording Transactions for Non-Sequencer Components](#)
- [Supporting VCS Three-Step Flow](#)

Note:

- Starting with Verdi 2015.09 and later versions, UVM 1.0-p1 is no longer provided.

You can use the following simple methods to enable automatic UVM transaction or message recording with these libraries, that is, you can basically get enough data within this UVM usage without adding dumping commands to control dumping data to an FSDB file. In addition, you can manually add the APIs provided by the Verdi platform to record more desired transaction and message data in the FSDB file for advanced usage (For details, see [Dumping Data Using SystemVerilog Wrapper Functions \(API\)](#)).

- The Verdi-modified library has added code for recording purposes only. No existing code is removed or modified and hence the functionality of the library remains the same.

Change the directory to the corresponding example path¹ and follow the steps described in this chapter to set the environment variables, and specify the compile and simulation options.

- The methods described in this document and the previous method for recording UVM transactions and messages cannot be mixed. Only one method can be used at the same time. Otherwise, recording transactions and messages would not be correct.

1. For UVM 1.1d usage, change directory to \$VERDI_HOME/share/vmlib/uvm/uvm/lib/examples/integrated/ubus_evdntb/examples. For UVM 1.2 usage, change directory to \$VERDI_HOME/share/vmlib/uvm/uvm-1.2/lib/examples/integrated/ubus/examples.

Compiling and Linking a Design With VCS and FSDB Dumper

This section describes how you can compile a design with UVM libraries.

Note:

If you want to specify a different UVM library, you can use the `VCS_UVM_HOME` environment variable to specify it.

Compiling With UVM-1.1d

Add `-ntb_opts uvm` and `-debug_access[+argument]` compilation options for the UVM library. For example:

```
%> vcs -full64 -debug_access+all -kdb -ntb_opts uvm -sverilog \
-l compile.log -timescale=1ns/1ns -V \
+incdir+
$VERDI_HOME/share/vmlib/uvm/uvm/lib/examples/integrated/ubus_evdp_ntb/sv
ubus_tb_top.sv
```

Note:

The `-debug_access[+argument]` VCS compile option is available only with VCS J-2014.12 or later versions.

Compiling With UVM-1.2

Add `-ntb_opts uvm` and `-debug_access[+argument]` compilation options for the UVM library. For example:

```
%> vcs -full64 -debug_access+all -kdb -ntb_opts uvm-1.2
-sverilog -l compile.log -timescale=1ns/1ns -V \
+incdir+
$VERDI_HOME/share/vmlib/uvm/uvm-1.2/lib/examples/ integrated/ubus /sv
ubus_tb_top.sv
```

Note:

The `-debug_access[+argument]` VCS option is available only with VCS J-2014.12 or later versions.

The `UVM_VERDI_PORT_RECORDING` compiler directive is provided to enable Verdi to record all port-level transaction into FSDB. It can be used for both UVM-1.1 and UVM-1.2.

Note:

You can load UVM-IEEE library using the `-ntb_opts uvm-ieee` compile-time option.

You can load UVM-IEEE-2020 library using the `-ntb_opts uvm-ieee-2020` compile-time option.

You can load UVM-IEEE-2020-2.0 library using the `-ntb_opts uvm-ieee-2020-2.0` compile-time option.

Running a Simulation

You need to add the `+UVM_VERDI_TRACE` runtime option to enable Verdi UVM flow with one of the following options:

- **+UVM_TR_RECORD**: Enable transaction recording and record transactions into the FSDB file and print to the console and log files.
- **+UVM_LOG_RECORD**: Enable message catching and record messages into the FSDB file and print to the console and log files.

For example,

```
// Enable both transaction recording and message catching
%>./simv <run_time_option> +UVM_VERDI_TRACE +UVM_LOG_RECORD
+UVM_TR_RECORD

// Only enable transaction recording
%>./simv <run_time_option> +UVM_VERDI_TRACE +UVM_TR_RECORD
```

Verdi is now used by default for transaction recording and message catching (instead of DVE), that is, without specifying the `+UVM_VERDI_TRACE` simulation option, in the following scenarios:

- Under the VC environment, if `+UVM_TR_RECORD` and `+UVM_LOG_RECORD` are used. For example, the following command records transactions and messages into the FSDB if it is under the VC environment.

```
./simv +UVM_TESTNAME=test_read_modify_write +UVM_TR_RECORD
+UVM_LOG_RECORD
```

- If environment variable is set to Verdi - `SNPS_VCS_DEFAULT_GUI=Verdi`, transactions and messages are recorded into the FSDB if `+UVM_TR_RECORD` and `+UVM_LOG_RECORD` are used.

For example, the following command records transactions and messages into the FSDB if the environment variable is set as `SNPS_VCS_DEFAULT_GUI=Verdi`:

```
./simv +UVM_TESTNAME=test_read_modify_write +UVM_TR_RECORD
+UVM_LOG_RECORD
```

- If the `simv` option `-gui=verdi` is specified, transactions and messages are recorded into the FSDB if `+UVM_TR_RECORD` and `+UVM_LOG_RECORD` are used. For example, the following command records transactions and messages into the FSDB:

```
./simv +UVM_TESTNAME=test_read_modify_write +UVM_TR_RECORD
+UVM_LOG_RECORD -gui=verdi
```

Note:

The `SNPS_SIM_DEFAULT_GUI` environment variable is available only with the Verdi J-2014.12-SP1 or later versions.

You can also use the `+UVM_VERDI_TRACE` runtime option to enable UVM port level transaction recording and Verdi UVM aware debug features.

See table [UVM_VERDI_TRACE Arguments](#) for the format and details.

Table 2 *UVM_VERDI_TRACE Arguments*

+UVM_VERDI_TRACE	
Usage	<code>+UVM_VERDI_TRACE = "UVM_AWARE RAL TLM TLM2 IMP HIER"</code> or <code>+UVM_VERDI_TRACE = "UVM_AWARE+RAL+TLM+TLM2+IMP+HIER"</code> . Note: Both + and are supported. It is recommended that you use + to combine the arguments.
Arguments	
UVM_AWARE	This argument is used for UVM Aware Debug only. For details, see <i>Verdi UVM Debug User Guide</i> .
RAL	This argument is used for UVM Aware Debug only. For details, see <i>Verdi UVM Debug User Guide</i> .
TLM	TLM port level transaction recording is enabled. The <code>+UVM_TR_RECORD</code> option is automatically enabled to record transactions into the FSDB file. Note: To enable this feature, the <code>UVM_VERDI_PORT_RECORDING</code> compiler directive is required.
TLM2	TLM2 port level transaction recording is enabled. The <code>+UVM_TR_RECORD</code> option is automatically enabled to record transactions into the FSDB file. Note: - To enable this feature, the <code>UVM_VERDI_PORT_RECORDING</code> compiler directive is required. - This argument is available only with the Verdi J-2014.12-SP1 or later versions.

Table 2 UVM_VERDI_TRACE Arguments (Continued)

IMP	Implementation port recording is enabled. This option requires TLM or TLM2 arguments. For example, Enable TLM port level transaction recording and implementation port recording: <pre>%> simv +UVM_VERDI_TRACE="TLM+IMP"</pre> Enable TLM2 port level transaction recording and implementation port recording: <pre>%> simv +UVM_VERDI_TRACE="TLM2+IMP"</pre> Note: This argument is available only with the Verdi J-2014.12-SP1 or later versions
HIER	This argument is used for UVM Aware Debug only. For details, see <i>Verdi UVM Debug User Guide</i>.

Multiple features can be enabled with “|” or “+” operator. For example, you can enable both TLM and TLM2 port level transaction recording and message dumping with the `+UVM_VERDI_TRACE="TLM+TLM2"` compound argument:

```
%>./simv +UVM_VERBOSITY=UVM_LOW
+UVM_TESTNAME=test_read_modify_write
+UVM_VERDI_TRACE="TLM|TLM2"
```

Note:

- After the simulation, the `novas.fsdb` FSDB file is generated by default and it includes UVM transactions and messages. Alternatively, the `+fsdbfile +filename` runtime option is also provided during simulation to direct both the signal and the transaction level data to the specified file name. For example:

```
%>./simv +UVM_LOG_RECORD +UVM_TR_RECORD +UVM_VERDI_TRACE
+fsdbfile+new.fsdb
```

- Similar to signal dumping, the `$fsdbDumpfile` task can also be used to change the FSDB file name. However, sometimes the transaction dumping tasks can happen before `$fsdbDumpfile`, leading to the two FSDB files being dumped. Therefore, it is highly recommended that `+fsdbfile +filename` be used if transaction dumping is involved.
- You need to use the Transaction Debug features described in the subsequent chapters to recognize transactions and messages in the `novas.fsdb` transaction-based FSDB file.

Compiling and Linking a UVM Example With Other Simulators

This section contains the following sub-sections:

- [Compiling and Simulating With UVM-1.1d](#)
- [Compiling and Simulating With UVM-1.2](#)

Compiling and Simulating With UVM-1.1d

Set the `LD_LIBRARY_PATH` environment variable with the path of the FSDB dumper before the compilation. For example,

```
%>setenv LD_LIBRARY_PATH $VERDI_HOME/share/PLI/MODELSIM/LINUX64
```

Explicitly specify the paths and the files to include the UVM library and the Verdi recorder and catcher during the compilation:

```
+incdir+$VERDI_HOME/etc/uvm \  
$VERDI_HOME/etc/uvm/uvm.sv \  
$VERDI_HOME/etc/uvm/dpi/uvm_dpi.cc \  
+incdir+$VERDI_HOME/etc/uvm/verdi \  
$VERDI_HOME/etc/uvm/verdi/  
uvm_custom_install_verdi_recorder.sv
```

For example,

```
%> vlog +incdir+$$VERDI_HOME/etc/uvm \  
$VERDI_HOME/etc/uvm/uvm.sv \  
$VERDI_HOME/etc/uvm/dpi/uvm_dpi.cc \  
+incdir+$VERDI_HOME/etc/uvm/verdi \  
$VERDI_HOME/etc/uvm/verdi/uvm_custom_install_verdi_recorder.sv \  
<compile_options>
```

Add the following runtime options to enable Verdi transaction recorder and message catcher:

- **+UVM_VERDI_TRACE=<Argument>**: Enables the Verdi UVM flow when added during simulation. You can use any of the following values as an input to the `<Argument>` parameter: `UVM_AWARE +RAL+TLM+MSG+HIER+PRINT` (see section [Running a Simulation](#) for details).
- **+UVM_TR_RECORD**: Enables Verdi transaction recorder.

- **+UVM_LOG_RECORD**: Enables Verdi message catcher.

Note:

It is mandatory to add the `+UVM_VERDI_TRACE=<Argument>` option.
However, you can add the other two options based on your requirements.
For example,

```
%> ./vsim +UVM_VERDI_TRACE +UVM_TR_RECORD +UVM_LOG_RECORD
```

Compiling and Simulating With Questa

Use the `Makefile.questa` file in the UVM-1.1d example directory with the following command:

```
%> make -f Makefile.questa all
```

If you use a 64-bit machine, add the `BITS` option:

```
%> make -f Makefile.questa all BITS=64
```

The content of the `Makefile.questa` file is:

```
UVM_HOME = ../../../../ USES_DPI = 1
include ../../../../Makefile.questa
all: run

#Add the +define+UVM_TR_RECORD compile option to compile the
Verdi recorder automatically.
comp: vlib
$(VLOG) +define+UVM_TR_RECORD +incdir+../sv \
ubus_tb_top.sv -l compile.log

#Add the +UVM_TR_RECORD run time option to dump transactions and the
+UVM_LOG_RECORD option to dump
messages.
run: comp
$(VSIM) +UVM_TESTNAME=test_read_modify_write \
+UVM_TR_RECORD +UVM_LOG_RECORD \
$(CHECK)
```

You can review other Questa options in the `Makefile.questa` file present at:

```
$VERDI_HOME/share/vmlib/uvm/uvm-1.1d/lib/examples
```

Compiling and Simulating With IUS

Use the `Makefile.ius` file in the UVM-1.1d example directory with the following command:

```
%> make -f Makefile.ius all
If you use a 64-bit machine, add the BITS option:
%> make -f Makefile.ius all BITS=64
The content of the Makefile.ius file is:
UVM_HOME = ../../../../
```

Chapter 3: Dumping Transactions into FSDB Automatically Recording UVM Transactions and Messages

```
include ../../../../Makefile.ius
all: comp run
comp:
$(IUS) -c +incdir+../sv \
+incdir+$(UVM_HOME)/src/verdi \
ubus_tb_top.sv \
$(UVM_HOME)/src/verdi/
uvm_custom_install_verdi_recorder.sv
run:
$(IUS) +incdir+../sv \
+incdir+$(UVM_HOME)/src/verdi \
ubus_tb_top.sv \
$(UVM_HOME)/src/verdi/
uvm_custom_install_verdi_recorder.sv \
+UVM_TESTNAME=test_read_modify_write
+UVM_TR_RECORD +UVM_VERDI_TRACE
$(CHECK)
```

You can review other IUS options in the `Makefile.ius` file present at: `$VERDI_HOME/share/uvmlib/uvm/uvm-1.1d/lib/examples`

Compiling and Simulating With UVM-1.2

For UVM 1.2 usage, change directory to the

`$VERDI_HOME/share/vmlib/uvm/uvm-1.2/lib/examples/integrated/ubus/examples` directory.

Compiling and Simulating with Questa

For Questa, you can compile and run the simulation using the following commands:

```
setenv UVM_HOME $VERDI_HOME/etc/uvm-1.2
```

```
make -f Makefile.questa all BITS=64
```

The content of `Makefile.questa` is as follows:

```
UVM_HOME          = ../../../../
USES_DPI = 1

include ../../../../Makefile.questa

all: run

comp: vlib
    $(VLOG) +incdir+../sv \
    +incdir+$(UVM_HOME)/src/verdi \

    $(UVM_HOME)/src/verdi/uvm_custom_install_verdi_recorder.sv \
    ubus_tb_top.sv -l compile.log
```

Chapter 3: Dumping Transactions into FSDB Automatically Recording UVM Transactions and Messages

```
run: comp
    $(VSIM) +UVM_TESTNAME=test_read_modify_write
    +UVM_VERDI_TRACE=UVM_AWARE+TLM +UVM_TR_RECORD +UVM_LOG_RECORD
    $(CHECK)

alt:
    qverilog \
    -timescale "1ns/1ns" \
    +acc=rmb \
    +incdir+$(UVM_HOME)/src+../sv \
    $(UVM_HOME)/src/uvm.sv \
    ubus_tb_top.sv \
    $(UVM_HOME)/src/dpi/uvm_dpi.cc \
    -R \
    +UVM_TESTNAME=test_read_modify_write \
    -c \
    -do "run -all; q" \
    -l questa.log
```

Compiling and Simulating With IUS

For IUS, you can compile and run the simulation using the following commands:

```
setenv UVM_HOME $VERDI_HOME/etc/uvm-1.2
```

```
make -f Makefile.ius BITS=64
```

The content of `Makefile.ius` is as follows:

```
UVM_HOME          = ../../../../..

include ../../../../Makefile.ius

all: comp run

comp:
    $(IUS) -uvmnocdnsextra -c +incdir+../sv \
    +incdir+$(UVM_HOME)/src/verdi \
    ubus_tb_top.sv \
    $(UVM_HOME)/src/verdi/uvm_custom_install_verdi_recorder.sv

run:
    $(IUS) -uvmnocdnsextra +incdir+../sv \
    +incdir+$(UVM_HOME)/src/verdi \
    ubus_tb_top.sv \
    $(UVM_HOME)/src/verdi/uvm_custom_install_verdi_recorder.sv
    \
    +UVM_TESTNAME=test_read_modify_write
    +UVM_VERDI_TRACE=UVM_AWARE +TLM +UVM_TR_RECORD +UVM_LOG_RECORD
    $(CHECK)
```

Recording Transactions for Non-Sequencer Components

If automatic recording for non-sequencer components (for example, drivers or monitors) is required, you need to modify the code of the components to explicitly call `void'(this.begin_tr())` and `void'(this.end_tr())` to trigger the recording hooks.

Note:

You need to make sure `begin_tr()` and `end_tr()` are called in pairs.

Following is an example showing how to explicitly call `void'(this.begin_tr())` and `void'(this.end_tr())` in a wishbone monitor:

```
class uart_base_monitor extends uvm_monitor;

    function void detect_bus_event(uart_bus_transaction bus_trans);
        m_bus_port.write(bus_trans);
        this.end_tr(bus_trans); // By Synopsys
    endfunction : detect_bus_event

endclass : uart_base_monitor

class uart_wishbone_monitor extends uart_base_monitor;

    //@ run related
    task run();
        //@ detect and put transaction
        fork
            monitor_addr();
            monitor_sel();
            monitor_dat_i();
            monitor_dat_o();
            monitor_we();
            monitor_stb();
            monitor_cyc();
            monitor_ack();
            monitor_int();
            monitor_baud();
        join_none
    endtask : run

    task monitor_addr();
        uart_bus_transaction    bus_trans;
        forever begin
            @m_bus_if.wb_addr_i;
            //@ bus event
            $cast(bus_trans , m_bus_trans.clone());
            void'(this.begin_tr(bus_trans)); // By Synopsys
            bus_trans.m_signal_type = uart_bus_transaction::WB_ADR_I;
            bus_trans.m_new_value = { 26'b0, m_bus_if.wb_addr_i };
            detect_bus_event(bus_trans);
        end
    endtask
endclass
```

```
endtask : monitor_addr

task monitor_sel();
    uart_bus_transaction    bus_trans;
    forever begin
        @m_bus_if.wb_sel_i;
        //@ bus event
        $cast(bus_trans , m_bus_trans.clone());
        void'(this.begin_tr(bus_trans)); // By Synopsys
        bus_trans.m_signal_type = uart_bus_transaction::WB_SEL_I;
        bus_trans.m_new_value = { 27'b0, m_bus_if.wb_sel_i };
        detect_bus_event(bus_trans);
    end
endtask : monitor_sel

endclass : uart_wishbone_monitor
```

Supporting VCS Three-Step Flow

The Transaction Debug feature of the Verdi platform also supports VCS three-step flow. For two-step compilations and elaboration, the `-ntb_opts` compile option is needed for the corresponding libraries. For example, compile UVM-1.0 with the *vlogan* utility and elaborate the design using the following commands:

```
%> vlogan -timescale=1ns/1ns -ntb_opts uvm-1.0
%> vlogan -ntb_opts uvm-1.0 +incdir+../sv ubus_tb_top.sv
%> vcs -debug_access+all -ntb_opts uvm-1.0 ubus_tb_top
    -P $(VERDI_HOME)/share/PLI/VCS/LINUX/novaS.tab
    $(VERDI_HOME)/share/PLI/VCS/LINUX/pli.a
```

For UVM-1.1d, compile with the *vlogan* utility and elaborate the design using the following commands:

```
%> vlogan -sverilog -timescale=1ns/1ns -ntb_opts uvm
%> vlogan -sverilog -ntb_opts uvm +incdir+../sv ubus_tb_top.sv
%> vcs -debug_access+all -ntb_opts uvm ubus_tb_top
```

See `README.txt` and `Makefile.vcs` files in the following directory for the 1.1d example details:

```
$(VERDI_HOME)/share/vmlib/uvm/uvm/lib/examples/integrated/ubus_evdp_ntb_u
um/examples
```

For UVM-1.2, compile with the *vlogan* utility and elaborate the design using the following commands:

```
%>vlogan -sverilog -timescale=1ns/1ns -ntb_opts uvm-1.2
%>vlogan -sverilog -ntb_opts uvm-1.2 +incdir+../sv ubus_tb_top.sv
%>vcs -debug_access+all -ntb_opts uvm-1.2 ubus_tb_top
```

Transaction begin_time

You are allowed to specify the transaction `begin_time`. If the `begin_time` is not specified, simulator will set the `begin_time` equal to current simulation time while creating the transaction..

If the VCS compile option `+define+UVM_VERDI_FORCE_BEGIN_TR_TIME` is added, simulator will ignore user specified `begin_time` and will use current simulation time as the `begin_time` of the transaction while creating all transactions.

It is suggested to use Verdi T-2022.06, 21.09-SP2-2 or later versions, for the feature of Transaction `begin_time`.

If you are using Verdi T-21.09-SP2-1 version, the default behavior is different and is described as follows:

The `begin_tr ()` function is used to indicate that the transaction is started. When Verdi recorder is enabled in the UVM, the `begin_time` argument of `begin_tr ()` is forced to use the simulator time.

The transaction `begin_time` specified by the user is accepted only when the following two conditions are met:

1. Add compiler directive, `+define+UVM_VERDI_NOT_FORCE_BEGIN_TR_TIME`
2. When you do not use the UVM Verdi recorder.

Otherwise, you will be forced to use the current simulator time as the `begin_time` for the transaction.

Note:

If the Verdi recorder is used and enabled, the specified `begin_time` in the `begin_tr()` function call will not work.

If you are using Verdi S-2021.09-SP2 and before versions, the default behavior is different and is described as follows:

- UVM 1.1d or IEEE libraries: If `begin_time` is specified, the specified `begin_time` is used. Otherwise, current simulation time is used.
- UVM 1.2 library: In this case, `begin_time` is forced to use current simulation time.

Dumping Data Using SystemVerilog Wrapper Functions (API)

For advanced users (that is, users that require recording different or more data than is provided with the Verdi-modified libraries), a PLI task-based API (application programming interface) with a SystemVerilog wrapper is provided to dump transactions to an FSDB

file through the FSDB dumper. In addition, a C5 function-based API is provided to dump transactions to an FSDB file through the FSDB writer. SystemC6 and SCV function-based APIs (SC-API and SCV-API) are also provided to dump transactions to an FSDB file.

Verdi also provides corresponding C and Tcl APIs to dump the transaction data into FSDB. For details, see *VC APPS Native Programming Interface (NPI) User Guide*.

The following sections describe the PLI task-based API usage and provide examples for reference:

- [Stream Tasks](#)
- [Transaction Tasks](#)
- [Transaction/Relation Tasks](#)
- [Miscellaneous Tasks](#)
- [Simple API Example](#)
- [Rules](#)
- [Coding Style for API](#)

You can configure the dumper to dump all the transaction data into a specified FSDB that is separate from the signal based FSDB.

The FSDB dumper provides the following two ways to specify the transaction FSDB:

- Set the environment variable: `FSDB_TRANS_FILE`. For example, the command `setenv FSDB_TRANS_FILE xxx` creates `xxx.fsdb`.
- Specify the option in the simulator command line: `+fsdb_trans_file+`. For example, the command `simv +fsdb_trans_file+yyy` creates `yyy.fsdb`.

You can use this environment variable or command line option when you want to dump transaction and signal data into separate FSDB files.

Note:

- Transaction based FDSB does not support `$fsdbAutoSwitchDumpFile`.
- You need to use the Transaction Debug features described in the subsequent chapters to recognize the transactions and messages in the `novas.fsdb` transaction-based FSDB file.

The PLI system tasks include the following categories: Stream, Transaction, Transaction/Relation and miscellaneous tasks (refer to [Stream Tasks](#), [Transaction Tasks](#), [Transaction/Relation Tasks](#), and [Miscellaneous Tasks](#) sections for details). To get better performance in the Verdi platform with version 2014.12, refer to the Coding Style section for the recommended coding style.

Note:

The data type of stream id and transaction id are both `longint` types.

Stream Tasks

The following tables describe the Stream Task API usage and provide examples for reference.

stream_id \$fsdbTrans_create_stream_begin	
Syntax	<code>\$fsdbTrans_create_stream_begin(string stream_fullname[, string option]);</code>
Description	Creates a stream and returns <code>stream_id</code> with the <code>longint</code> type. If the returned value is 0, it indicates that something is wrong and the stream cannot be created.
Arguments	
stream_fullname	Specifies the full hierarchy name of a stream. Use the <code>.</code> character as a delimiter. For example, <code>top.sub.s</code> is placed into the <code>top/sub</code> hierarchy and named <code>s</code> .
option	<code>["+fsdbfile+filename"]</code> Specifies the output FSDB file name. Note: If this option and the <code>+fsdbfile+filename</code> runtime option are used simultaneously, the FSDB file is overwritten by the file specified in the <code>+fsdbfile+filename</code> runtime option.

void \$fsdbTrans_define_attribute	
Syntax	<code>\$fsdbTrans_define_attribute(stream_id sid, attribute_var[, string option]);</code>
Description	Use this task to define the dense attribute. This task must be called after the <code>\$fsdbTrans_create_stream_begin</code> task and before the <code>\$fsdbTrans_create_stream_end</code> task, and can be called multiple times.
Arguments	
sid	Specifies the stream id it belongs to.

attribute_var	The attribute template. The name and data type of <code>attribute_var</code> is used as the name and the data type of the predefined attribute. For example, if <code>addr</code> is declared as <code>int addr</code> , and is used as <code>attribute_var</code> , the attribute name would be <code>addr</code> and the data type would be <code>int</code> .
option	<code>["+name+attrname"]</code> Specifies a new attribute name to replace the name of <code>attribute_var</code> as the attribute name.
option	<code>["+radix+bin"]</code> <code>["+radix+oct"]</code> <code>["+radix+dec"]</code> <code>["+radix+hex"]</code> Specifies the default radix.
option	<code>["+default_value"]</code> Specifies a default value to store for the default attribute.

void \$fsdbTrans_add_stream_attribute	
Syntax	<code>\$fsdbTrans_add_stream_attribute(stream_id sid, attribute_var[, string option]);</code>
Description	Use this task to add attribute for the specified stream. This task must be called after the <code>\$fsdbTrans_create_stream_begin</code> task and before the <code>\$fsdbTrans_create_stream_end</code> task, and can be called multiple times.
Arguments	
sid	Specifies the stream ID it belongs to.
attribute_var	The attribute template. Name, data type, and value of <code>attribute_var</code> is used as the name, data type, and value of the added attribute. For example, if <code>addr</code> is declared as <code>int addr = 'h1234</code> , and is used as <code>attribute_var</code> , the attribute name would be <code>addr</code> , the data type would be "int", and the attribute value would be <code>'h1234</code> .
option	<code>["+name+attrname"]</code> Specifies a new attribute name to replace the name of <code>attribute_var</code> as the attribute name.

void \$fsdbTrans_create_stream_end	
Syntax	<code>\$fsdbTrans_create_stream_end(stream_id sid);</code>
Description	Ends the stream creation. Only after this task is called, the stream is completely created and ready to add transactions.
Arguments	

sid	Specifies the stream ID.
------------	--------------------------

Transaction Tasks

The following tables describe the Transaction Task API usage and provide examples for reference.

trans_id \$fsdbTrans_begin	
Syntax	<code>\$fsdbTrans_begin(stream_id sid, string trans_type);</code>
Description	Creates a transaction and returns a transaction ID with <code>longint</code> type. Use <code>transaction_id</code> to create transaction details or add relationship between transactions. If the returned value is 0, it indicates that something is wrong and the transaction cannot be created.
Arguments	
sid	Specifies the stream ID it belongs to.
trans_type	Specifies the transaction type. The Transaction Debug windows use different representations (GUI symbols) for each type. <code>trans_type := ["+type+message"] ["+type+action"] ["+type+group"] ["+type+transaction"]</code>

void \$fsdbTrans_set_label	
Syntax	<code>\$fsdbTrans_set_label(trans_id tid, string label);</code>
Description	Sets the label name of a transaction. This task is only applicable before invoking the <code>\$fsdbTrans_end</code> task.
Arguments	
tid	Specifies the transaction.
label	Specifies a new label.

void \$fsdbTrans_add_attribute	
Syntax	<code>\$fsdbTrans_add_attribute(trans_id tid, attribute_var[, string option]);</code>

Description	Adds an attribute into the <code>tid</code> transaction. If the attribute of the same name and data type has been predefined when creating the streams, a dense attribute is added. Otherwise, a sparse attribute is added. This task is only applicable before invoking the <code>\$fsdbTrans_end</code> task.
Arguments	
tid	Specifies the transaction.
attribute_var	The attribute template. Name, data type, and value of <code>attribute_var</code> is used as name, data type, and value of the added attribute. For example, if addr is declared as <code>int addr = 'h1234</code> , and is used as <code>attribute_var</code> , the attribute name would be <code>addr</code> , the data type would be <code>int</code> , and the attribute value would be <code>'h1234</code> .
option	<code>["+name+attrname"]</code> Specifies a new attribute name.
option	<code>["+radix+bin"]</code> <code>["+radix+oct"]</code> <code>["+radix+dec"]</code> <code>["+radix+hex"]</code> Specifies the default radix.
option	<code>["+expect", expect_value]</code> Specifies an expected value for this attribute. The type of the expected value must be the same as the type of <code>attribute_var</code> . Otherwise, a warning is reported.

void \$fsdbTrans_add_tag	
Syntax	<code>\$fsdbTrans_add_tag(trans_id tid, string tag);</code>
Description	Add a tag to a transaction. Multiple tags can be added to a transaction. This task is only applicable before invoking the <code>\$fsdbTrans_end</code> task.
Arguments	
tid	Specifies the transaction ID.
tag	Specifies a new tag

void \$fsdbTrans_end	
Syntax	<code>\$fsdbTrans_end(trans_id tid);</code>
Description	Ends a transaction.
Arguments	

tid	Specifies the transaction.
------------	----------------------------

Transaction/Relation Tasks

The following tables describe the Transaction/Relation Task API usage and provide examples for reference.

void \$fsdbTrans_add_relation	
Syntax	<code>\$fsdbTrans_add_relation(string rel_name, trans_id master, trans_id slave);</code>
Description	Adds the relationship between two transactions. This task can be called either before or after the transaction is ended.
Arguments	
rel_name	Specifies the relationship name. The Transaction Debug windows use different representations for each predefined relation. rel_name := ["parent_child"] ["belong_to"] ["succ_pred"] ["annotate_to"] or user-defined
master, slave	Based on the relation being added, master and slave should be as mentioned in the table below.

rel_name	master	slave
parent_child	parent	child
belong_to	group	member
succ_pred	predecessor	successor
annotate_to	annotator	annotee

Miscellaneous Tasks

The following tables describe the Miscellaneous Task API usage and provide examples for reference.

void \$fsdbTrans_add_scope_attribute	
Syntax	<code>\$fsdbTrans_add_scope_attribute(string scope_fullname, attribute_var[, string option]);</code>

Description	Adds an attribute for a scope of transactions. If the specified scope does not exist, the scope is created.
Arguments	
scope_fullname	Specifies the scope with the full hierarchy name. Use the . character as the delimiter.
attribute_var	Specifies the attribute.
option	["+fsdbfile+filename"] Specifies a file name.
option	["+name+attrname"] Specifies a new attribute name.

error_state \$fsdbTrans_get_error_code	
Syntax	<code>\$fsdbTrans_get_error_code();</code>
Description	Returns the error (execution) state of the task with the prefix \$fsdbTrans_ occurring right before it.
Return Value	
error_state	The data type of return value is an integer type: 0: when the task is successful 1: when the task completes with some warning messages 2: when the task fails with some errors

Simple API Example

This section discusses a simple example to create a stream containing two transactions with a parent-child relationship. The transaction `tid1` has label *Request*. The transaction `tid2` has label *Response*. The relationship between `tid1` and `tid2` is parent-child.

```
module top;
  initial begin
    static longint unsigned sid;
    static longint unsigned tid1;
    static longint unsigned tid2;
    static int severity = 0;
    static int verbosity;
    int err_state;
    static bit [127:0] addr = 100, data = 400;

    sid = $fsdbTrans_create_stream_begin ("top");
```

```
$display("sid is %d",sid);
$fsdbTrans_create_stream_end (sid);

severity = 2; verbosity = 2;

tid1 = $fsdbTrans_begin (sid, "+type+transaction");
      $fsdbTrans_set_label (tid1, "Request");
      $fsdbTrans_add_attribute (tid1, severity);
      $fsdbTrans_add_attribute (tid1, verbosity);
      $fsdbTrans_add_attribute (tid1, addr,
                                "+radix+bin");

      $fsdbTrans_add_tag (tid1, "RED");
      $fsdbTrans_end (tid1);

tid2 = $fsdbTrans_begin (sid, "+type+transaction");
      $fsdbTrans_set_label (tid2, "Response");
      $fsdbTrans_add_attribute (tid2, severity);
      $fsdbTrans_add_attribute (tid2, verbosity);
      $fsdbTrans_add_attribute (tid2, data,
                                "+radix+bin");

      $fsdbTrans_add_tag (tid2, "GREEN");
      //get the execution status of $fsdbTrans_add_tag task to
err_state
      err_state = $fsdbTrans_get_error_code();
      $fsdbTrans_end (tid2);
      $fsdbTrans_add_relation ("parent_child", tid1, tid2);
end
endmodule
```

Rules

The PLI-based API requires that some rules be followed. The rules are described below, along with the action taken when a rule is violated:

1. The stream name should be unique for each stream. If users pass an existing stream name, the PLI does not create it and reports error messages.
2. Stream attributes should be generated before the end of the stream. Otherwise, the PLI skips the recording and reports error messages.
3. Each stream attribute should have a unique name. Otherwise, the PLI skips the recording and reports error messages. The attribute name should be unique in both dense and sparse attributes.
4. The character “.” is regarded as a delimiter in the stream name.
5. User should avoid using a space, the backslash (/) character, and double quotations (") in the stream name. The PLI shows a warning message if this rule is violated.
6. Transaction should be generated after the end of the stream. Otherwise, the PLI skips the recording and reports error messages.

7. A label can be set more than one time. However, the previous setting is replaced with the current one.
8. Each attribute and tag should have a unique name. Otherwise, the PLI skips the recording and reports error messages.
9. The attribute and tag should be generated before the end of the transaction. Otherwise, the PLI skips the recording and reports error messages.
10. The transaction IDs between the relationships should be different. Otherwise, the PLI reports warning messages.
11. Avoid generating loop parent-child relationships and multiple parents.
12. Group members cannot be created earlier than the group.

Coding Style for API

The following coding style is suggested for the arguments of the API:

- Whenever possible, pass constants instead of variables as arguments. For example:

Recommended:

```
$fsdbTrans_add_attribute(st_handle, st_value, attr_name, "+radix+dec",  
    "+numbit+32");
```

Not Recommended:

```
string radix_str = "+radix+dec";  
string numbit_str = "+numbit+32";  
$fsdbTrans_add_attribute(st_handle, st_value, attr_name, radix_str,  
    numbit_str);
```

- If constants cannot be passed, use static variables whenever possible. For example:

Recommended:

```
function void set_message_attribute_int(int handle,  
    string attrName, int value);  
  
    static string attr_name = "";  
    static int st_handle=0, st_value = 0;  
    $sformat(attr_name, "+name+%s", attrName);  
    st_handle = handle;  
    st_value = value;  
  
    $fsdbTrans_add_attribute(st_handle, st_value, attr_name,  
        "+radix+dec", "+numbit+32");  
endfunction
```

Not Recommended:

```
function void set_message_attribute_int(int handle,
    string attrName, int value);

    string attr_name = "";
    $sformat(attr_name, "+name+%s", attrName);

    $fsdbTrans_add_attribute(handle, value, attr_name,
        "+radix+dec", "+numbit+32");
endfunction
```

Automatically Dumping Transactions From Synopsys VIP

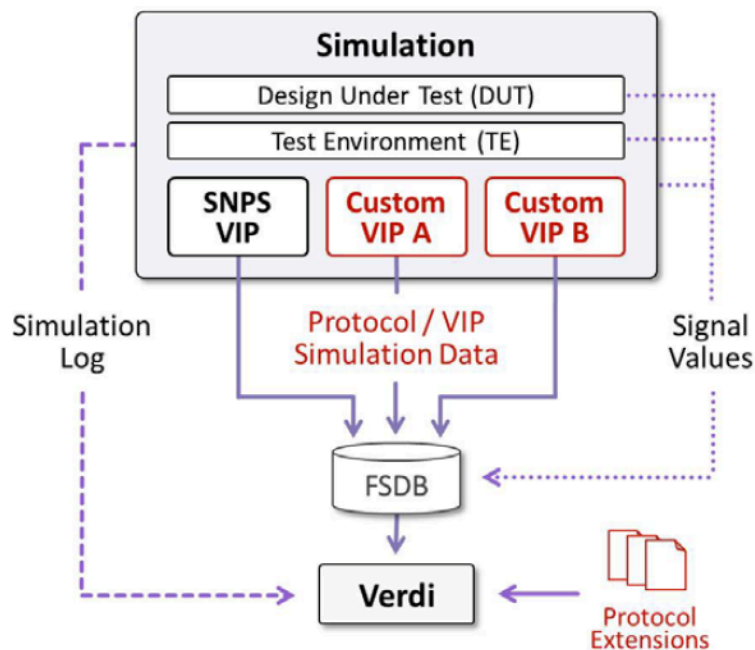
For details on how to dump protocol transaction data from Synopsys VIPs, refer to the respective Synopsys VIP User Guides.

Using VC APPS Protocol Analyzer to Dump Transactions From Custom VIP

The *VC APPS Protocol Analyzer* provides an interface to integrate third-party VIPs into the Verdi environment. The objective of this feature is to standardize the Verdi Protocol Analyzer as the common debug platform for Synopsys VIPs and third-party VIPs. This interface supports the components that can generate both simple and complex (hierarchical) protocol-style transactions.

The Verdi Protocol Analyzer utilizes protocol-specific data that is stored in a common FSDB database along with the protocol knowledge provided by the Synopsys VIP to create a high level, interactive display of the protocol-related comprehensive analysis and debug capabilities. Similarly, for integrating third-party VIPs into the Verdi environment, you must capture the protocol-specific activity and store that data in an FSDB. You must have access to the VIP source code to capture the protocol-specific activities. In addition, you must specify the protocol information that is required to facilitate the protocol-specific debug features available in the Verdi environment. The protocol specification defines the activity that occurs in each layer along with its associated data and the manner in which data communicates between the layers.

Figure 1 Verdi Protocol Analyzer Utilizes Protocol Data Stored in FSDB Database



For additional information or documentation on *VC APPS Protocol Analyzer*, see *VC Apps Protocol Analyzer User Guide*.

Verdi also provides corresponding C and Tcl APIs to dump the protocol related transaction data from custom VIP into FSDB. For details, see *VC APPS Native Programming Interface (NPI) User Guide*.

4

Using Transaction and Protocol Analyzer

Transaction and Protocol Analyzer allows you to view different types of simulation data including VIP transactions, protocol transactions, and UVM transactions or messages in a single application window, which helps the users to debug testbench simulation data more efficiently.

The *Transaction and Protocol Analyzer* window provides a software-oriented waveform-like view (compared to the hardware hierarchical view in the *nTrace* window) for testbench debugging. The waveform view supports new transaction types and the virtual streams that are recognized transactions from different streams. The *Transaction and Protocol Analyzer* window can be used to debug and analyze transactions and their temporal activity by the time and stream axes.

The *Unified Transaction Browser Mode* is applicable for using *Verdi Transaction and Protocol Analyzer*, which helps you to view different types of simulation data including VIP transactions, protocol transactions, and UVM transactions or messages in a single application window. The following sections describe the usage of *Transaction and Protocol Analyzer* in *Unified Transaction Browser Mode*:

- [Invoking Transaction Debug Model](#)
- [Invoking Transaction and Protocol Analyzer](#)
- [Importing an FSDB to the Transaction and Protocol Analyzer Window](#)
- [Transaction and Protocol Analyzer Window Overview](#)
- [Preference Options](#)
- [Transaction and Protocol Analyzer Operations](#)
- [Interlinking Between Transactions and Signals](#)
- [Connecting nWave and Transaction and Protocol Analyzer](#)

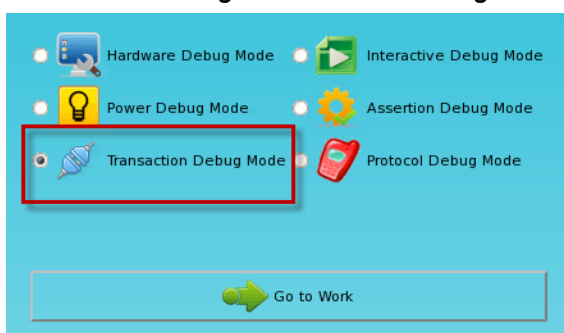
Invoking Transaction Debug Model

The recommended flow to browse the abstract data with the Transaction Debug solution is:

1. Start the Verdi platform with the whole system, including the design and testbench.

```
%> verdi <options> &
```
2. Click the **Work Modes** icon in the *Welcome Page*, select the **Transaction Debug Mode** option, and then press the **Go to Work** button.

Figure 2 Selecting Transaction Debug Work Mode



Or use the `-workMode` command line option directly to invoke Verdi in *Transaction Debug* mode:

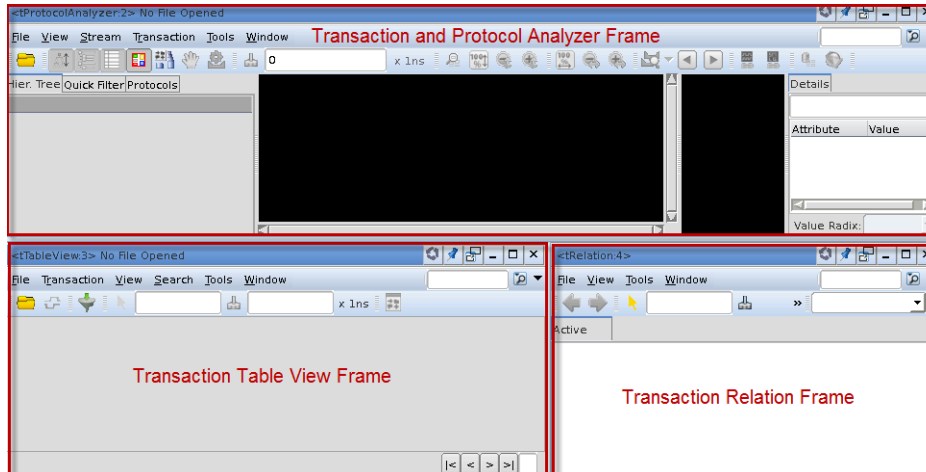
```
%> verdi -workMode transactionDebug <options> &
```

The *Transaction Debug* mode opens the original *nTrace* windows and all the necessary Transaction Debug windows for you to debug the Verilog/SystemVerilog system at a higher level of abstraction. Alternatively, each of the *Transaction Debug* windows can be manually opened using the **Tools** menu in the *nTrace* main window.

The Transaction Debug panes are displayed in the main window by default. As with other panes in the Verdi platform, the *Transaction and Protocol Analyzer*, the *Transaction Table View*, and the *Transaction Relation Navigator* panes can also be docked or undocked, or

dragged and dropped to other parts of the main window. The main window is shown in the following figure:

Figure 3 Transaction Debug Mode



Invoking Transaction and Protocol Analyzer

You can invoke *Transaction and Protocol Analyzer* using the **Tools -> Transaction Debug -> Transaction and Protocol Analyzer** menu command in *nTrace*.

You can also invoke Transaction and Protocol Analyzer using the following menu commands:

- **Tools -> Transaction Debug -> Transaction and Protocol Analyzer** command in the Verdi main window.
- **Tools -> New Transaction and Protocol Analyzer Window** command in the *Transaction Table View* window.
- **Tools -> New Transaction and Protocol Analyzer Window** command in the *tRelation* Window.
- **Window -> Connect Transaction and Protocol Analyzer and nWave** command in the nWave Window.
 - The  icon opens Transaction and Protocol Analyzer along with nWave.

Note:

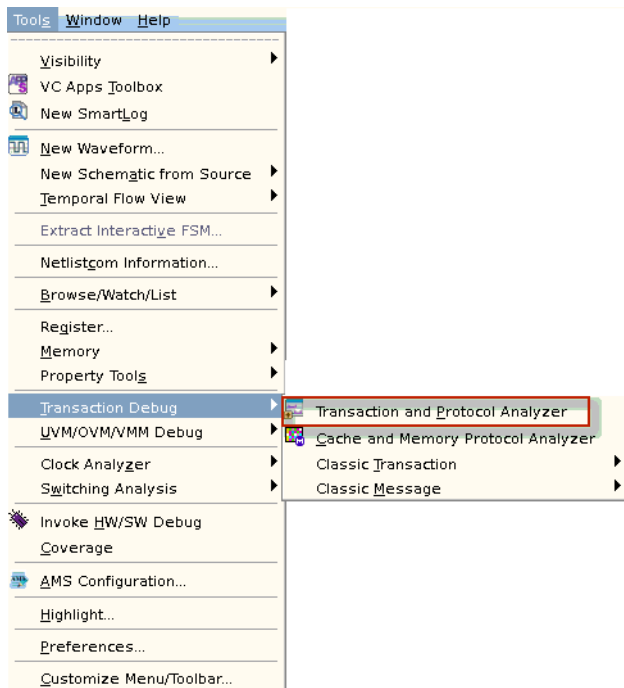
Verdi Transaction and Protocol Analyzer also provides a different mode (the vertical view) to display transaction data dumped from the VIP - the **VIP**

Protocol Analyzer mode. For details, see [Appendix A: Using Transaction and Protocol Analyzer \(Protocol Debug Mode\)](#).

Importing an FSDB to the Transaction and Protocol Analyzer Window

To open the Transaction and Protocol Analyzer window, you also can select the **Tools -> Transaction Debug -> Transaction and Protocol Analyzer** command in the *nTrace* window as shown in the following figure after invoking the Verdi platform.

Figure 4 Invoking Transaction and Protocol Analyzer



Click the  button directly or select the **File -> Open** command to import the FSDB file. One Transaction and Protocol Analyzer window can only open one FSDB file.

Transaction and Protocol Analyzer Window Overview

The *Transaction and Protocol Analyzer* window can display all the supported transaction data types (For details, see Chapter [Using Data Types in Transaction Debug](#)).

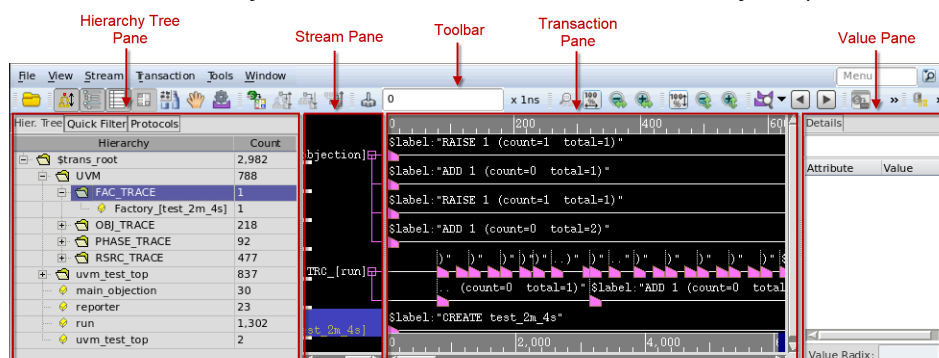
Chapter 4: Using Transaction and Protocol Analyzer Transaction and Protocol Analyzer Window Overview

The default layout of the *Transaction and Protocol Analyzer* window is as follows:

- The left region contains the **Hierarchy Tree** pane
- The middle region contains **Stream** pane and **Transaction** pane
- The right region contains the **Value** pane

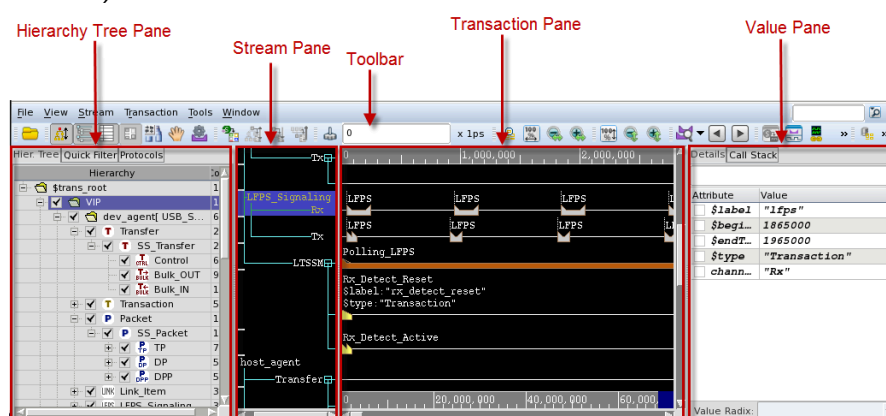
If the FSDB file contains UVM transactions related data, the default layout for the *Transaction and Protocol Analyzer* window looks as follows:

Figure 5 Default Layout of Transaction and Protocol Analyzer (UVM Transactions)

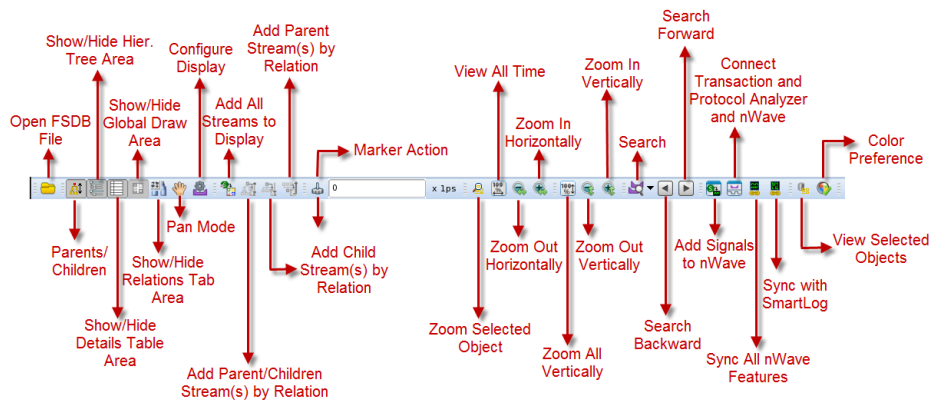


If the FSDB file contains VIP transactions related data, the default layout for the *Transaction and Protocol Analyzer* window looks as follows:

Figure 6 Default Layout of the Transaction and Protocol Analyzer Window (VIP Transactions)



The following figure illustrates the Transaction and Protocol Analyzer toolbar:



Hierarchy Tree Pane

The Hierarchy Tree pane contains three tabs: *Hier. Tree*, *Quick Filter*, and *Protocols* tab. You can switch between the tabs by clicking the tab labels. After importing an FSDB file, if the *Hierarchy Tree* pane is not visible, invoke the **Show/Hide Hier. Tree Area** command by clicking the  button to view the *Hierarchy Tree* pane.

Hier. Tree Tab Behavior for Non-VIP (UVM) Transactions

If the FSDB file contains Non-VIP (UVM) transactions related data, the **Hier. Tree** tab in the *Hierarchy Tree* pane shows the hierarchy tree as illustrated in the following figure:

Hier. Tree Quick Filter Protocols	
Hierarchy	Count
\$trans_root	2,982
UVM	788
FAC_TRACE	1
Factory [test_2m_4s]	1
OBJ_TRACE	218
PHASE_TRACE	92
RSRC_TRACE	477
uvm_test_top	837
ubus_example_tb0	837
main_objection	30
reporter	23
run	1,302
uvm_test_top	2

Adding Non-VIP Streams and Scopes to the Transaction Pane

To add non-VIP (UVM) streams to the *Transaction* pane, double-click the required stream in the *Hier. Tree* tab, or click the **Add Stream to Display** right-click option (for streams), or click the **Add All Streams to Display**  icon in the toolbar.

To add non-VIP (UVM) scopes (with all the streams under the scope) to the *Transaction* pane, click the **Add All Streams To Display** right-click option.

Newly-added streams are displayed at the bottom of the *Transaction* pane based on the insertion order.

Adding Duplicate Streams

To add duplicate streams into the *Transaction* pane, double click the required non-VIP stream or click **Add All Streams To Display** or **Add Stream to Display** right-click options. Duplicate streams are added at the end of the *Transaction* pane based on the insertion order.

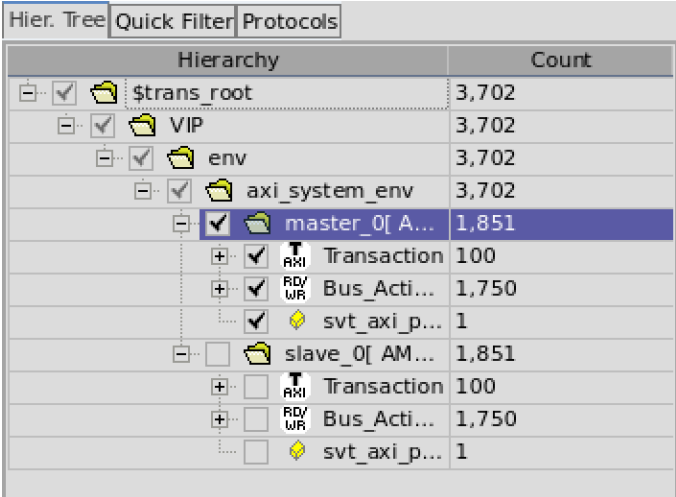
Drag and Drop

Transaction and Protocol Analyzer provides the capability to drag and drop a stream from the project *Hier. Tree* to other widgets or from other widgets to the project *Hier. Tree*.

Dragging and dropping duplicate streams is not supported and dragging and dropping a stream/scope into the same hierarchy tree in the same frame is not supported.

Hier. Tree Tab Behavior for VIP Transactions

If the FSDB file contains VIP transactions related data, the **Hier. Tree** tab in the *Hierarchy Tree* pane looks as illustrated in the following figure:



Hierarchy		Count
☒	\$trans_root	3,702
☒	VIP	3,702
☒	env	3,702
☒	axi_system_env	3,702
☒	master_0[A...	1,851
☒	Transaction	100
☒	Bus_Acti...	1,750
☒	svt_axi_p...	1
☐	slave_0[AM...	1,851
☐	Transaction	100
☐	Bus_Acti...	1,750
☐	svt_axi_p...	1

Adding VIP Streams and Scopes to the Transaction Pane

The *Hier. Tree* pane in *Transaction and Protocol Analyzer* displays a check box next to the VIP streams and scopes. If the loaded FSDB file has only VIP streams, then the \$trans_root top scope would also have a check box.

To add the VIP streams and scopes to the *Transaction* pane, select the check box respective to the VIP stream and scope in the *Hier. Tree* pane.

To remove the VIP streams and scopes from the *Transaction* pane, uncheck the check box respective to the VIP stream and scope in the *Hier. Tree* pane.

Alternatively, you can also select the **Add Stream to Display** or **Add All Streams to Display** right-click options, or click the **Add All Streams to Display**  icon in the toolbar.

Adding Duplicate Streams

Adding duplicate streams is not supported for VIP streams.

Drag and Drop

Transaction and Protocol Analyzer provides the capability to drag and drop a stream from the project *Hier. Tree* to other widgets or from other widgets to the project *Hier. Tree*.

To add a VIP stream/scope to the *Transaction* pane, drag and drop the stream/scope from the project *Hier. Tree* to the *Transaction* pane. The check box next to the stream/scope in the *Hier. Tree* tab changes to a checked state.

Deleting the Selected Transaction Type

To remove the unwanted transactions of a similar type from the *Transaction* pane, right-click a transaction and click **Remove all 'xxx' objects** right-click option ('xxx' signifies the display text of the object). This option removes all the objects of the selected stream and its associated streams.

Note:

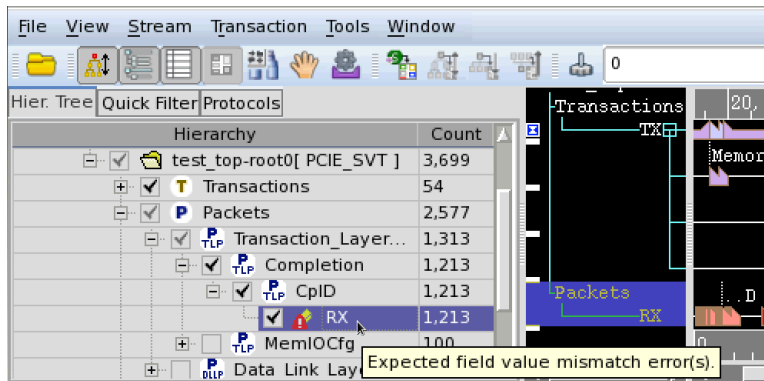
This feature is applicable only for the selected object of the type VIP stream.

Error Indications in Hier. Tree

Transaction and Protocol Analyzer marks erroneous transactions/messages in the *Hier Tree* to enable you to view only those transactions /messages that are causing errors. Transaction and Protocol Analyzer displays the streams having error(s) with  red-colored

exclamatory icon on top of the original icon in the *Hier. Tree*. The ToolTip also displays the error type for the selected scope/stream. This is illustrated in the following figure:

Figure 7 Error Indications in Hier. Tree



If a stream has an error, then the stream and its parent scope gets marked with an exclamatory error icon. This helps you to identify a stream when the corresponding stream or scope is in a collapsed state.

The transaction display also highlights the transactions/messages having error with the red-color  icon.

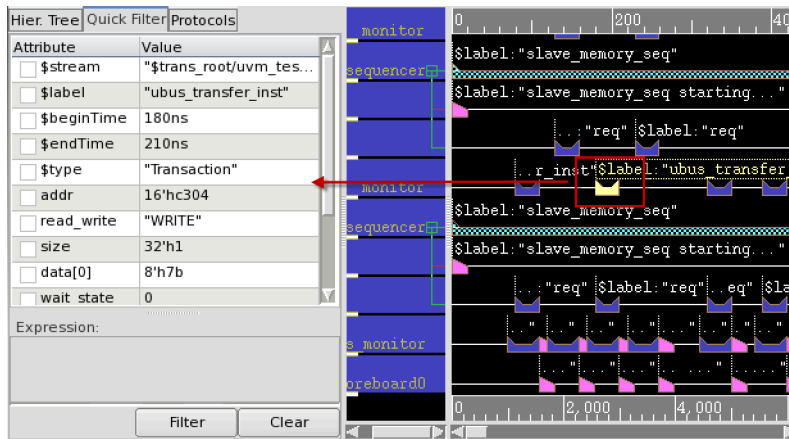
You can also select the `has_errors()` function as the search expression in the *Search* dialog in *Transaction and Protocol Analyzer* to search all the transactions having expected field value mismatch errors or error messages and display those in the *Search Results* view.

Quick Filter Tab

The *Quick Filter* tab of the *Hierarchy Tree* pane in *Transaction and Protocol Analyzer* provides filtering functionality to filter unnecessary or uninteresting transactions/streams.

On selecting a transaction (as shown in the following figure, the selected one is a transaction with *Transaction* type), the attributes of the transaction are displayed on the *Quick Filter* tab. You can select multiple streams first and choose these attributes (of previously selected transactions) as the filtering input applied to the selected streams. The *Value* column is editable as needed.

Figure 8 Selected Transaction Attributes



For example, for observing the sequence. [Exp1.Exp1] stream that starts at 60ns and ends at 110ns, select the sequence. [Exp1.Exp1] stream, click the \$begin_time and \$end_time fields: (see [Figure: Specify the Filter Condition](#)), the \$beginTime == 60ns && \$endTime == 110ns expression is automatically displayed in the **Expression** field, and then click the **Filter** button.

Figure 9 Specify the Filter Condition

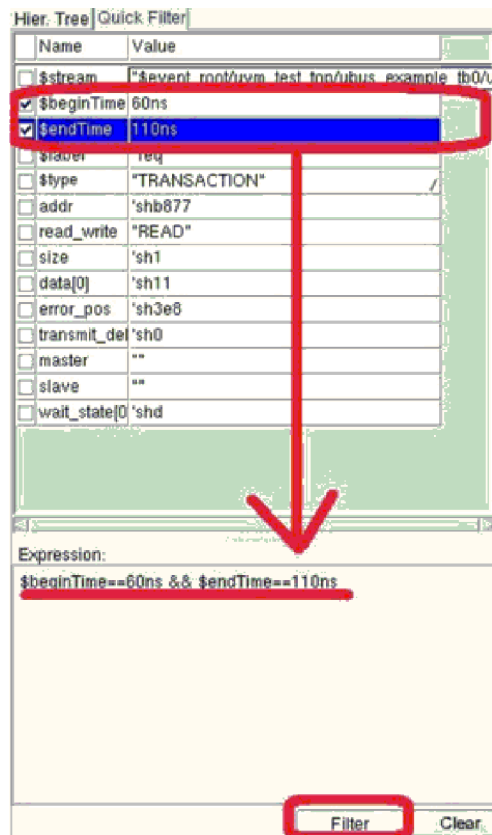


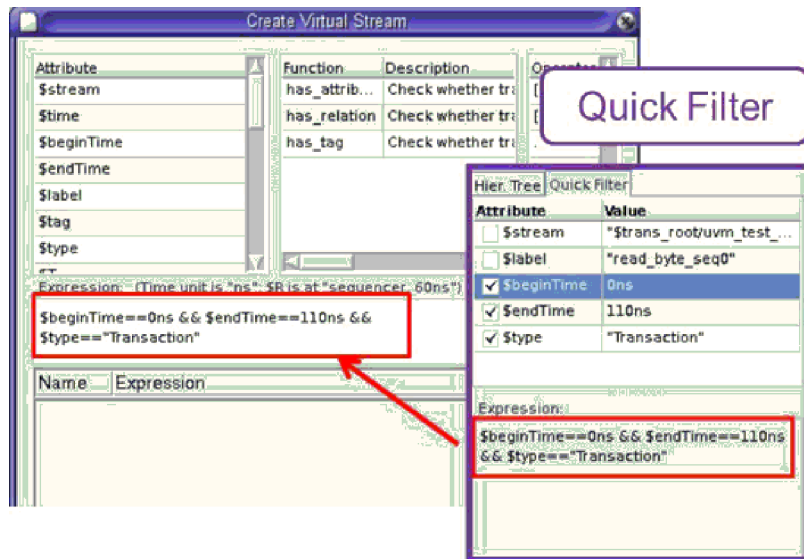
Figure: Filter Stream with Filtering Condition shows the results of the filtered stream that now only displays the transactions that start at 60ns and ends at 110ns.

Figure 10 Filter Stream with Filtering Condition



The **Expression** field shows the expression of the selected filter condition that you can copy and add to search dialog and filter/colorize dialog. You can also apply the copied condition to the process of creating a virtual stream as shown in [Figure: Apply the Condition to Create Virtual Stream](#). See the [Creating Virtual Streams](#) section for details on creating virtual streams.

Figure 11 Apply the Condition to Create Virtual Stream



Save Streams

The configured streams can be saved using the **File -> Save Stream** command with the *file_name.rc* naming. The content of the saved file contains the data path of the FSDB file, zoom ratio, time range, and stream/group list, but it does not include virtual streams and runtime signals. You can restore the save file using the **File -> Restore Stream** command without opening the FSDB file first.

Display Class Definition

If a transaction has the *class_name* attribute, you can use the **Display Class Definition** command in the right-click command menu to trace the class definition in the source code frame of the transaction.

- This command only works in the following conditions: Design is imported into the Verdi platform, and
- Source code frame is active to see the results of invoking this command.

Protocols Tab

The *Protocols* tab displays all the protocol definitions that have been read by the *Transaction and Protocol Analyzer*. This tree is constructed by reading the *protocol*

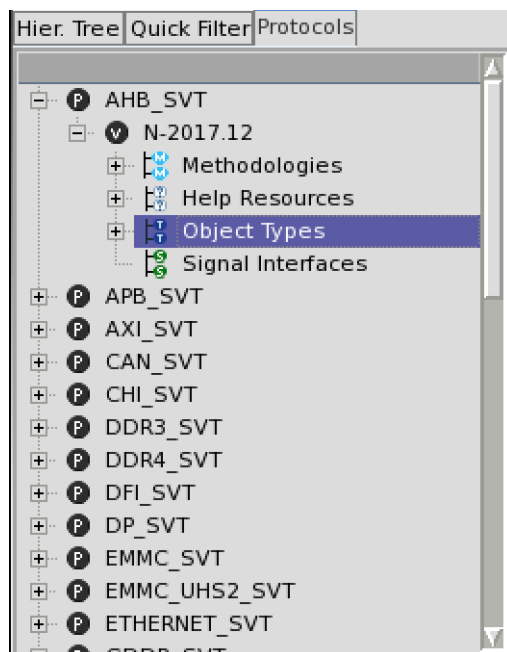
definitions files from VIP installed in the path specified with the `DESIGNWARE_HOME` environment variable and *protocol definition* files for custom or proprietary protocols. The location of proprietary protocol definitions is specified using the `PA_CUSTOM_VIP_PATH` environment variable.

Note:

The *Protocols* tab shows the relevant data only if the protocol definition files are available for Synopsys VIPs or custom VIPs.

The following figure shows the *Protocols* tab:

Figure 12 Protocols Tab

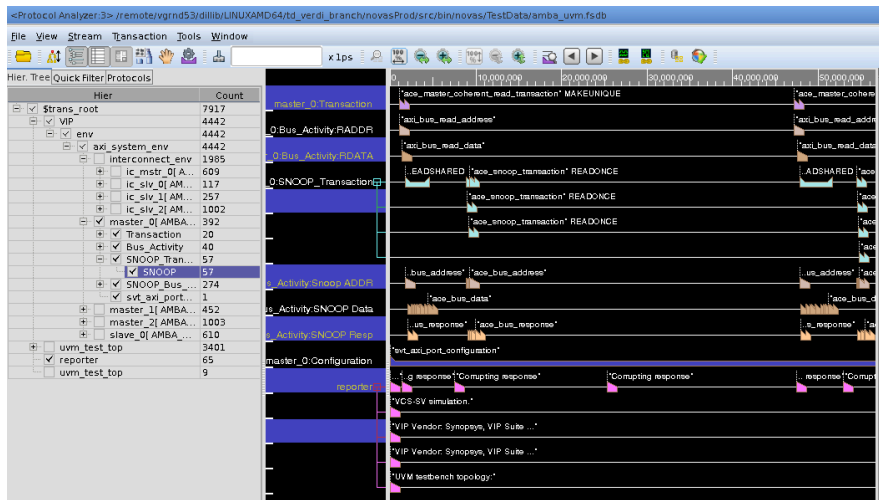


Stream Pane

Transaction and Protocol Analyzer allows you to select any of the transaction/message rows in the *Stream* pane, as illustrated in the [Figure: Selecting Rows in Stream Display View](#).

You can hide a stream by using the **Stream -> Hide** command and recover a hidden stream by using the **Stream -> Unhide** command. You can search a stream in the *Stream* pane by using the **Stream -> Find Stream** command.

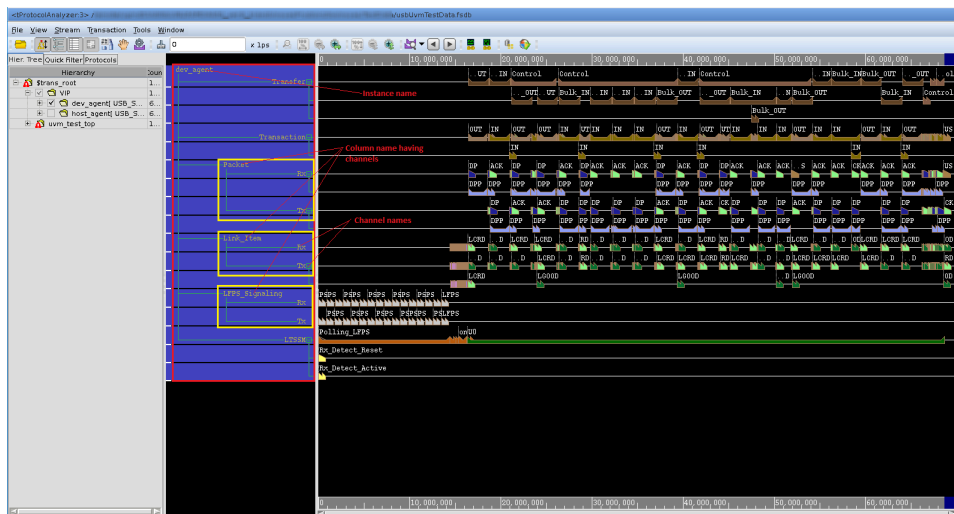
Figure 13 Selecting Rows in Stream Display View



Stream Pane Behavior for VIP Streams and Scopes

The FSDb file generated by the VIP contains streams grouped with channels, columns, and instances. The *Stream* pane in *Transaction and Protocol Analyzer* displays connections between channels, columns, and instances name. This makes it easier to identify channels, columns, and instances that belong to each other.

The following figure illustrates the `dev_agent` instance name connecting to `Transfer`, `Transaction`, `Packet`, `Link_Item`, `LFPS_Signalling`, and `LTSSM Packet` columns. Here, `Link_Item` and `LFPS_Signalling` column names are connecting to Rx and Tx channels.



Points to Note

- For instances whose first stream row and column have channels - the stream row heights are increased automatically to display the newly added instance and column names.
- The height of the above stream rows automatically increase only if the application default height is not changed. In case you have already executed vertical zoom in/out actions, the stream uses the application default height and displays the name.
- If you change the *Stream* pane width by dragging on the splitter using a mouse action, then newly added VIP instance or column names are displayed horizontally as compact as possible; otherwise, a scrollbar is added if available.
- If any of the newly added instance names or column name having channels cannot be displayed on top of the channel name, then the maximum width required to draw a column/instance is used for all other columns/instances as well.
- On selecting the **Full Stream Name** option in the **View** menu command, the instance name in the *Stream* pane displays the full path of the instance; there is no change in the display text for other columns or channels.

Stream Pane Behavior for Non-VIP Streams and Scopes

The FSDB file containing Non-VIP (UVM) transactions does not display groupings for channels, columns, or instances in the *Stream* pane.

You can move the streams up/down in the *Stream* pane by dragging and dropping the stream to the required position.

You can also create virtual streams for FSDB file containing UVM transactions. For details, see [Creating Virtual Streams](#).

Transaction Pane

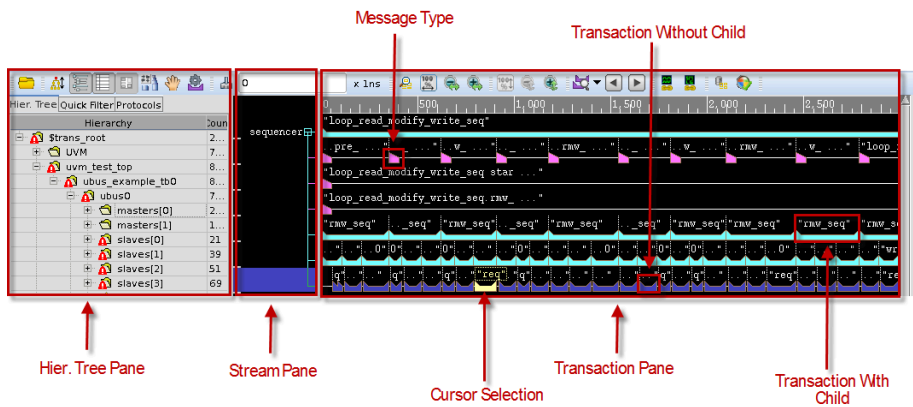
The *Transaction* pane displays the transaction recordings according to the streams; different transactions are shown with different shapes. See [Table: Transaction Pane Symbols](#) for classifications. You also can check the symbols in the *Transaction and Protocol Analyzer* tab of the *Legend* form using the **Help -> Legend** command.

Table 3 *Transaction Pane Symbols*

Symbol	Data Type
	Action
	Message

Table 3 Transaction Pane Symbols (Continued)

	Transaction with one or more child
	Unfinished Transaction
	Transaction without child
	Very short Transaction without showing end time well
	The selected transaction
	Transactions with same start time and end time



Transaction Pane Behavior for VIP Transactions

If the FSDB file contains VIP transactions, the *Transaction* pane in *Transaction and Protocol Analyzer* displays the transactions based on the default colors defined in the protocol definition files of VIPs. Additionally, it displays common text and additional text (predefined by the VIP) next to each of the transactions.

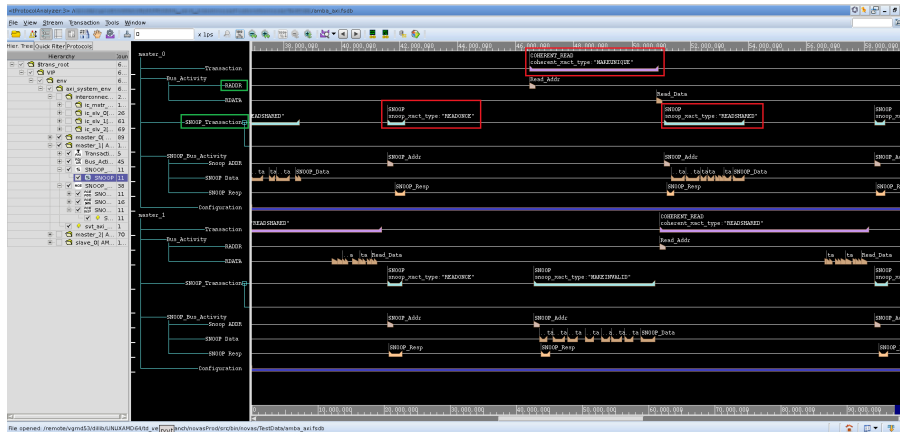
Note:

The common text is displayed only if the **Detail Transaction Information** option in the **View** menu is not selected.

As illustrated in the following figure, in the *Stream* pane, all the *RADDR* stream transactions have the common text as *Read_Addr* and *SNOOP_Transaction* stream transactions have a common display text as *SNOOP* (highlighted in green color in the figure).

As illustrated in the figure,

- For RADDR stream type transactions, `coherent_xact_type: "MAKEUNIQUE"` is the additional text displayed below the common text. Here, `coherent_xact_type` is the attribute name and `MAKEUNIQUE` is the value of the attribute for that transaction.
- For SNOOP_Transaction stream type transactions, `snoop_xact_type: "READONCE"` is the additional text displayed below the common text. Here, `snoop_xact_type` is the attribute name and `READONCE` is the value of the attribute for that transaction.



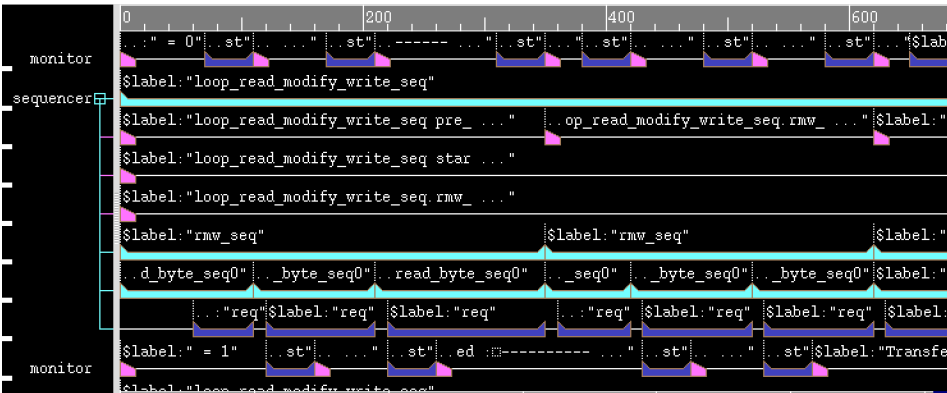
Points to Note

- By default, the attribute value is displayed in the *Transaction* pane. If the **View -> Display Attribute Name** menu command is selected, then the attribute name is displayed along with its value.
- If the newly added stream in the *Transaction* pane has predefined fields defined by the VIP, then height of the stream is increased automatically to display the additional text of a transaction. So, you can see both common text and additional text for all such transactions.
- When adding a stream to the *Transaction* pane, the height of the stream automatically increases only if the application default height is not changed. In case you have already executed vertical zoom in/out actions, the stream uses the application default height. Common text and additional text are displayed according to the available height.
- In case of adding virtual streams having both VIP/UVM type transactions to the *Transaction* pane,
 - VIP type transactions contain common text and additional text.
 - UVM type transactions display text continues to show the value of a top attribute present in the *Details* tab.

- Changes in the order of the configuration dialog attribute makes display text changes only for UVM type transactions.
- When collapsing the channels/columns using the  icon for the first time, the height of the new stream is the same as the first stream row height. However, on collapsing a channel/column from second time onwards, the stream uses the previous stream row height which is available at the time of previous expand.
- When expanding the channels/columns using the  icon, the height of the new channel/column rows are the same as the channel/columns rows which were available at the time of collapse.

Transaction Pane Behavior for Non-VIP Transactions

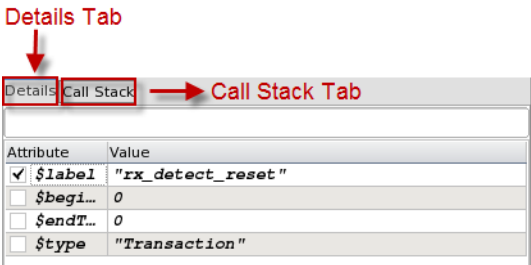
If the FSDB file contains non-VIP (UVM) transactions, the *Transaction* pane displays transactions with default colors specified in *Transaction and Protocol Analyzer*. Additionally, the stream name (\$label) is displayed next to each of the stream/transaction in the *Transaction* pane, as illustrated in the following figure:



Value Pane

The *Value* pane contains two tabs: *Details* tab and *Call Stack* tab. As illustrated in [Figure: Value Pane](#), the *Value* pane displays the details or the call stack (switch them by selecting the tab label) for the selected transaction.

Figure 14 Value Pane

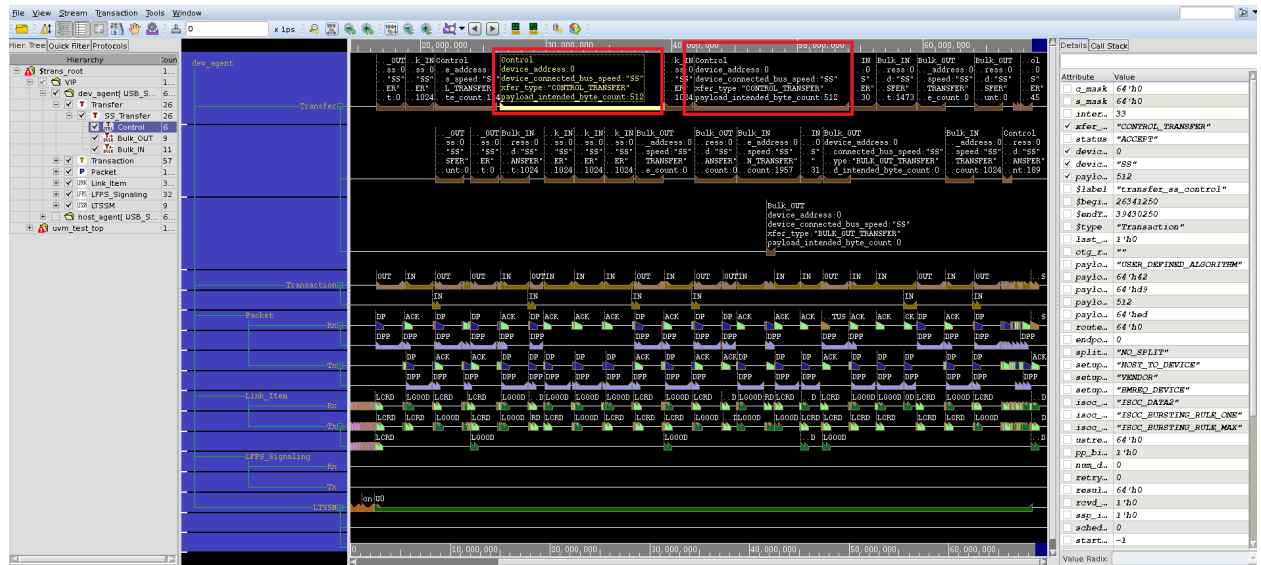


Details Tab

The *Details* tab in the *Value* pane displays the attributes for the selected transaction along with their value; a check-box is also displayed next to each of the attributes.

The detailed information of the selected transaction is shown in the *Detail* tab of the *Value* pane.

Figure 15 Value Pane and Status Bar

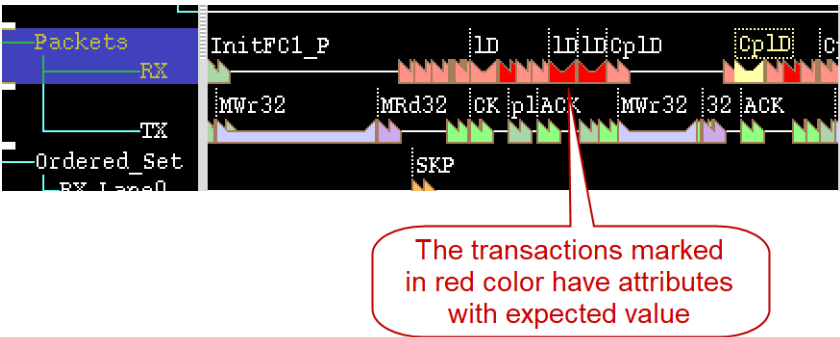


Points to Note

- The names and the values of the checked attributes are displayed below the common text based on the order shown in the *Details* tab.
- On checking an attribute in the *Details* tab, height of the stream row increases or decreases automatically (if required) to display the checked attributes' values as additional text. Checking too many attributes in the *Details* tab may consume the maximum height of the available *Transaction* pane, hence, you must be careful when checking the attributes in the *Details* tab.
- If you add a stream and check/un-check the required attributes' name and remove the stream from the *Transaction* pane and then add the same stream again, *Transaction* and *Protocol Analyzer* still retains the last state of the checked/un-checked attributes and displays only those attribute values as additional text.
- The check-box in the *Details* tab is not displayed if the **View -> Detail Transaction Information** menu command is selected or the **Detail Transaction Information** option is selected in the *Preferences -> Transaction Debug -> Transaction and Protocol Analyzer* page.

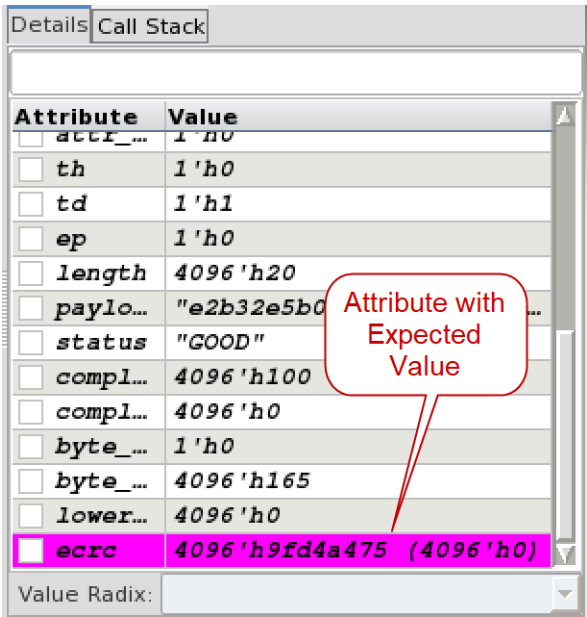
Details Tab Behavior for VIP Transactions

When the attributes of the transactions have expected value, the symbols are marked in red color in the *Transaction* pane, as shown in the following figure:



The attributes having expected value are highlighted in magenta color in the *Details* tab and the expected value is mentioned in brackets along with the actual value of the attribute. This is illustrated in the following figure:

Figure 16 Attribute Values



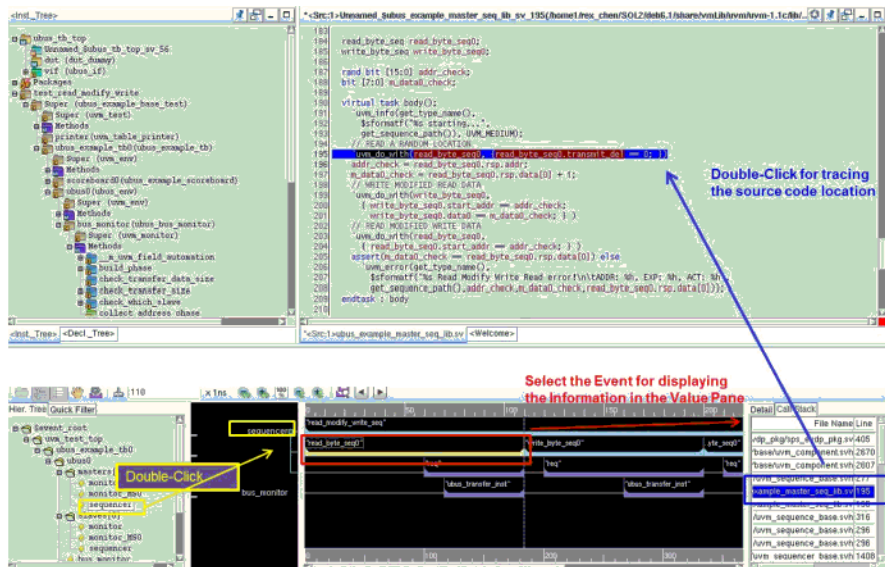
Call Stack Tab

You can use the *Call Stack* tab to check the location in the source code of the calling function by double-clicking the stack line (see [Figure: Use Transaction and Protocol Analyzer Window to View a Stream](#) the following text is the old one... [Figure: Use Transaction and Protocol Analyzer Window to View a Stream](#)).

Chapter 4: Using Transaction and Protocol Analyzer

Transaction and Protocol Analyzer Window Overview

Figure 17 Use Transaction and Protocol Analyzer Window to View a Stream



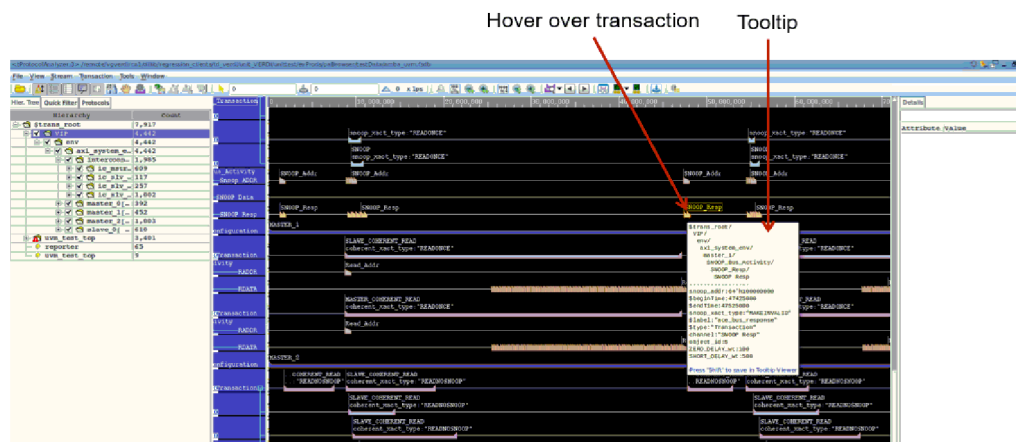
Tooltip

You can view multiple transaction details using the tooltip. The tooltip is visible on mouse hover for each transaction. The top few attributes for each transaction are displayed. For viewing specific attributes on the tooltip, use the **Configure attribute dialog** to order the attributes. At any given time only one tool tip is seen per transaction.

Transaction Tooltip

The following figure shows how hover over a transaction is used to display the tooltip with stream hierarchy and transaction attributes:

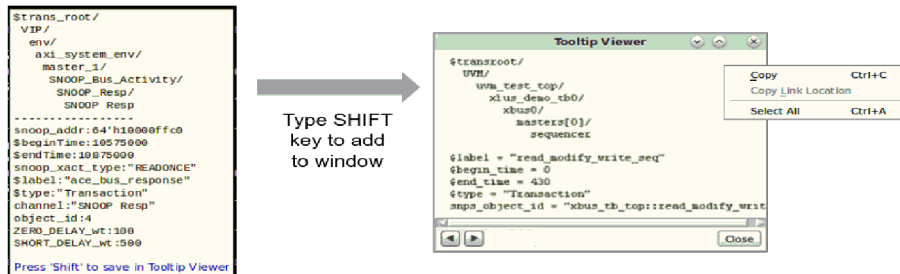
Figure 18 Hover Over Transaction



Tooltip Viewer

Use the SHIFT key with a tooltip displayed to add the transaction to the **Tooltip Viewer** window as shown in the following figure:

Figure 19 Hover Over using SHIFT Key



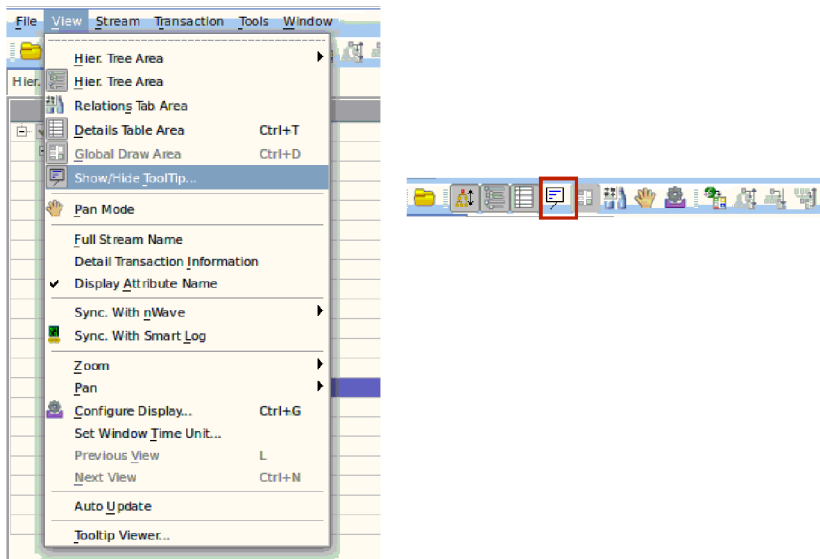
The **Tooltip Viewer** window contains the history of previously-displayed transactions. You can use the arrow buttons (or arrow keys) to navigate through the transaction display history. Use the **Copy** function (or CTRL-C) to copy the selected contents of the window to the clipboard. The **Copy** function is only available when the user opens the RMB context menu.

The tooltip window can be minimized or closed using the upper-right buttons or the "Close" button.

Enabling and Disabling Tooltip

By default, the tool tip is disabled and you can enable it through the menu or the tool bar option

Figure 20 Enabling and Disabling Tooltip



Tooltip Text Length

The text length of attributes shown in the tooltip can be controlled using the following environment variable:

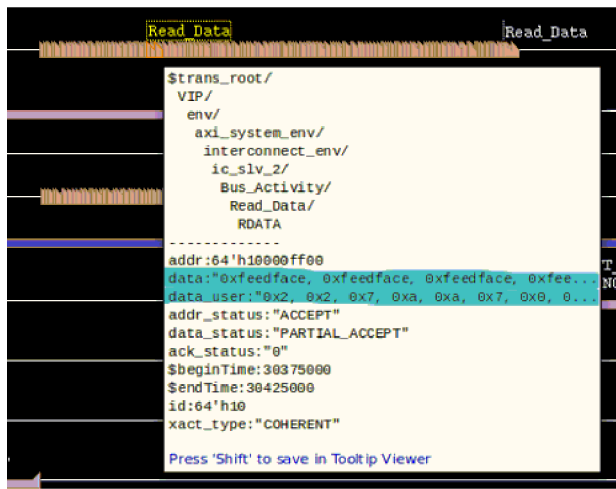
```
TNPA_TOOLTIP_TEXT_LEN  
setenv TNPA_TOOLTIP_TEXT_LEN 70
```

The default value is set to 50. If you set the value to 0 or less then the whole text is displayed. The following figures shows how the tooltip appears when the attribute length is too long and when the value is truncated.

Figure 21 Before Truncated



Figure 22 After Truncated



Preference Options

You can change the default setting for showing/hiding the display of the panes or information in the Transaction and Protocol Analyzer window. The height of the stream and initial zoom ratio are also configurable.

To access the preference options, use the **Tools -> Preferences** menu command and select the **Transaction and Protocol Analyzer** page under the *Transaction Debug* folder in the left pane.

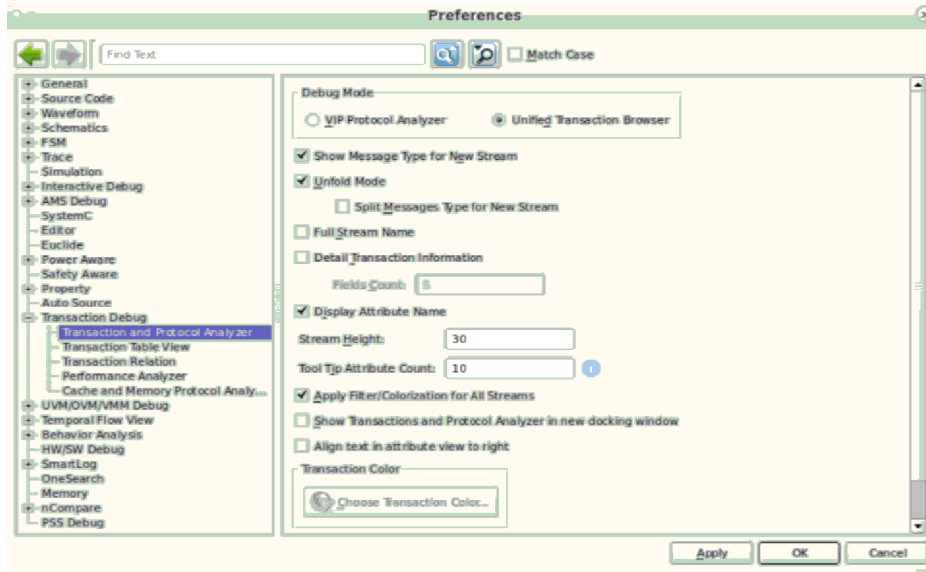
The following Debug Mode options are available in the **Transaction and Protocol Analyzer** page of the *Preferences* form:

- **VIP Protocol Analyzer** - Select this mode to switch the application type to VIP Protocol Analyzer. On selecting this mode, all the preference page options are hidden.
- **Unified Transaction Browser** - Select this mode to access preference settings for Transaction and Protocol Analyzer (that is, for both *Protocol Analyzer* and *Transaction Browser*).

Note:

It is recommended to use the **Unified Transaction Browser** mode, which is the default mode. This chapter only talks about **Unified Transaction Browser** mode. For details on the **VIP Protocol Analyzer** mode, see [Appendix A: Using Transaction and Protocol Analyzer \(Protocol Debug Mode\)](#).

The **Transaction and Protocol Analyzer** page of the *Preferences* form is illustrated in the following figure:



The following is the list of available Transaction and Protocol Analyzer preference options:

- **Show Message Type for New Stream**

If enabled, Transaction and Protocol Analyzer displays the message types for a new stream.

- **Unfold Mode**

This option automatically expands all the child streams and then splits overlapping parent/child streams when you add a stream (assuming the transactions of this stream have children) in the *Unfold Mode*. It is enabled by default.

If you add a stream with the *Unfold Mode* disabled, the stream is added without any expansion. In this case, if the stream has a child, you can double-click the stream to expand to a new stream or double-click a transaction to expand to the original stream.

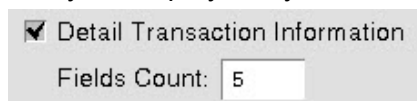
- **Full Stream Name**

If enabled, this option displays the full stream name of all streams in *Stream* pane.

- **Detail Transaction Information**

If enabled, this option allows you to choose the number of transaction fields to be viewed in the transaction display by entering the count in the **Fields Count** field. By default, the **Fields Count** value is 0 and *Transaction and Protocol Analyzer* displays all the available fields of transactions in the *Transaction* display pane.

For example, if 5 is specified in the **Fields Count** field, Transaction and Protocol Analyzer displays only the first five fields based on the order in the *Details* tab.



A screenshot of a software interface showing a checkbox labeled 'Detail Transaction Information' which is checked. Below it is a text input field labeled 'Fields Count:' with the number '5' entered.

- **Display Attribute Name**

If enabled, this option displays attribute names in the *Transaction* pane.

- **Stream Height**

The default value is 30.

- **Tool Tip Attribute Count**

The default Value is 10 but can be changed up to 20.

- **Apply Filter/Colorization for All Streams**

If enabled, this option applies filter/colorization settings to all the streams or channels present in the *Transaction* pane. For newly added streams, previously specified filter/colorization settings are applied and displayed in the *Transaction* pane.

To apply different filter/colorization settings for different sets of streams, you must disable/un-check the **Apply Filter/Colorization for All Streams** option in the *Preferences* page of *Transaction and Protocol Analyzer*.

When the **Apply Filter/Colorization for All Streams** option is in the selected state, you can clear all the filtered settings applied in all the streams loaded in the *Transaction* pane by using the **Stream -> Clear Filtered Results** menu command. This option enables *Transaction and Protocol Analyzer* to stop applying filter settings to newly added streams until any other filter settings are applied. Similar behavior is applicable for the **Stream -> Clear Colored Results** menu command as well.

If no filter/colorization setting is specified, then newly added streams do not have any filter or colorization; if required, click the **Stream -> Filter/Colorize Transaction and Protocol Analyzer** menu command.

Transaction and Protocol Analyzer Operations

This section describes the *Transaction and Protocol Analyzer* operations in the following sub-sections:

- [Displaying Attribute Name and Value as Additional Text in Transaction Pane](#)
- [Highlighting by Relation](#)

Chapter 4: Using Transaction and Protocol Analyzer Transaction and Protocol Analyzer Operations

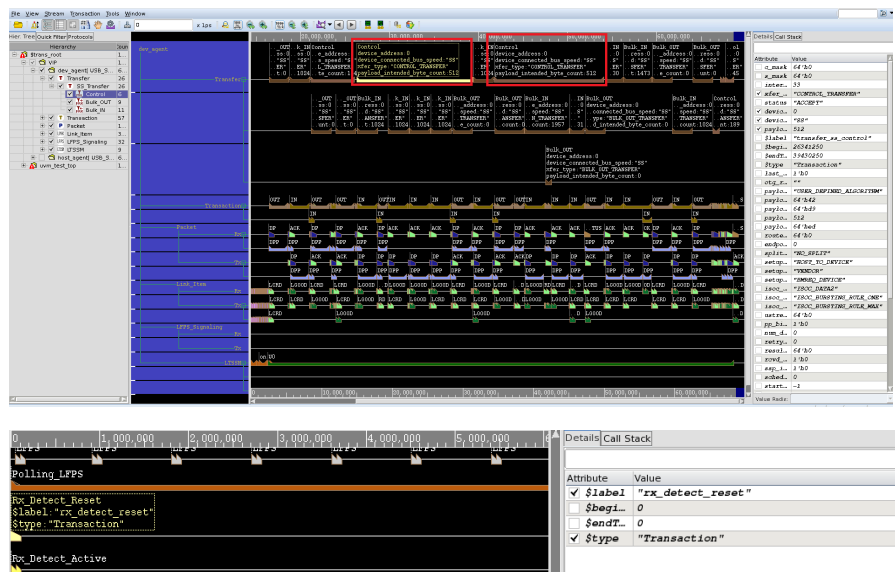
- Adding Streams by Relation From the Selected Transaction
- Search Capability in Hier. Tree Tab
- Search or Filter Capability in Details Tab
- Search Capability for Transactions
- Configuring Display
- Filter/Colorize the Transactions
- Creating Virtual Streams
- Copying Stream Name From Protocol Analyzer Window

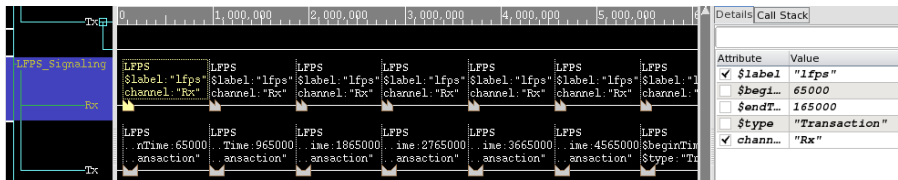
Displaying Attribute Name and Value as Additional Text in Transaction Pane

To add or display the attribute name as additional text for a transaction in the *Transaction* pane, the **View -> Display Attribute Name** menu command must be selected/enabled. The names of all the checked attributes are displayed along with their values.

To add or display the attribute value as additional text for a transaction in the *Transaction* pane, check the check-box next to the required attribute.

The checked state of an attribute is retained across all other transactions of the same stream. So, on selecting another transaction of the same stream, the attribute value for that transaction is also displayed as additional text in the *Transaction* pane. This is also illustrated in the following figure:





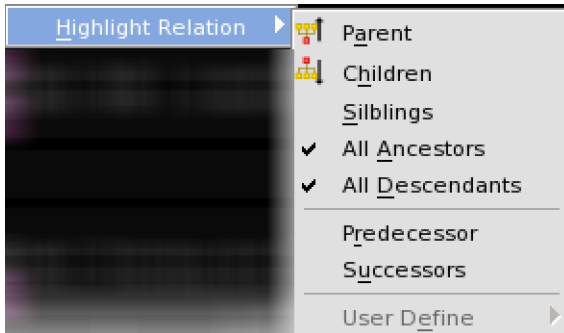
To remove an attribute value displayed as additional text for a transaction in the *Transaction* pane, uncheck the check-box next to the required attribute.

If the selected transaction supports the display of some default attributes for its corresponding stream, then all those attributes are checked in the *Details* tab and their values are also displayed as additional text below the common text for the selected transaction. The attribute values are displayed in a separate row for each of the checked attribute.

Highlighting by Relation

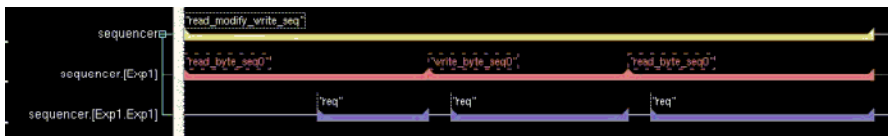
Transaction and Protocol Analyzer provides the capability to highlight the relation between two transactions. You can highlight the related parent/children/siblings of the selected transaction using the **Highlight Relation** right-click option and toggle on the related sub-options. This option is illustrated in the following figure:

Figure 23 Highlight Relation Right-Click Option



For example, on clicking the **Highlight Relation -> Children** right-click option, the child/children of the selected transaction gets highlighted in red color (as illustrated in [Parent/Child Relationship](#)).

Figure 24 Parent/Child Relationship



Adding Streams by Relation From the Selected Transaction

Transaction and Protocol Analyzer provides the capability to add a parent/child stream to a selected stream using the following right-click options in the *Transaction* pane:

- **Add Parent Stream(s) by Relation**

This option adds parent streams to the selected streams and highlights the parent streams. The parent streams are displayed in front of the selected streams.

- **Add Child Stream(s) by Relation**

This option adds the child streams to the selected streams and highlights the child streams. The child streams are displayed next to the selected stream.

- **Add Predecessor Stream(s) by Relation**

This option loads the predecessor streams of the selected transactions (if not loaded) and highlights the predecessor streams. The predecessor streams are displayed next to the selected stream.

- **Add Successor Stream(s) by Relation**

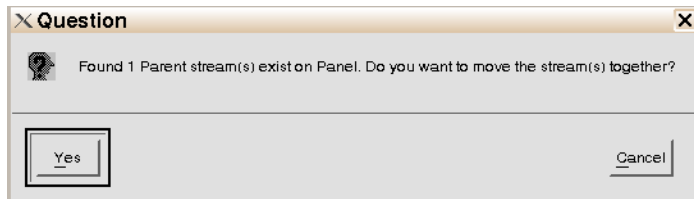
This option loads the successor streams of the selected transactions (if not loaded) and highlights the successor streams. The successor streams are displayed next to the selected stream.

Note:

On selecting these options, the corresponding relation type sub-options in the **Highlight Relation** option are also toggled **ON**. For example, on selecting **Add Child Stream(s) by Relation** option, the immediate child stream gets added and the **Children** sub-option of the **Highlight Relation** right-click option is toggled **ON**.

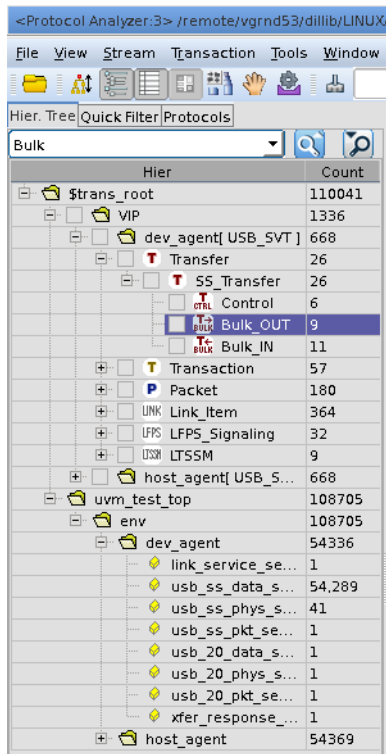
For non-VIP streams, if you select a stream that is already present on the stream panel, a pop-up is displayed with a message indicating the number of streams already present on the panel and questioning you to either go ahead or move the existing stream(s). Click **Yes** to move the existing parent/child streams. Choose **Cancel** to stop the operation. The message is illustrated in the following figure:

Figure 25 Message Dialog



Search Capability in Hier. Tree Tab

Transaction and Protocol Analyzer provides the capability to search a stream or scope in the project *Hier. Tree* tab. You can enter the search string in the text field displayed at the top of the project *Hier. Tree*; the matching scope or stream gets selected and displayed, as illustrated in the following figure:



To display or hide the Navigation text field, select the **Show navigation text field** right-click option in the project *Hier. Tree* tab.

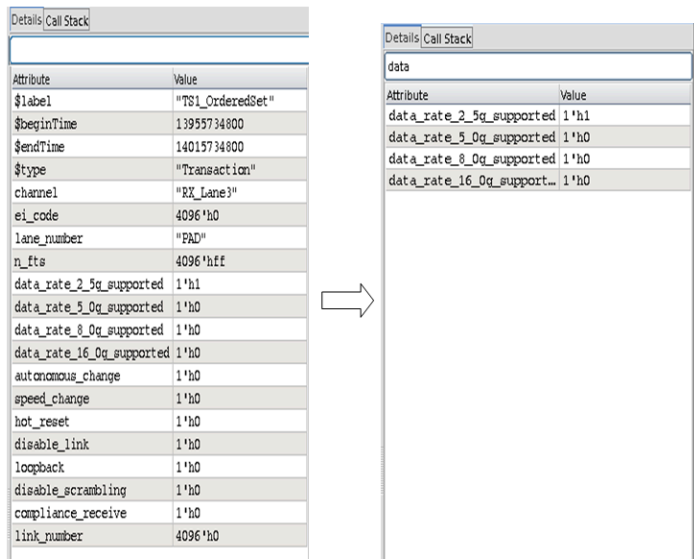
Points to Note

- The search string is case-insensitive
- Transaction and Protocol Analyzer checks occurrences in the Project *Hierarchy* tree.
- To traverse matching items upwards, click the  icon.
- To traverse matching items downwards, click the  icon.
- Regular expressions and wildcards are not supported in the search field.

Search or Filter Capability in Details Tab

Transaction and Protocol Analyzer provides search and filter capabilities to search for the required fields in the protocol recorded transactions in the *Details* tab. You can enter the search string in the text field displayed at the top of the details tab; matching scopes/streams get displayed as illustrated in the following figure.

Figure 26 Search Capability in Details Pane



Note:

To search multiple fields, enter multiple strings in the search text field with space as a delimiter.

The search text field supports wildcards with "?" and "*" characters.

Search Capability for Transactions

Transaction and Protocol Analyzer provides the capability to search the transactions that have a particular attribute or search the transactions that match a specified expression.

Click the  icon to set the search attribute using the *Set Search Attribute* form, as illustrated in the following figure.

Figure 27 Set Search Attribute Form

Set Search Attribute

Search Type

☐ Any Transaction ☒ By Attribute

By Attributes

Attribute	Function	Description	Operator
\$stream	has_attrb...	Check whether transaction has this attri	[]
\$time	has_relati...	Check whether transaction has parent/c	[:]
\$beginTime	has_tag	Check whether transaction has tag. ex: !	.
\$endTime	is_unfinish	Check whether transaction is unfinished	'
\$label			-
\$tag			!
\$type			~
\$T			&
			~&

ack nak sen num
Expression: (Time unit is "fs"; \$R is at "RX, 22940000000fs")
address_translation==4096'h0 && status=="GOOD"

Name Expression

exp1 address_translation==4096'h0 && status=="GOOD"

Import
Clear
Add
Remove
Modify
Save...
Append...

Search Close

To search by any transaction, select the **Any Transaction** option in the *Search Type* field in the form. Use **Search Backward** and **Search Forward** icons   on the toolbar to synchronize and select the next and previous transactions based on time.

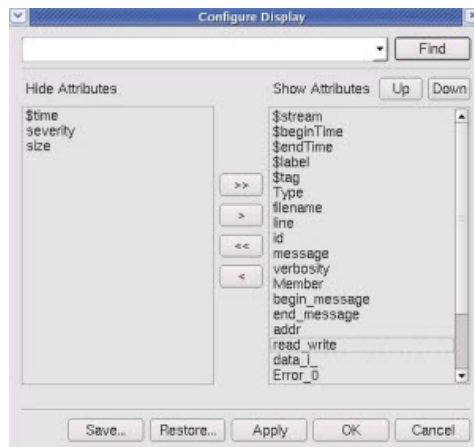
To search by an attribute, select the **By Attribute** option in the *Search Type* field in the form. The **Search Backward** and **Search Forward** icons   on the toolbar synchronize to select next and previous search transactions based on the search query entered in the *Expression* field of the *Set Search Attribute* form. If the query is successful, the search results are displayed in a tabular form below the *Expression* field in the *Set Search Attribute* form. Additionally, the Search icon in toolbar changes to a pull-down menu  toolbar icon, using which you can change the required search type.

Configuring Display

You can use the **View -> Configure Display** command to hide/show some attributes in all streams in the current window. This command will open the Configure Display form (see figure [Configure Display](#)). The string entered in the **Find** field will find the partial matched attributes in both lists and be highlighted. Use the arrow buttons to decide the **Hide** and **Show Attributes**.

You can also change the attributes display order of all the panes in this Transaction and Protocol Analyzer window using the **Up** and the **Down** buttons.

Figure 28 *Configure Display*



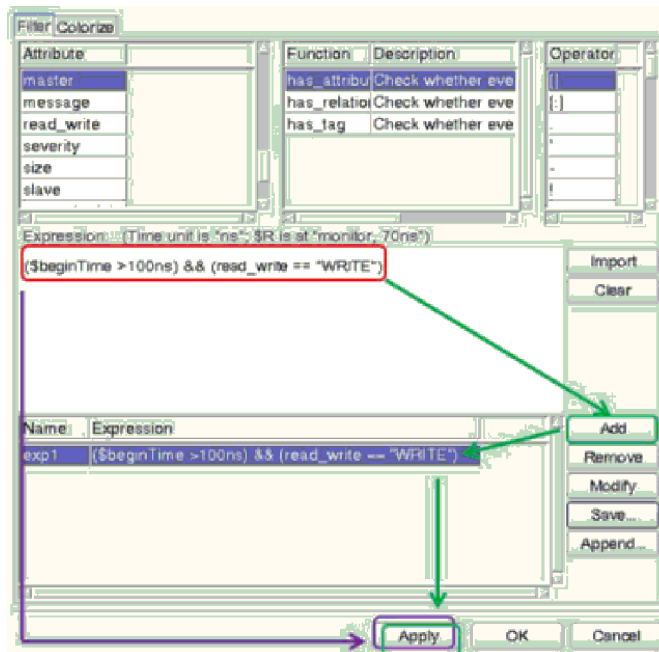
You can save or restore the configuration by clicking the **Save** or **Restore** buttons.

Filter/Colorize the Transactions

The transactions may be filtered based on more complex expressions. Select the streams and display the *Filter/Colorize* form by selecting the **Stream -> Filter/Colorize** command. The usage of entering the expression is the same as the usage of creating the virtual stream. See section [Creating Virtual Streams](#) for details.

After entering the desired expression, click the **Apply** button and the filtering result is applied to the selected streams in the *Transaction and Protocol Analyzer* window. You also can select the expression that has been added into the history field by clicking the **Add** button and applying to the selected streams. You can save or append the list of created virtual streams to a list file by clicking the **Save** or **Append** buttons.

Figure 29 *Filter/Colorize Form*



You can colorize the streams to highlight the transaction in a specified color and easily observe the transactions/streams matching some conditions. After entering the desired expression, click the **Add** button and then the **Color** column (see figure [Select the Specified Color](#)) will be displayed in the history field. Specify the color and click the **Apply** button, the colorized result is applied to the selected streams in the *Transaction and Protocol Analyzer* window.

Figure 30 *Select the Specified Color*

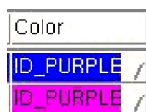
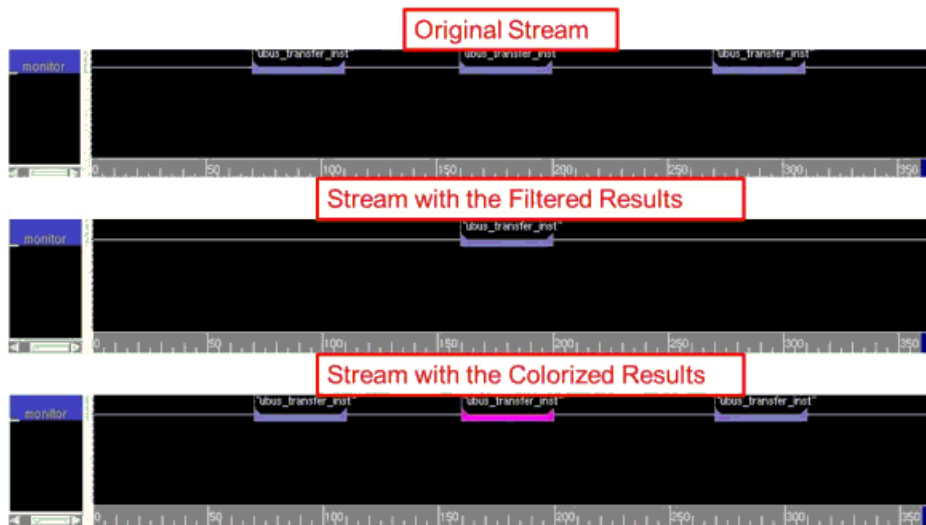


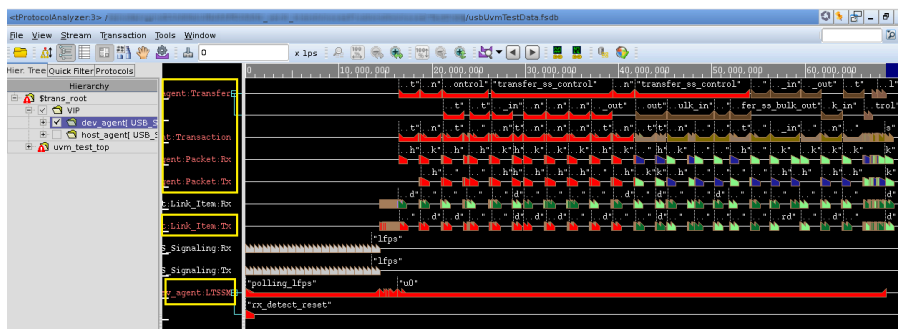
Figure shows the results of filtering and colorizing the *monitor* stream with the `$beginTime > 100ns && read write == "write"` expression.

Figure 31 Filtered and Colorized Results



You can use the **Stream -> Clear Filtering Results** or **Stream -> Clear Colorization Results** commands to clear the filtered or the colorized result.

The *Transaction* pane shows stream/channel/column names with red color to indicate the stream is has filter/colorization settings applied, as shown in the following figure:



Creating Virtual Streams

The Transaction Debug solution provides users a way to combine a set of transactions into one stream and view them together through virtual streams. You can write a flexible expression to select the matching transactions, create a stream which contains only the matching transactions, and view them together. This helps the users to create their own stream dynamically with a meaningful name having certain transactions and/or messages and segregate the transactions into several streams for better debugging.

Note:

This feature/functionality is not supported for VIP transactions.

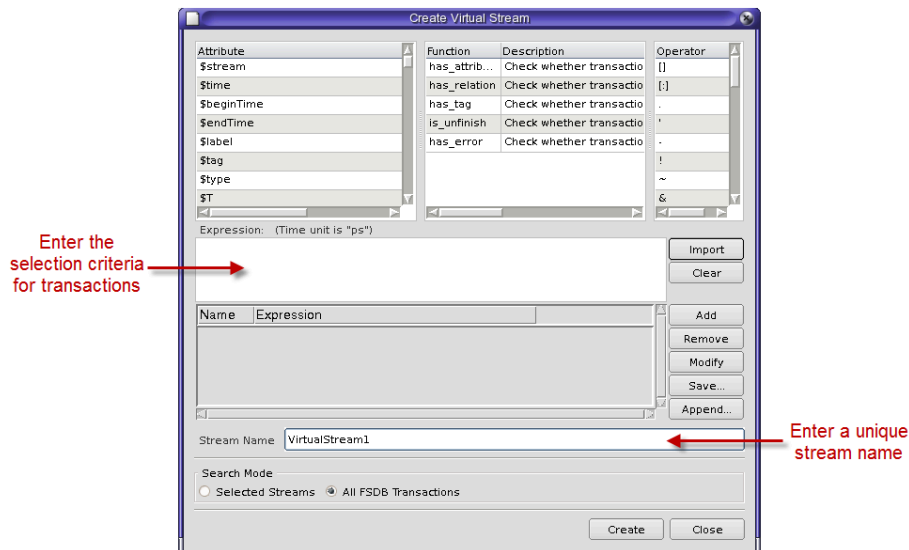
You can create a virtual stream by using any of the following commands:

- The **Stream -> Create Virtual Stream** menu command.

This command opens the *Create Virtual Stream* form. The **All FSDB Transactions** option is selected by default as the *Search Mode*. This is illustrated in the figure below.

Note:

The **Create Virtual Stream** option is enabled only if an FSDB file is loaded into Verdi.



In the *Create Virtual Stream* form, enter the selection criteria for transactions in the *Expression* field and enter a unique stream name in the *Stream Name* field. Then click the **Create** button to create a virtual stream with all the transactions matching the criteria.

- **Create Virtual Stream** RMB option in the *Stream* display view.

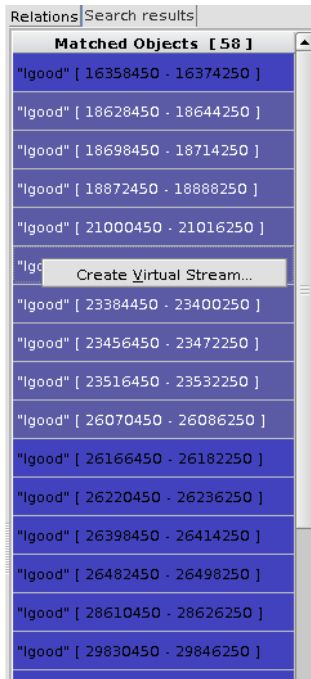
This command is enabled only if a row is selected in the stream display.

This command opens the *Create Virtual Stream* form and **Selected Streams** option is selected by default in the search mode.

- **Create Virtual Stream** RMB option in the *Search results* view.

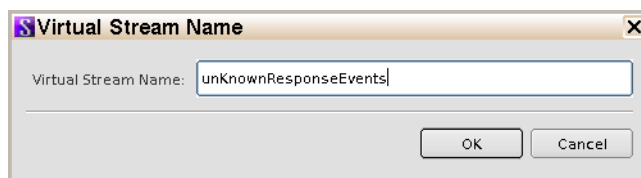
You can create a virtual stream by grouping all selected transactions in the *Search Results* view. The **Create Virtual Stream** RMB option is enabled only if a row is selected in the *Search results* view, as illustrated in the following figure:

Figure 32 Create Virtual Stream RMB Option



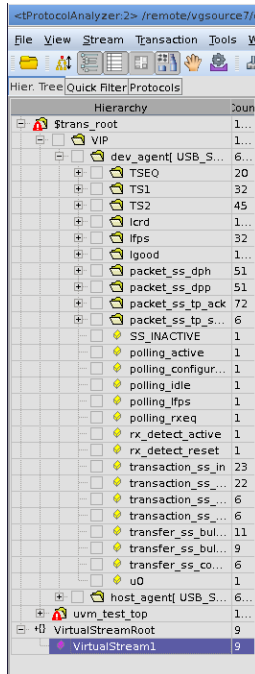
The **Create Virtual Stream** RMB option opens the *Virtual Stream Name* form as illustrated in the following figure:

Figure 33 Virtual Stream Name Form



In the *Virtual Stream Name Form*, enter the stream name in the **Virtual Stream Name** field and click **OK**. Transaction and Protocol Analyzer groups all selected transactions and creates a virtual stream with the specified name.

Figure 34 *Virtual Stream in the Hier. Tree Tab*



The **Function** field in the *Create Virtual Stream* form provides three predefined functions for use as an operand: **has_attribute()**, **has_relation()** and **has_tag()**. You can enter the attribute/relation/tag you want to match in the parentheses after clicking the function. Keeping the parameter blank in the expression will result in TRUE value if transactions have any attribute/relation/tag(s). For example:

```
has attribute("att1") && has relation("child") && has tag()
```

For the **has_relation** function, the expression support:

- **has_relation(“relation_name”)**, for example, **has_relation(“ud1”)** will be true for both the Master and Slave sides if there is a **ud1** relationship between them.
- **has_relation(“relation_name_fr”)**, for example, **has_relation(“ud1_fr”)** will be true if the **ud1** transaction occurs in the Slave side.
- **has_relation(“relation_name_to”)**, for example, **has_relation(“ud1_to”)** will be true if the **ud1** transaction occurs in the Master side.

The **Expression** field also provides a **\$R** operand. **\$R** represents the selected transaction.

You also can get the expression from the *Quick Filter* pane using following steps (see figure [Create Virtual Stream through Quick Filter](#)):

1. Check the desired attributes in the *Quick Filter* pane.
2. The expression of the combined attributes is automatically shown in the **Expression** field.
3. Click the **Import** button in the *Create Virtual Stream* form.
4. As shown in figure [Get Expression from Quick Filter](#), the expression in the *Quick Filter* pane is imported to the **Expression** field in the *Create Virtual Stream* form.

Figure 35 Create Virtual Stream through Quick Filter

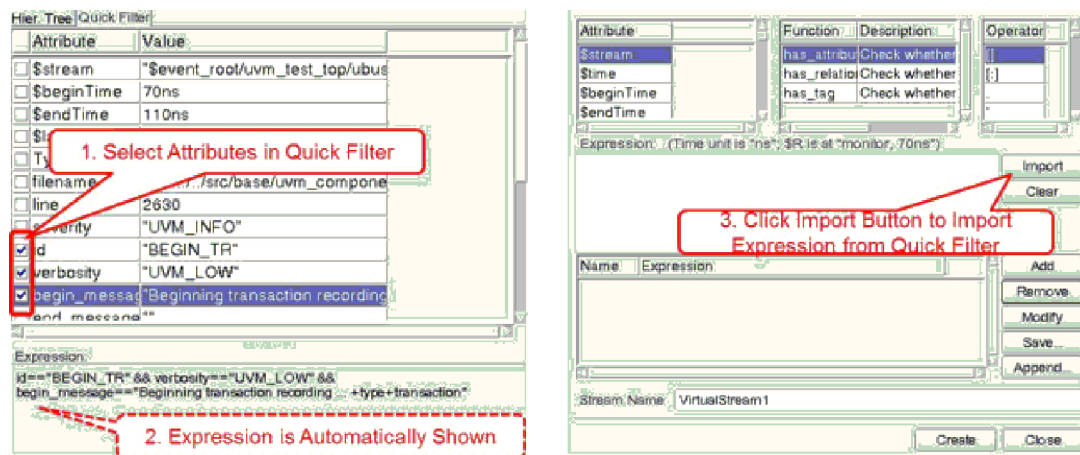
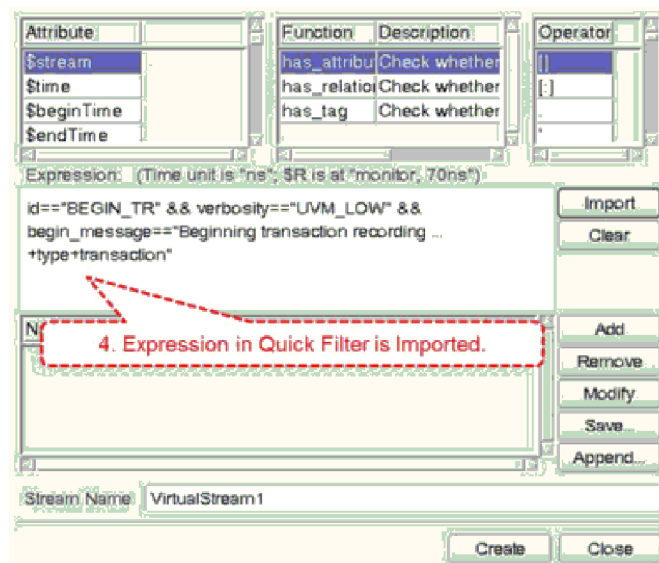


Figure 36 Get Expression from Quick Filter

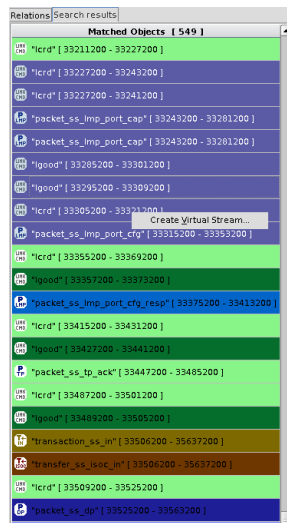


After setting up the expression, click the **Add** button to add the stream into the history (lower) field. The **Name** column is editable for keeping more meaningful details in the history list. After you click the **Create** button, the created virtual stream will be added into the stream pane. The stream name is shown in the **Stream Name** field which is also editable. You can save or append the list of created virtual streams to a list file by clicking the **Save** or **Append** buttons. The **Clear** button is provided to clear the strings in the **Expression** field for the next new expression input.

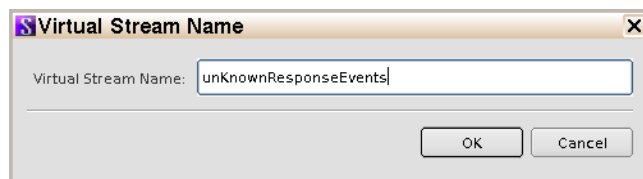
Search Results View

You can create a virtual stream by grouping all the selected transactions in the *Search Results* tab and selecting the **Create Virtual Stream** right-click option. as illustrated in the following figure:

Figure 37 Creating Virtual Stream by Grouping Transactions in Search Results View



On selecting the **Create Virtual Stream** right-click option, the Virtual Stream Name form opens as illustrated in the following figure:



Note that the **Create Virtual Stream** right-click option is enabled only if any of the row(s) is selected in the *Search Results* view.

Search Modes

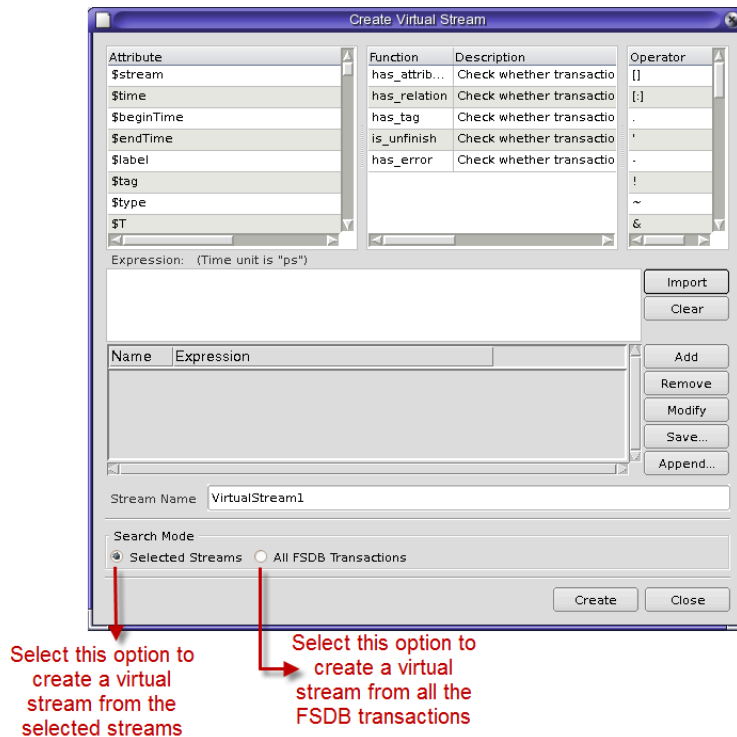
Virtual stream supports the following search modes:

- Selected Streams

This mode allows you to create a virtual stream by considering only the matching transactions available in the selected streams in the *Stream* display view.

To create a virtual stream from the selected streams,

1. Select the streams in the *Stream* display view.
2. Click the **Stream -> Create Virtual Stream** menu command or click the **Create Virtual Stream** RMB command.
3. In the *Create Virtual Stream* form, select **Selected Streams** in the *Search Mode* field as illustrated in the following figure:



- All FSDB Transactions

This mode allows you to create a virtual stream by considering only the matching transactions available in the FSDB file. This search mode is selected by default in the *Create Virtual Stream* form.

Note:

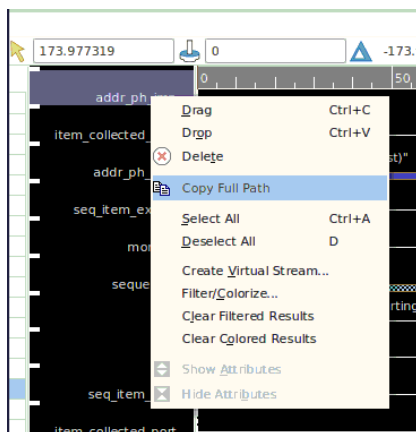
If any virtual stream is added or removed by other applications, Transaction and Protocol Analyzer updates the hierarchy tree to reflect the changes.

Copying Stream Name From Protocol Analyzer Window

You can copy signal names and hierarchy in the *Protocol Analyzer* window.

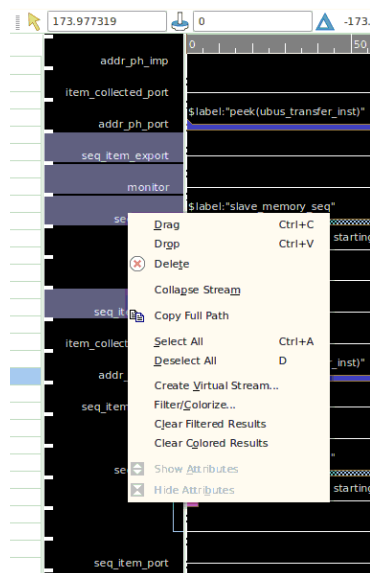
The following figure shows that you can copy the entire stream name using the **Copy Full Path** option.

Figure 38 *Copy Full Path Option*



When you use the **Copy Full Path** option in multiple selections, the full path related to the first selected instance gets copied as shown in the following figure:

Figure 39 Full Path Related to the First Selected Instance Gets Copied



Interlinking Between Transactions and Signals

Transaction and Protocol Analyzer is tightly integrated with *nWave*, which is helpful in debugging testbench simulation using transactions and waveform signals. You can open the *Transaction and Protocol Analyzer* application from *nWave*, load the corresponding stream, and view the selected associated streams and transactions of a signal in *Transaction and Protocol Analyzer*. You can also load the signals from *Transaction and Protocol Analyzer* to the *nWave* window.

This section consists of the following sub-sections:

- [Loading UVM Interface Signals From the Selected Transaction](#)
- [Loading Protocol Interface Signals to Selected nWave Application](#)
- [Loading Transactions to Selected Transaction and Protocol Analyzer Application](#)
- [Synchronization With nWave](#)
- [Points to Note](#)

Note:

Only some VIPs support the features: [Loading Protocol Interface Signals to Selected nWave Application](#) and [Loading Transactions to Selected Transaction and Protocol Analyzer Application](#). For details on which VIPs support these features, contact Synopsys support.

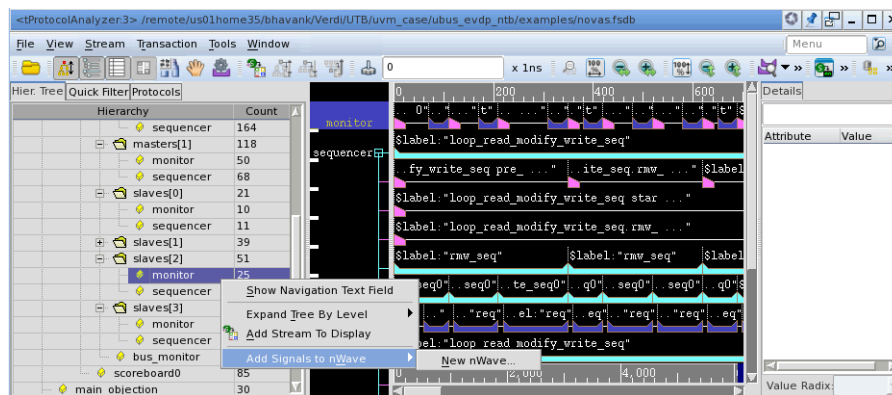
Loading UVM Interface Signals From the Selected Transaction

After the simulation, the generated default `novas.fsdb` FSDB file includes virtual interface information.

To load the UVM interface signals from the selected transaction, perform the following steps:

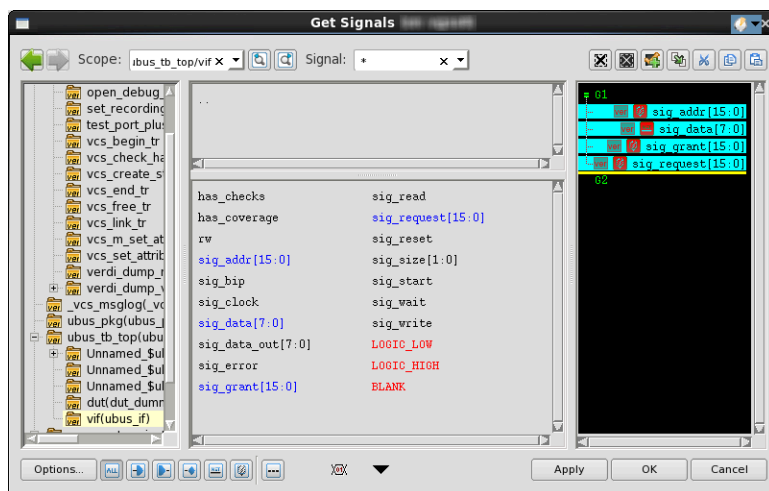
1. Open the *Transaction and Protocol Analyzer* application in Verdi and load the FSDB file.
2. In the *Hier. Tree* tab in *Transaction and Protocol Analyzer*, select a stream and select the **Add Signals to nWave** right-click option. This option is enabled only if an FSDB file has an interface signal path for the selected object instance.

Figure 40 Add Signals to nWave Right-Click Option



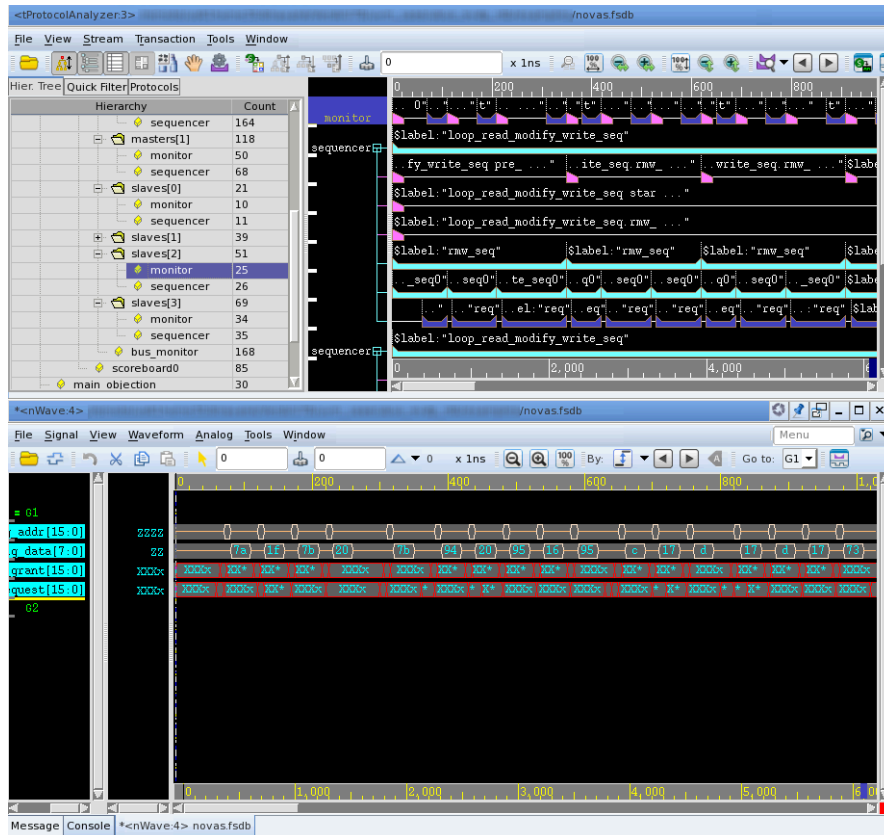
3. The *Get Signals* form opens, select the virtual interface signals, and click **OK** to view the signals in *nWave*, as illustrated in the following figure:

Figure 41 *Get Signals Form*

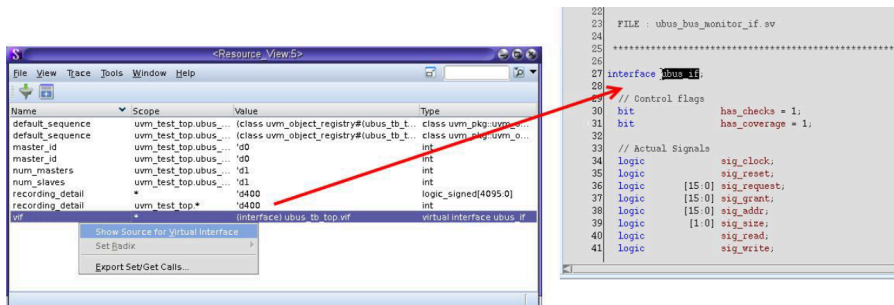


4. The selected signals get added to nWave, as illustrated in the following figure:

Figure 42 Signals Added to nWave



If you also invoke the *Resource View* frame of UVM Debug, you can see the virtual interface resource in the *Resource View* frame and use the **Show Source for Virtual Interface** right-click command to check the definition code in the *Source Code* pane.



Note:

To enable dumping virtual interface with IUS, make sure to specify the following file and options during compilation:

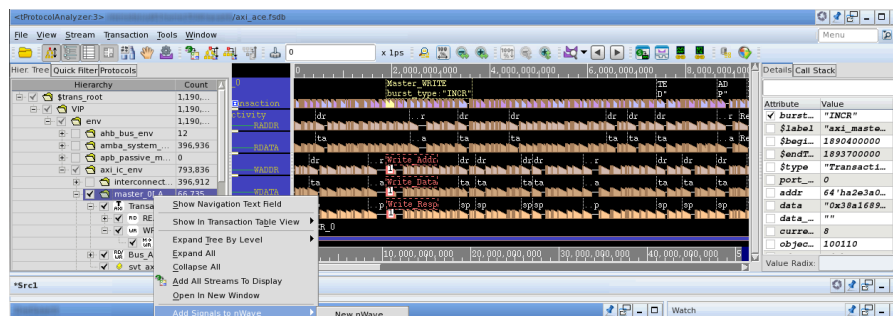
```
$VERDI_HOME/etc/uvm-1.1/verdi/dpi/uvm_verdi_dpi.cpp
+define+UVM_VERDI_VIF_RECORD
+define+UVM_VERDI_DPI
-DINCA
```

Loading Protocol Interface Signals to Selected nWave Application

To load the protocol interface signals into the selected *nWave* application, perform the following steps:

1. Open the *Transaction and Protocol Analyzer* application in Verdi.
2. Load the FSDB file containing HDL signals.
3. In the *Hier. Tree* tab in *Transaction and Protocol Analyzer*, select a hierarchy or a stream item below or on the same level of the instance hierarchy and select the **Add Signals to nWave** right-click option as illustrated in the following figure:

Figure 43 Add Signals to nWave Option



The following sub-options are present for the **Add Signals to nWave** option:

- **Add Signals to nWave -> New nWave**
On clicking the above option, a new *nWave* window is opened and the selected signal or instance path is loaded to it.
- **Add Signals to nWave -> Add to nWave X** (where **X** signifies the opened *nWave* window number. All *nWave* windows which are already opened are shown as actions under the **Add Signals to nWave** option.)

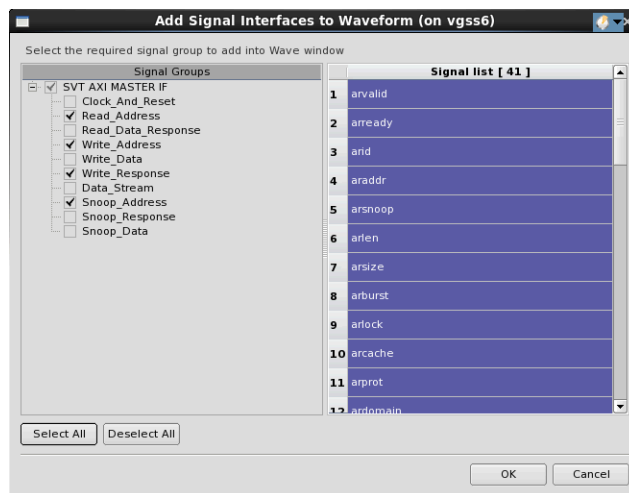
On clicking the above option, the selected signal or instance path is loaded to the selected *nWave* window.

Note:

Clicking the  icon adds the selected signals to the primary *nWave* window. Similarly, the **Add Signals to nWave** right-click option in the *Transaction* pane of *Transaction and Protocol Analyzer* also adds the selected signals to the primary *nWave* window.

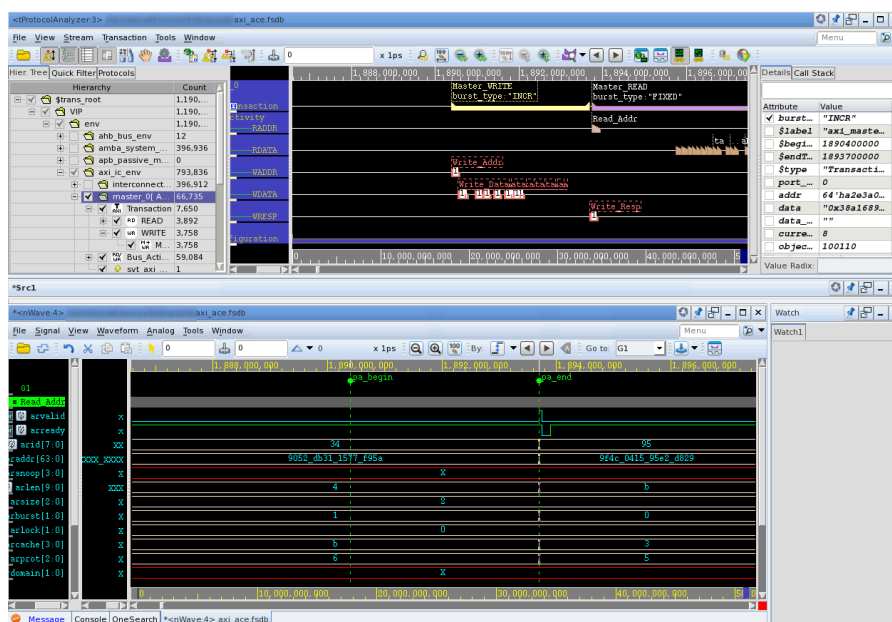
4. The *Add Signal Interfaces to Waveform* form opens. In this form, select the signal groups that you want to add to *nWave*, as illustrated in the following figure:

Figure 44 Add Signal Interfaces to Waveform Form



5. The selected signal groups are added to *nWave*, as illustrated in the following figure:

Chapter 4: Using Transaction and Protocol Analyzer Interlinking Between Transactions and Signals



If only one group is specified, all the signals of that group are loaded into *nWave*. If more than one group is specified, the interface signal form with the specified groups and its signals are displayed and you can select the required groups or signals and load them into *nWave*.

Loading Transactions to Selected Transaction and Protocol Analyzer Application

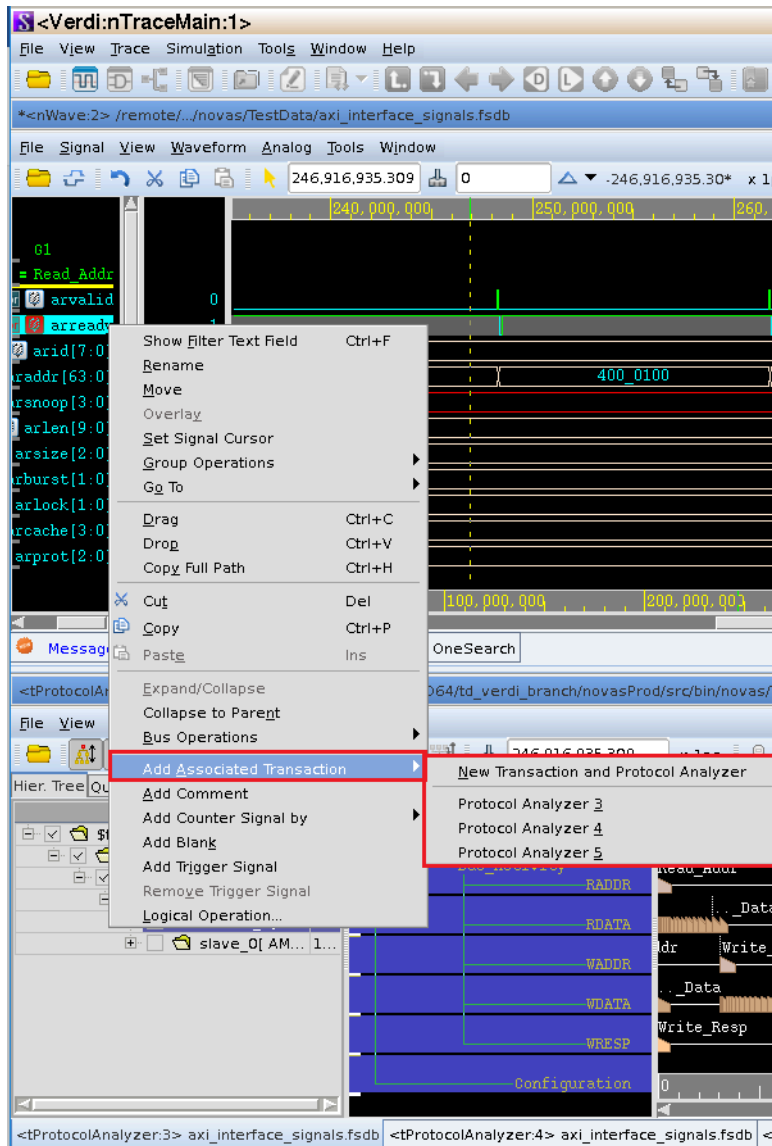
To open the *Transaction and Protocol Analyzer* application from the *nWave* window, perform the following steps:

1. Invoke Verdi and open the *nWave* window.
2. In the *nWave* window, load the FSDB file containing HDL signals using the **File -> Open** menu command.
3. Load the signals by clicking the **Signal -> Get Signals** command and selecting the required signals from the opened *Get Signals* form.
4. In the *nWave* window, select one signal from the list of loaded signals in the *Signal* pane, right-click and select the **Add Associated Transaction** right-click option as illustrated in [Figure: Add Associated Transaction Right-Click Option](#).

Note:

Both HDL signals and transactions must be captured in one FSDB file along with the interface signals and the interface path information.

Figure 45 Add Associated Transaction Right-Click Option



The following sub-options are present for the **Add Associated Transaction** option:

- **Add Associated Transaction -> New Transaction and Protocol Analyzer**

On clicking the above option, a new *Transaction and Protocol Analyzer* application opens and the associated streams and transactions of the selected signal are loaded to it.

- **Add Associated Transaction -> Protocol Analyzer X** (where **X** signifies the opened *Transaction and Protocol Analyzer* application number. All *Transaction and Protocol*

Analyzer applications which are already opened are shown as actions under the **Add Associated Transaction** option.)

On clicking the above option, the selected *Transaction and Protocol Analyzer* application (1/2/3 and so on) opens up and the selected signals' associated streams and transactions are loaded to it.

Note:

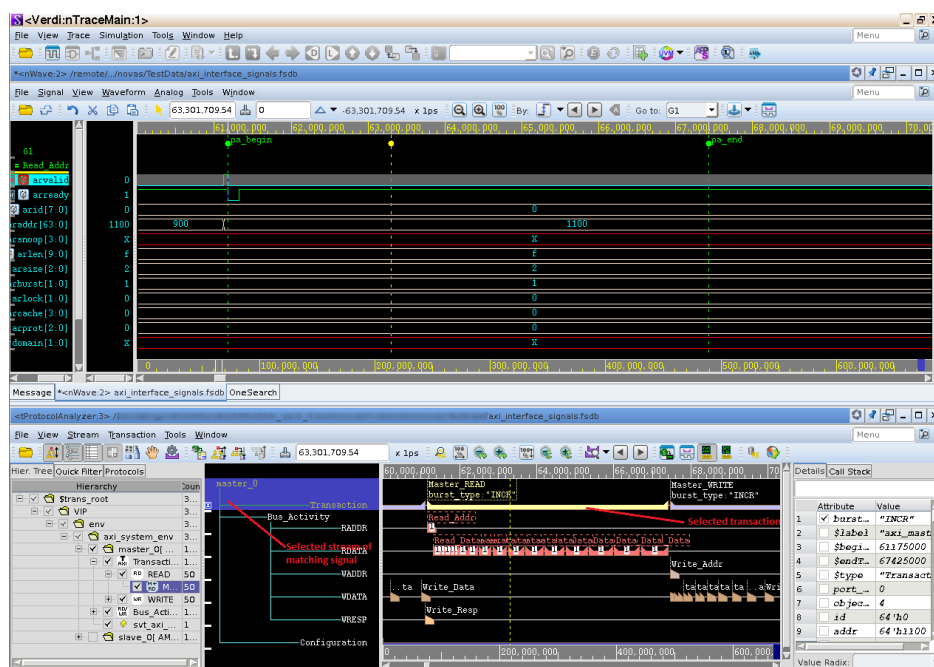
If the selected *Transaction and Protocol Analyzer* application loaded FSDB file is not recorded with the selected interface signal path from the *nWave* window, then the details are not updated in the selected application, instead a warning message is issued. Following is an example of the warning message: "Transactions are not recorded for interface path '/u/v/wx_yz[0]' in the loaded FSDB '/a/b/c/pcie.fsdb'"

Additionally, the following actions are performed on clicking the **Add Associated Transaction -> Protocol Analyzer X** option:

1. All streams are added to the *Waveform* pane under the instance hierarchy, recorded with same interface path for selected signal's interface path from the *nWave* window.
2. The new/chosen *Transaction and Protocol Analyzer* application view range is adjusted similar to that in the *nWave* window.
3. If the selected signal name from *nWave* is part of a signal group defined by the VIP and that signal group is also referred to any stream defined by the VIP, then those added streams channel is selected in the *Stream* pane in *nWave*.
4. If a cursor is added in the *nWave* window, then the new/chosen *Transaction and Protocol Analyzer* application also adds the cursor on time.
5. If the associated stream transactions of any of the selected signal are in between the cursor time, then it also selects first matching transactions.

The following figure illustrates the selected stream in the *Stream* pane and the transaction in the *Waveform* pane in the *nWave* window.

Chapter 4: Using Transaction and Protocol Analyzer Interlinking Between Transactions and Signals



All the opened *Transaction and Protocol Analyzer* applications are synchronized with the primary *nWave* window. Hence, changing the view range or selecting the cursor in primary *nWave* window also updates the view range or changes the cursor in all the opened *Transaction and Protocol Analyzer* applications if **nWave Sync View Range** or **nWave Sync All** option is selected.

The **Add Associated Transaction** option is not shown if any of the following condition is present:

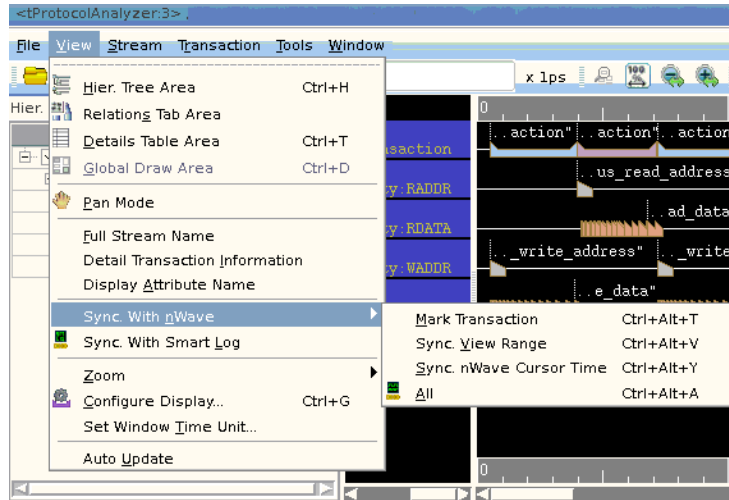
- More than one signal is selected in the *Signal* pane in *nWave*.
- The interface path is not recorded at instance level for VIP generated FSDB or stream/scope level for UVM generated FSDB file.
- The FSDB file generated for a VIP does not have any signal groups for selected signal name.

Synchronization With nWave

The cursor actions between *Transaction and Protocol Analyzer* and the primary *nWave* window can also be synchronized and you can check the transactions and HDL signals at the same time. *Transaction and Protocol Analyzer* provides the **View -> Sync. with nWave** command to synchronize transactions along with three sub-options - **Mark**

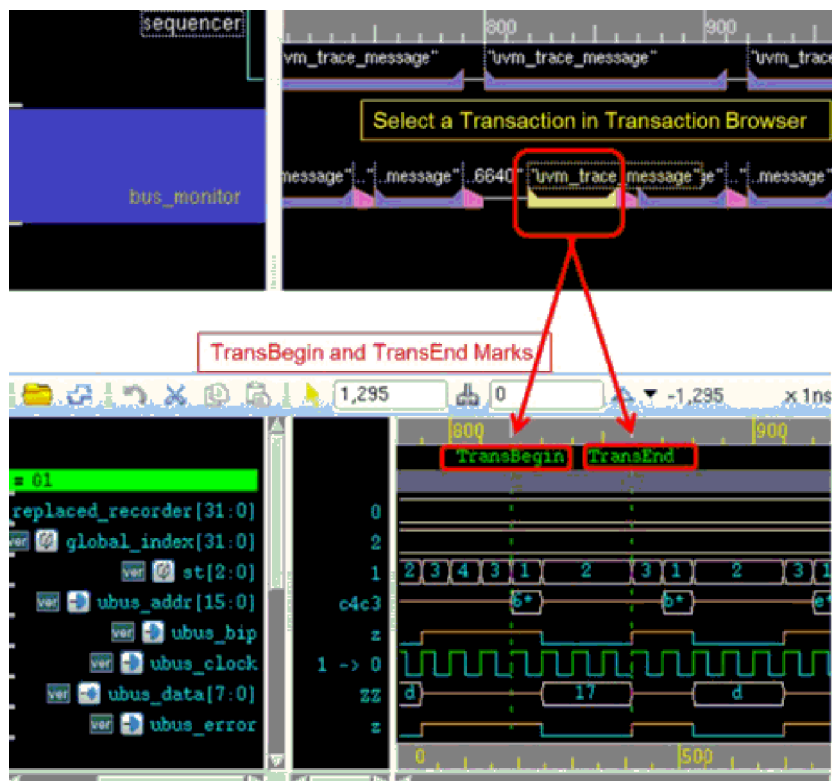
Transaction (Ctrl+Alt+T), **Sync. View Range** (Ctrl+Alt+V) and **Sync. nWave Cursor Time** (Ctrl+Alt+Y) as shown in the following figure:

Figure 46 Synchronizing Transaction and Protocol Analyzer with nWave



To see the duration of a transaction in the *nWave* window, enable the **View -> Sync. with nWave -> Mark Transaction** toggle command and select a transaction in the *Transaction and Protocol Analyzer* window. As shown in [Figure: Mark Transaction in nWave Window](#), the begin time and end time of the selected transaction in the *Transaction and Protocol Analyzer* window are displayed in the *nWave* window with green dashed lines to indicate with the **TransBegin** and **TransEnd** marks. You can clearly see the signal changes during the selected transaction. If the begin time is equal to the end time, there is only one **TransTime** mark.

Figure 47 Mark Transaction in nWave Window



The horizontal scroll bar and zoom ratio between the *Transaction and Protocol Analyzer* and the *nWave* windows can be synchronized after enabling the **View -> Sync. with nWave -> Sync. View Range** toggle command in the *Transaction and Protocol Analyzer* window. On moving the horizontal scroll bar in the *nWave* window and the scroll bar in the *Transaction and Protocol Analyzer* window also moves to the same time point and vice versa. **Zoom Out**, **Zoom In**, and **Zoom All Time** commands in the *Transaction and Protocol Analyzer* window are also synchronized with **Zoom Out**, **Zoom In** and **Zoom All** commands in the *nWave* window.

The cursor time between the *Transaction and Protocol Analyzer* and the *nWave* windows can be synchronized after enabling the **View -> Sync. with nWave -> Sync. nWave Cursor Time** toggle command in the *Transaction and Protocol Analyzer* window. On placing the cursor in the *nWave* window by left-clicking at a point, the cursor in the *Transaction and Protocol Analyzer* window is also placed at the same time point and vice versa. The only operation that requires particular attention is using the middle-click to place the cursor in the *Transaction and Protocol Analyzer* window. Zoom Cursor option is also supported in the *Transaction and Protocol Analyzer* window.

The following **Pan options** are supported in the *Transaction and Protocol Analyzer* window:

Pan Left, Pan Right, Pan Up, and Pan Down

The **Next view** action is also supported.

The **Highlight Mouse Hover Object** option is also supported in Transaction menu and the context menu of the displayed area. This option is selected by default. It highlights the transaction when mouse hovers the transaction.

Points to Note

The following points must be noted for the synchronization functionality behavior between nWave and *Transaction and Protocol Analyzer*:

- All *Transaction and Protocol Analyzer* applications synchronize only with the primary *nWave* window.
- Each *Transaction and Protocol Analyzer* application remembers the loaded signals in nWave and ensures the same signals are not loaded again from the same *Transaction and Protocol Analyzer* application.
- If any *Transaction and Protocol Analyzer* application is enabled with **Sync. View Range** or **Sync. All** option, the view range changes between the primary nWave and the *Transaction and Protocol Analyzer* application are synchronized.
- If any *Transaction and Protocol Analyzer* application is enabled with **Sync. nWave Cursor Time** or **Sync. All** option, cursor time selection between the primary nWave and the *Transaction and Protocol Analyzer* application is synchronized.
- If any *Transaction and Protocol Analyzer* application is enabled with **Mark Transaction** or **Sync. All** option, then selecting the transaction from *Transaction* pane marks `pa_begin` and `pa_end` markers in the primary *nWave* application.
- Any view range changes or cursor marker changes in other *nWave* (non-primary) application does not effect the *Transaction and Protocol Analyzer* application (even though those applications are created or signals loaded from the *Transaction and Protocol Analyzer* application).

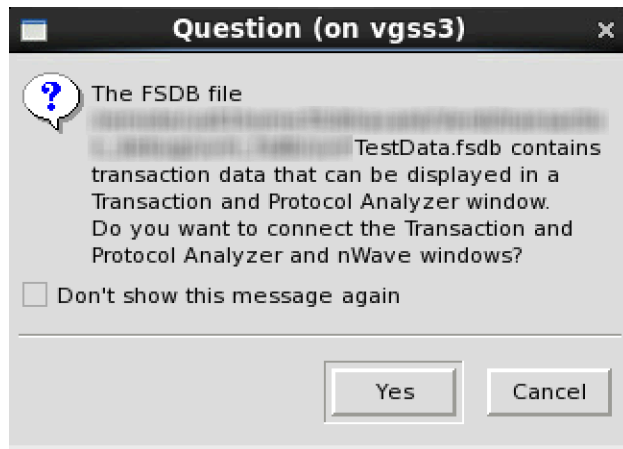
Connecting nWave and Transaction and Protocol Analyzer

When your FSDB file contains both transaction and signal levels, you can *automatically* connect Transaction and Protocol Analyzer and the primary *nWave* to debug your design with both waveforms together. The viewing range and cursor operations will be automatically synchronized.

Connecting Transaction and Protocol Analyzer from nWave

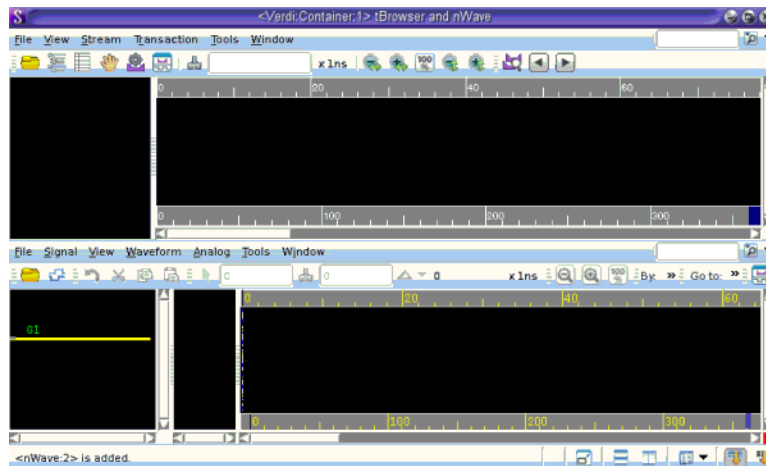
If you invoke the first *nWave* frame in the Verdi platform, and then use the **File -> Open** command load an FSDB file with transaction data into the *nWave* frame first time, you will be notified with the Question dialog (see [Figure: Automatically Connect to Transaction and Protocol Analyzer](#)). This question notifies that your FSDB file contains transaction data and the FSDB can be displayed in a Transaction and Protocol Analyzer window. It also can help you to automatically connect the Transaction and Protocol Analyzer and *nWave*.

Figure 48 Automatically Connect to Transaction and Protocol Analyzer



Click the **Yes** button to connect the *Transaction and Protocol Analyzer* and *nWave*. An undocked container appears. As shown in figure [Combined Transaction and Protocol Analyzer and nWave](#), it combines the two windows and the FSDB file is loaded both into Transaction and Protocol Analyzer and *nWave*.

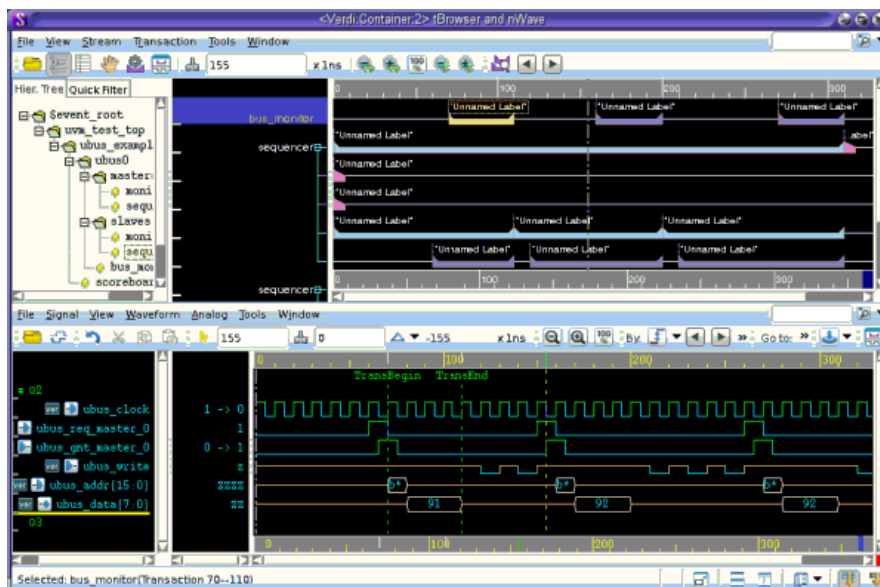
Figure 49 Combined Transaction and Protocol Analyzer and nWave



Transaction and Protocol Analyzer and *nWave* in the container have their own menu commands in the container. You can add streams in the Transaction and Protocol Analyzer and signals the *nWave* separately (see figure [Show Waveform in Container](#)).

All the synchronization features are automatically enabled. When the view range is scrolled, zoomed in/out horizontally in one window, the same action happens in the other window simultaneously. In addition, if one transaction is selected in Transaction and Protocol Analyzer, the **begin_time** and **end_time** will be marked in *nWave* (see section [Synchronization With nWave](#) for more details about the synchronization features).

Figure 50 Show Waveform in Container



You can click the  icon in the bottom to separate *Transaction and Protocol Analyzer* and *nWave* frames and undock both from the main window of Verdi.

When your FSDB file is not loaded in a standalone *nWave* frame in the first time, you can import the FSDB file and then use the **Window -> Connect Transaction and Protocol**

Analyzer and nWave command or click the  icon of the toolbar in the *nWave* frame to invoke a standalone container including both Transaction and Protocol Analyzer and *nWave*.

Connecting nWave from Transaction and Protocol Analyzer

If an FSDB file with signal level data is loaded in the Transaction and Protocol Analyzer, you can also use the **Window -> Connect Transaction and Protocol Analyzer and**

nWave command or click the  icon to open *nWave* and combine the two frames into a standalone window.

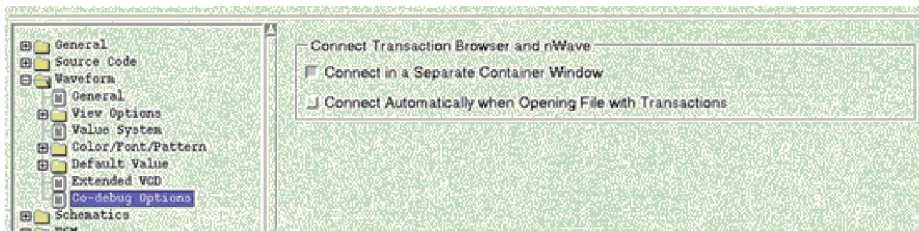
Configurations

You can set the behavior of the connection in the **Waveform -> Co-debug Options** page in the *Preferences* form (invoked with the **Tools -> Preferences** command).

As shown in figure [Co-debug Options](#), you can enable the following options for the connection of Transaction and Protocol Analyzer and *nWave*:

- **Connect in a Separate Container Window:** Connects the primary *nWave* and Transaction and Protocol Analyzer windows in a dock container. It is enabled by default.
- **Connect Automatically when Opening File with Transactions:** Connects the primary *nWave* and Transaction and Protocol Analyzer windows automatically when opening an FSDB file with transactions in *nWave*. The **Connect Transaction and Protocol Analyzer and nWave** command will be automatically enabled. The value of this option is disabled by default.

Figure 51 Co-debug Options



The following keyboard shortcuts are available for *nWave* synchronization actions:

Action	Keyboard Shortcut
Mark Transaction	Ctrl+Alt+T
Synchronize View Range	Ctrl+Alt+V
Synchronize nWave Cursor Time	Ctrl+Alt+Y
Synchronize All	Ctrl+Alt+A

Note:

These keyboard shortcuts are enabled only in the Transaction and Protocol Analyzer mode.

5

Using Cache and Memory Protocol Analyzer (Memory Mode)

Synopsys Memory VIPs are based on Synopsys SystemVerilog Technology (SVT) infrastructure that includes SystemVerilog classes and Memory Server. The Memory Server is one of the key elements of SVT Memory VIP infrastructure because the actual memory values are stored in the Memory Server instead of a SystemVerilog array. The primary advantage of using this approach is that the performance of the Memory VIP does not rely on the quality of the implementation of the equivalent sparse array server of each simulator. For multi-GB memories, the simulation typically accesses only a small portion of the overall address space. In addition, this approach supports virtual patterns which enable memory initialization without incurring any significant memory or performance overhead.

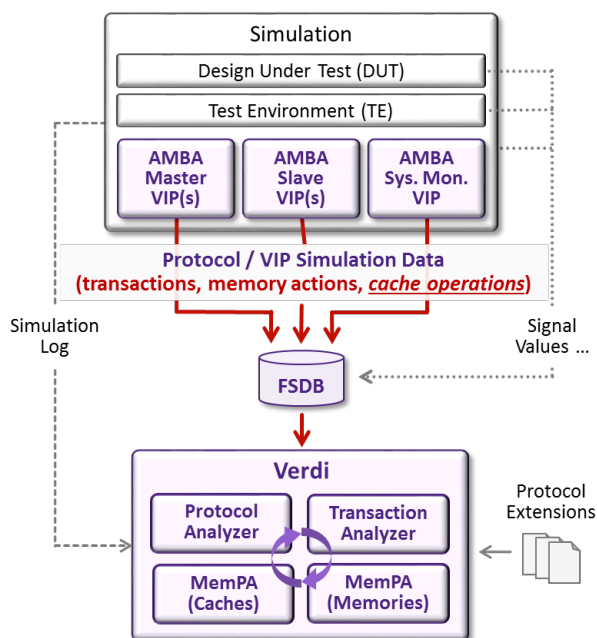
In a Memory VIP, protocol-level transactions and memory actions are stored in an FSDB file. The Memory Server can be configured to record all memory actions to the same FSDB file to view the memory-related data in the *Verdi Cache and Memory Protocol Analyzer*. With the information recorded by the Memory Server, you can determine the value of any memory address at any time during simulation.

The SVT Memory Server has been enhanced to store the memory actions associated with the server in an FSDB file as transactions. The streams of these transactions are tagged with the hidden attributes so that the memory instances can be recognized by the Verdi environment and viewed in the new Memory Array browser. In addition, you can view the memory actions of the Memory Server in *Transaction Table View* or *Transaction and Protocol Analyzer* windows.

For performance considerations, each instance of a Synopsys Memory VIP must be configured to dump data to the FSDB file. By default, this capability is not enabled. Synopsys Memory VIPs provide controls for dumping protocol-level transactions, the status of internal state machines, and the memory actions performed by the Memory Server.

The following figure illustrates the high-level flow for *Cache and Memory Protocol Analyzer*:

Figure 52 High-Level Flow for Cache and Memory Protocol Analyzer



Note:

To enable and use the *Cache and Memory Protocol Analyzer (Cache mode)* in Verdi, it is mandatory to specify the `-lca` option during compilation. For usage details, see Verdi and Siloti LCA Features Guide.

The following sections describe the *Cache and Memory Protocol Analyzer (Memory Mode)* application supported by Verdi:

- [Cache and Memory Protocol Analyzer Overview](#)
- [Invoking Cache and Memory Protocol Analyzer](#)
- [Cache and Memory Protocol Analyzer Window Overview](#)
- [Synchronization With Verdi Transaction and Protocol Analyzer](#)
- [Synchronization With Transaction Table View](#)

Cache and Memory Protocol Analyzer Overview

Cache and Memory Protocol Analyzer provides the following features:

- Viewing the values and most recent memory actions for a specified address range at a specified time in a 2-D array format.
- Viewing the attributes associated with a specific address at a specific time.
- Viewing a list of memory actions associated with a specific address up to a specific time.
- Viewing a list of all memory actions and the associated attributes.
- The ability to correlate a Memory VIP protocol-level transaction with one or more addresses at a specific time in the 2-D array of an address range or with one or more specific memory actions in the list view of memory actions.

Note:

The *Cache and Memory Protocol Analyzer* feature is dependent on the Verdi FSDB SystemVerilog Writer API.

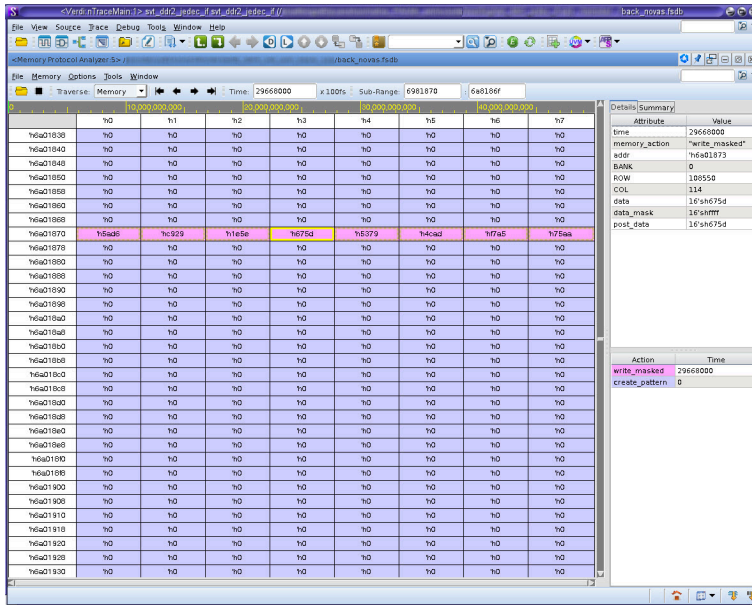
Invoking Cache and Memory Protocol Analyzer

The recommended flow to launch *Cache and Memory Protocol Analyzer* is as follows:

1. To invoke *Cache and Memory Protocol Analyzer* in Verdi, click the **Tools -> Transaction Debug -> Cache and Memory Protocol Analyzer** menu command in the Verdi GUI window.
2. If the FSDB file has not been loaded into Verdi, click the  button or select the **File -> Open** command to import the FSDB file.
3. Click the  button to select a memory instance.

The *Cache and Memory Protocol Analyzer* window appears as shown in the following figure:

Figure 53 Cache and Memory Protocol Analyzer Window



Cache and Memory Protocol Analyzer Window Overview

Cache and Memory Protocol Analyzer window in *Memory Mode* comprises of the following panes/views:

- [Memory Array Pane](#)
- [Details Pane](#)

Memory Array Pane

The *Memory Array* pane is the main view for examining memory address values. The *Memory Array* pane displays the value of an address location at any particular time. The displayed columns of the Memory Array can be modified using the **Options -> Column count** command.

There are three traverse modes to navigate through the Memory Array: **Memory**, **Selected**, and **Sub-range**. Select a mode by clicking the **Traverse** selection box.

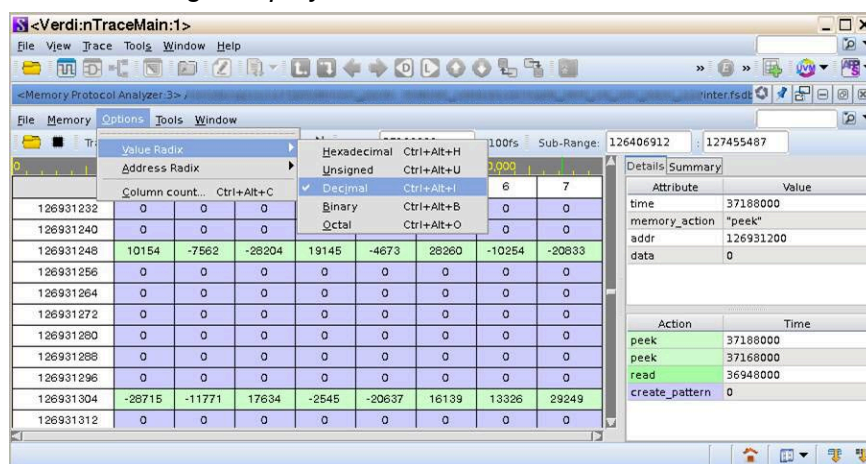
Chapter 5: Using Cache and Memory Protocol Analyzer (Memory Mode) Cache and Memory Protocol Analyzer Window Overview

- In the **Memory** mode, you can traverse through memory actions of the complete Memory Array.
- In the **Selected** mode, you can traverse through memory actions for the selected address.
- In the **Sub-range** mode, specify the address range of interest, and you can traverse through memory actions within the specified address range.

After a traverse mode is selected, navigate through memory actions by clicking on the  icon for the first memory action, the  icon for the previous memory action, the  icon for the next memory action, and the  icon for the last memory action.

You can change the display radix for data values using the **Options -> Value Radix** command.

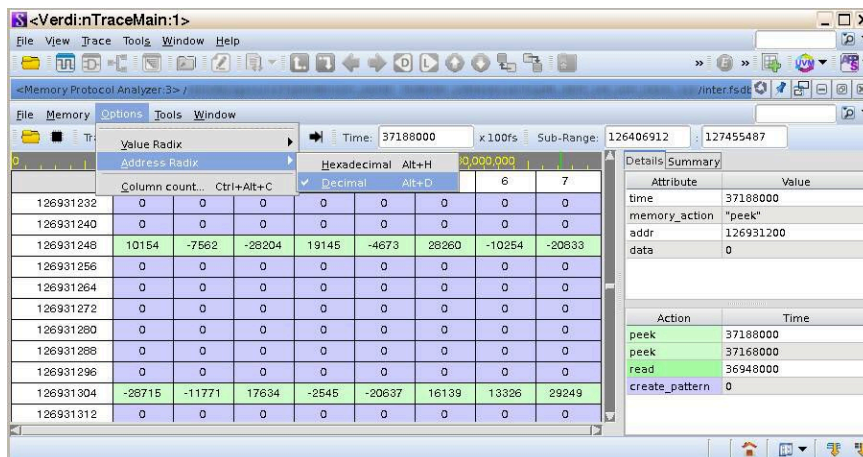
Figure 54 Change Display Radix for Data Value with Value Radix Command



You can change the display radix for address values using the **Options -> Address Radix** command.

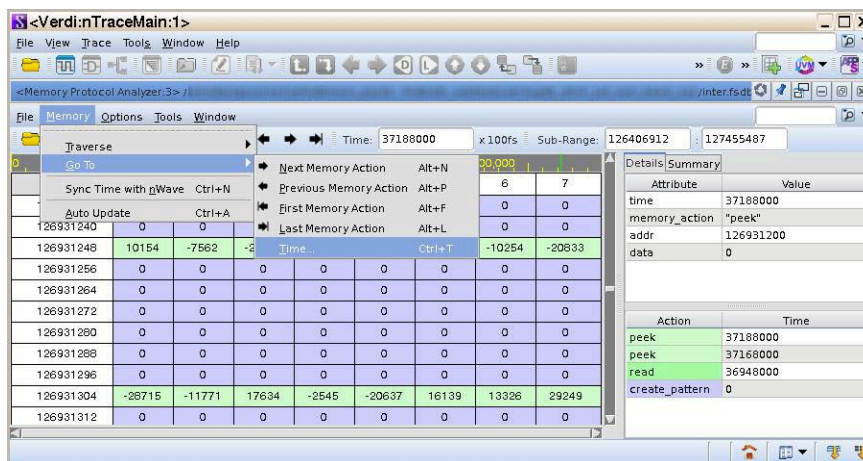
Chapter 5: Using Cache and Memory Protocol Analyzer (Memory Mode) Cache and Memory Protocol Analyzer Window Overview

Figure 55 Change Display Radix for Address Value with Address Radix Command



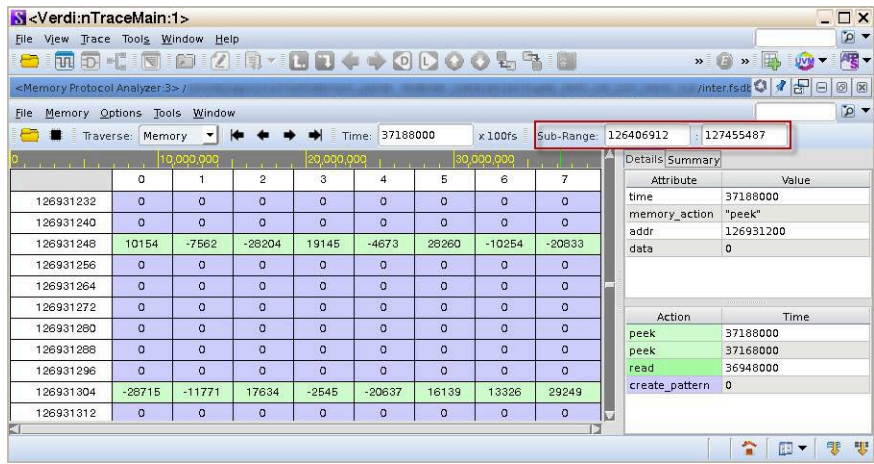
You can traverse to a specific simulation time using the **Memory -> Go To -> Time** command or by entering the simulation time in the **Time:** field in the *Cache and Memory Protocol Analyzer* toolbar.

Figure 56 Traverse to Specific Simulation Time with Go To -> Time command



You can specify an address range to view or traverse by entering the start address and end address in the **Sub-Range** fields in the *Cache and Memory Protocol Analyzer* toolbar.

Figure 57 Specify Address Range by Entering Start and End Addresses



Details Pane

The *Details* pane comprises of the *Details* tab and the *Summary* tab.

Details Tab

The top portion of the *Details* tab displays the attributes and values associated with the most recent memory action for the selected memory address. The bottom portion of the *Details* pane displays all of the memory actions that have been applied to the selected address up to the specified time.

Figure 58 Details Tab

Attribute	Value
time	31332000
memory_action	"write_masked"
addr	'h6a0186b
BANK	0
ROW	108550
COL	104
data	16'shad4b
data_mask	16'shffff
post_data	16'shad4b
.....	
Action	Time
write_masked	31332000
create_pattern	0

Summary Tab

The *Summary* tab displays basic information about the memory instance being viewed.

Figure 59 Summary Tab

Details Summary	
Action Type	Count
Address width	30
Data width	16
Logic states	2
Access type	read-write

Synchronization With Verdi Transaction and Protocol Analyzer

If the Memory VIP has been configured to capture both protocol-level transactions and the memory actions performed by the Memory Server, high-level memory read and write operations can be correlated with the corresponding low-level Memory Server operations by enabling selection synchronization between the two windows.

To invoke *Transaction and Protocol Analyzer*,

- Click the **Tools -> Transaction Debug -> Transaction and Protocol Analyzer** command in the Verdi GUI window.

Or

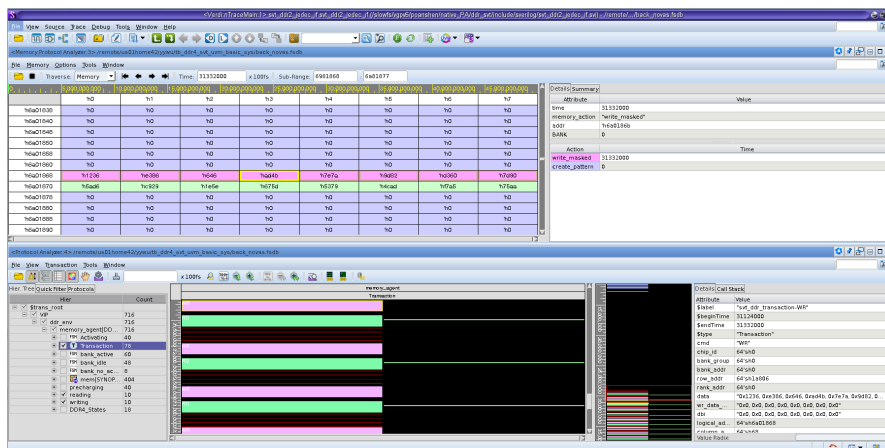
- Click the **Tools -> New Transaction and Protocol Analyzer Window** in the *Cache and Memory Protocol Analyzer* window.

To synchronize *Transaction and Protocol Analyzer* with *Cache and Memory Protocol Analyzer*, click on the synchronize selection  icons on the toolbar of both *Transaction and Protocol Analyzer* and *Cache and Memory Protocol Analyzer* windows. The 

Chapter 5: Using Cache and Memory Protocol Analyzer (Memory Mode) Synchronization With Transaction Table View

icon indicates that selection synchronization is enabled. The following figure shows synchronization between both the windows:

Figure 60 Correlating Memory Transaction with Memory Actions



Synchronization With Transaction Table View

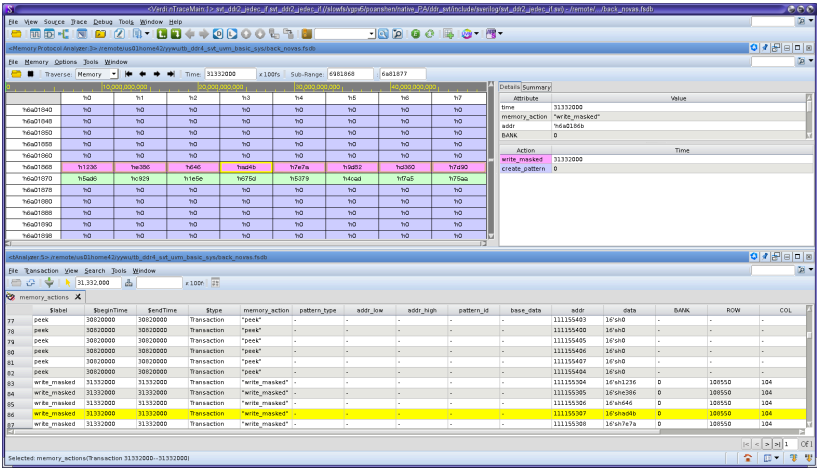
To open a *Transaction Table View* window, use the **Tools -> New Transaction Table View** command in the *Cache and Memory Protocol Analyzer* window.

Click on the  synchronize signal selection icons on the toolbar of both *Transaction Table View* and *Cache and Memory Protocol Analyzer* windows. The  icon indicates that selection synchronization is enabled. The following figure shows synchronization of *Cache and Memory Protocol Analyzer* with *Transaction Table View*.

You can search for specific memory actions and/or address values using the search capabilities of the *Transaction Table View*.

Chapter 5: Using Cache and Memory Protocol Analyzer (Memory Mode) Synchronization With Transaction Table View

Figure 61 Synchronization of Cache and Memory Protocol Analyzer with Transaction Table View



Note:

All the operations mentioned above can be used during interactive mode.

6

Using Transaction Table View

The following sections describes the usage of the *Transaction Table View*:

- Opening Transaction Table View
- Adding Items to Table View
- Exporting the Table Content

The *Transaction Table View* window enables you to view multiple activities in a single window with ease and review related transactions with all the attributes in a table view.

Figure 62 *Transaction Table View Window*

[illegible]

Opening Transaction Table View

You can use the **Tools -> New Transaction Table View** command in the *Transaction and Protocol Analyzer* window to open the *Transaction Table View* window. The hierarchy tree from the loaded FSDB file is shown in the left pane. You can add a stream or scope by directly selecting it from the hierarchy tree.

Figure 63 Transaction Table View Window When You First Open It

The screenshot shows the Qt Creator IDE with a Qt Console window. The console displays the output of a QtWebView widget, which is a web application. The web application has a title bar '<TableView:4> (on odclegacy0141)' and a menu bar with 'File', 'Tools', 'Window', and 'Help'. The main content area is divided into two panes: 'HierTree' and 'Quick Filter'.

The 'HierTree' pane shows a hierarchical tree structure of a database schema. The tree is rooted at 'strans_root' (1008). It has a child 'VIP' (1008), which has a child 'env' (1008). 'env' has a child 'apb' (559), which has a child 'mon' (559). 'mon' has a child 'Transaction' (559), which has two children: 'READ' (414) and 'WRITE' (145). 'Transaction' also has a child 'vip' (449). 'vip' has a child 'Symbol' (449), which has a child 'DATA' (256). 'DATA' has two children: 'Tx' (128) and 'Rx' (128). 'DATA' also has a child 'DON'T_CARE' (36). 'DON'T_CARE' has two children: 'Tx' (18) and 'Rx' (18). 'DON'T_CARE' also has a child 'ESC' (2). 'ESC' has a child 'Tx' (2). 'ESC' also has a child 'IDLE' (90). 'IDLE' has two children: 'Tx' (44) and 'Rx' (46). 'IDLE' also has a child 'SYNC' (65). 'SYNC' has two children: 'Tx' (33) and 'Rx' (32).

The 'Quick Filter' pane shows a table with columns 'label', '\$beginTime', '\$endTime', '\$type', 'addr', and 'chan'. The table is currently empty.

At the bottom of the console window, there is a status bar that says 'TableView_4 be window.'

In the *Transaction Table View* window, the transactions of the selected streams or scopes from the hierarchy tree are listed in a table view format.

Adding Items to Table View

You can click the plus (+) icon in the hierarchy view item to expand the tree node and select a node, either the scope or a stream, to add it to the right pane.

Figure 64 Adding Items to Table View

Check box to create new tab in Table view

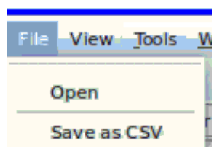
The transactions belonging to the selected item in the tree opens in the right pane, with all the streams available under that selected item.

Figure 65 Transaction Table View Window After Selecting a Node

HierTree	Quick Filter	Count	Tab 1
\$trans_root		1008	apb_write_transaction 690450 690550 Transact
VIP		1008	apb_write_transaction 691050 691150 Transact
env		1008	
apb		559	apb_write_transaction 691650 691750 Transact
mon		559	apb_write_transaction 692250 692350 Transact
Transaction		559	apb_write_transaction 692850 692950 Transact
READ		414	apb_write_transaction 804050 804150 Transact
WRITE		145	apb_write_transaction 804650 804750 Transact
vip		449	apb_write_transaction 805250 805350 Transact
Symbol		449	apb_write_transaction 805850 805950 Transact
DATA		256	apb_write_transaction 806450 806550 Transact
Tx		128	apb_write_transaction 807050 807150 Transact
Rx		128	apb_write_transaction 807650 807750 Transact
DON'T CARE		36	apb_write_transaction 808250 808350 Transact
Tx		18	apb_write_transaction 808850 808950 Transact
Rx		18	apb_write_transaction 893750 893850 Transact
ESC		2	
Tx		2	
IDLE		90	
Tx		44	
Rx		46	
SYNC		65	
Tx		33	
Rx		32	

Exporting the Table Content

You can export the table content to CSV format by selecting either the hierarchy or the table page. The export to CSV option is available under the **File -> Save as CSV** command.



7

Using Transaction Relation Navigator

The *Transaction Relation Navigator* window provides a detailed spreadsheet-like viewer for relationships and a cockpit for navigating and tracing relationships between transactions. The spreadsheet shows the transactions with relation hierarchy. Users can more easily and clearly view the relations.

The following sections describe the usage of *Transaction Relation Navigator*:

- [Invoking the Transaction Relation Navigator Window](#)
- [Transaction Relation Navigator Window](#)
- [Analyzing Transaction Relationships in Transaction Relation Navigator](#)

Invoking the Transaction Relation Navigator Window

When you use *Transaction and Protocol Analyzer* or *Transaction Table View* windows and are interested in viewing the detailed relationship between transactions (including relationship hierarchy, relationship types and all other attributes), then you can use the *Transaction Relation Navigator* window. Select the **Tools -> New Transaction Relation Navigator Window** command to invoke the *Transaction Relation Navigator* window. Alternatively, you can directly double-click the **Relationship** column of a transaction (or right-click and select the **Show Transaction Relation Navigator** command) in the *Transaction Table View* window. The new *Transaction Relation Navigator* window opens with the transaction and the relationships of this transaction.

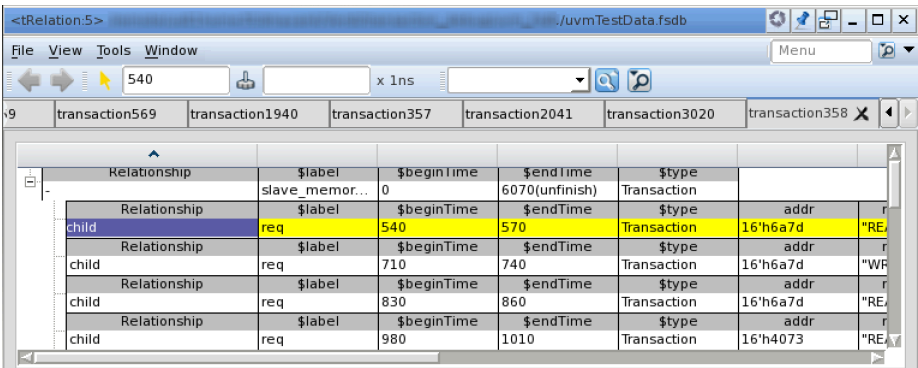
Transaction Relation Navigator Window

The *Transaction Relation Navigator* window displays all selected transactions (selecting multiple transactions is supported in the *Transaction Table View* window) in each tab and all its relationships of the transaction per tab. The features are as follows:

- Each tab contains a root node and two kinds of relationship groups: the upper is the **Hierarchical Relations** and the lower is the **Other Relations**.
- Each row shows a transaction and the columns are all the normal attributes and visible predefined attributes.
- The selected transaction gets highlighted in yellow (the cyan field occurs in the field where the cursor was last selected), The *Status* bar shows the selected transaction and the marked row is highlighted in red.

See [Figure: Transaction Relation Navigator Window Overview](#), the transaction in the second row is the *child* of the transaction in the first row.

Figure 66 Transaction Relation Navigator Window Overview



When the attributes of the transaction have expected value, the row of the transaction is highlighted in magenta. Additionally, the attributes are shown with the expected value suffix in the column.

Figure 67 Attributes with Expected Values

The screenshot shows a table with columns: id, ep, length, payload, status, completion_id, completion_status, byte_count, lower_address, and crc. The row for 'transaction2260' is highlighted in magenta. A red callout points to the 'payload' cell, which contains the text 'Attribute with Expected Value'.

id	ep	length	payload	status	completion_id	completion_status	byte_count	lower_address	crc
1'h1	1'h0	4096'h20	*e2b32e5b01	*GOOD*	4096'h100	4096'h0	1'h0	4096'h165	4096'h9d444
command_num	address_translation								
3	4096'h0								

By default, all the attribute names are displayed inside the cell, on top of the attribute value as shown in the following figure:

Chapter 7: Using Transaction Relation Navigator Transaction Relation Navigator Window

transaction2152						
Relationship	Attribute Name	Attribute Value	xfer type	device address	endpoint number	intended byte
Relationship	device connected bus speed	speed	ISOCHRONOU...	124	7	1024
child	-	-	xact_type	payload_byte_cou...	endpoint_number	device_address
Relationship	33525200	\$beginTime	"IN_TRANSCAC...	1024	7	124
child	-	-	\$endTime	\$label	\$type	channel
Relationship	33525200	channel	33563200	packet_ss_dp	Transaction	"Tx"
child	"Tx"	-	speed	pkt_type	device_address	ept_num
child	-	-	-	-	-	-

You can change the alignment setting for the table cells in the *Transaction Relation* page under the *Transaction Debug* folder in the *Preferences* form opened by the **Tools -> Preferences** command.

To change the default layout and view the attributes aligned by columns, enable the **Align Transaction Attributes by Columns** preference option in the *Transaction Relation* page of *Transaction Debug* folder (invoked by **Tools -> Preferences** option).

The screenshot displays the Transaction Relation Navigator window. At the top, a timeline shows transaction durations with labels like '0', '210', and 'x 1ns'. Below the timeline, a table lists transaction relationships. A red dashed box highlights the 'Root Node Transaction' (transaction212). A red arrow points to the 'Marked Transaction' (transaction212). A blue arrow points to the 'Selected Transaction' (transaction212). The table has columns: Relationship, \$label, \$beginTime, \$endTime, Type, and filename.

Relationship	\$label	\$beginTime	\$endTime	Type	filename	
Relationship	uvrm_trace_message	0	350	"transaction"	"../../../../src/base..."	
child	uvrm_trace_message	0	110	"transaction"	"../../../../src/base..."	
child	uvrm_trace_message	110	180	"transaction"	"../../../../src/base..."	
child	uvrm_trace_message	210	350	"transaction"	"../../../../src/base..."	
Other Relations	parent	uvrm_trace_message	0	1810	"transaction"	"../../../../src/base..."

All the VIP attributes are displayed based on the pre-defined order decided by the VIP. The ordering of the VIP attributes displayed in the tab is as follows:

- Firstly, the predefined attributes in the XML file with the `field_order` keyword are displayed.
- Other attributes' order follows the current column configuration defined using the **View -> Configure Columns** command.

Analyzing Transaction Relationships in Transaction Relation Navigator

This section contains the following sub-sections:

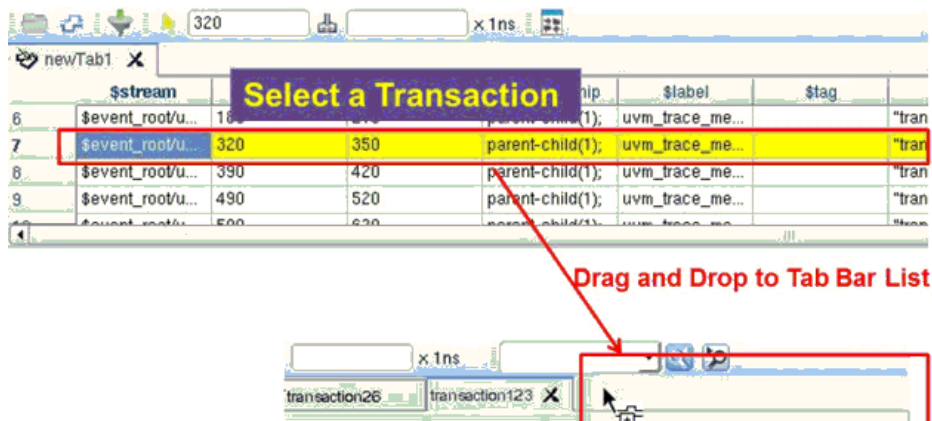
- [Importing Transactions into Transaction Relation Navigator](#)
- [Relationship Types](#)
- [Viewing Operations](#)

Importing Transactions into Transaction Relation Navigator

After invoking a *Transaction Relation Navigator* window, there are several methods provided to add other transactions into the different tabs of the same *Transaction Relation Navigator* window:

- Drag and drop the transaction from *Transaction and Protocol Analyzer* or *Transaction Table View* windows to the tab bar list (as shown in [Figure: Drag and Drop a Transaction to Transaction Relation Navigator](#)). If dropping the transaction to an existing tab label, this tab is replaced with the dropped transaction.
- Double-click the *Relationship* column of a transaction in the *Transaction Table View* window.
- Right-click the *Relationship* column of a transaction in the *Transaction Table View* window and select the **Show Transaction Relation Navigator** command.

Figure 68 Drag and Drop a Transaction to Transaction Relation Navigator



- Right-clicking the row of a transaction in the *Transaction Relation Navigator* window and selecting the **Open in Current Tab** command replaces the current tab with the selected transaction.

- Right-clicking the row of a transaction in the *Transaction Relation Navigator* window and selecting the **Open in New Tab** command opens a new tab with the selected transaction. If the transaction is already a root node in an existing tab, using this command switches to the existing tab.

Selecting a transaction in the *Transaction and Protocol Analyzer* window and using the **Show Transaction Relation Navigator** right-click command invokes a new *Transaction Relation Navigator* window.

Relationship Types

Each tab contains a transaction Hierarchical Relations group and an Other Relations group.

- **Hierarchical Relations:** Displays all relationship (includes user-defined type) types of the transaction in the root node. The relationships are listed in descending hierarchical order by default:
 - child
 - belong-to
 - annotate
- **Other Relations:** Displays the non-hierarchical relationships, including parent, owner, predecessor, successor, annotator, and user-defined types with flatten view.

Viewing Operations

The following sections describe some operations in the *Transaction Relation Navigator* window to let you view the relationship between transactions.

Synchronization

The selection action between the *Transaction Debug* windows can be synchronized by clicking the  icon on the right upper position of the current window or using the **View -> Sync. All Signal Selection** command in the main window. When a transaction is selected in one window in synchronization mode, the same transaction in other windows in synchronization mode gets highlighted as well.

Active Tab

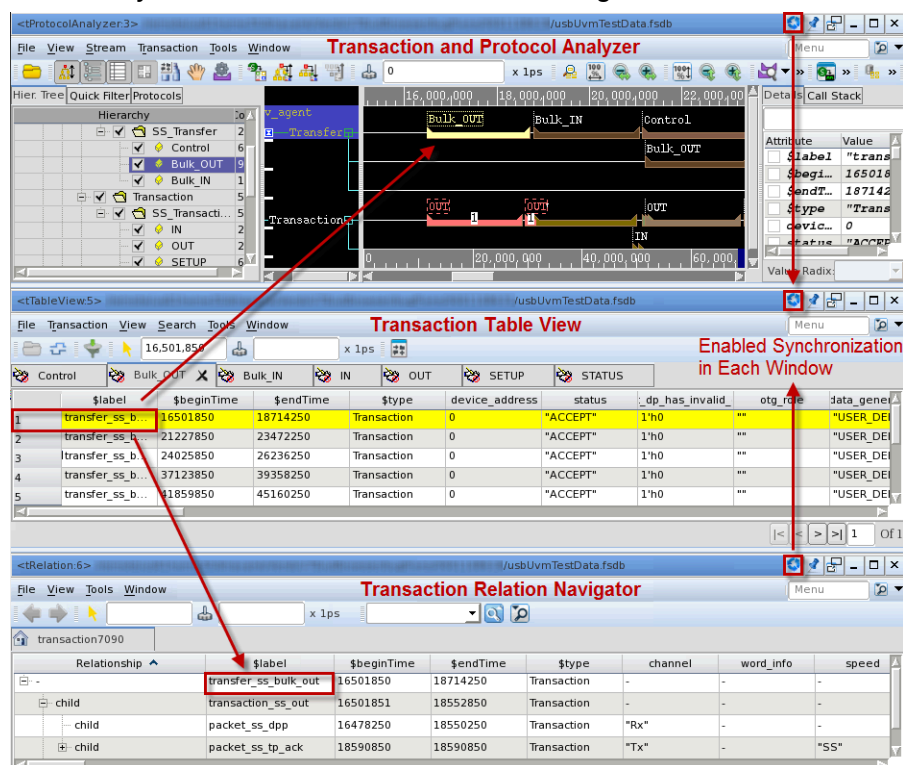
The first tab in the *Transaction Relation Navigator* window is always the active tab and cannot be closed. Double-clicking on the active tab label duplicates the content of this tab to a new tab. If the active tab is closed by the **File -> Close Current Tab** command, the next tab becomes the active tab. If the *Transaction Relation Navigator* window only

Chapter 7: Using Transaction Relation Navigator

Analyzing Transaction Relationships in Transaction Relation Navigator

has one active tab, this command is disabled. The active tab automatically switches to the selected transaction that is selected in another window if the windows are in the synchronization mode. For example, as shown in figure [Synchronization in Transaction Debug Windows](#), when you select a transaction in the *Transaction Table View* window, the transaction in the *Transaction and Protocol Analyzer* window gets highlighted in yellow and the active tab in the *Transaction Relation Navigator* window gets replaced with the contents of this transaction relationship.

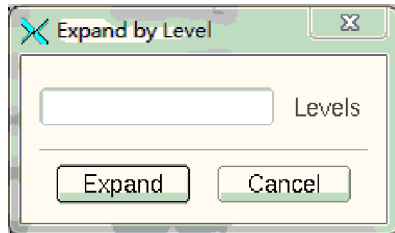
Figure 69 Synchronization in Transaction Debug Windows



- **Backward/Forward History:** The **Backward History** command is provided if the root node of the current tab has been changed. You can click the  icon to change the root node back to be the previous one. The **Forward History** command is also provided by clicking the  icon to change the root node to be the next one.
- **Search:** You can search all the transactions or attributes in the current tab using the **View -> Find** command to locate the entry that contains a partial matching string value. See [Figure: Find String](#), if the **All Tabs** option is enabled, the searching is performed through all transaction tabs.
- **Expand Tree by Level / Collapse All Transactions:** The hierarchy of the root node transaction is expanded one level by default. You can click the  icon to expand

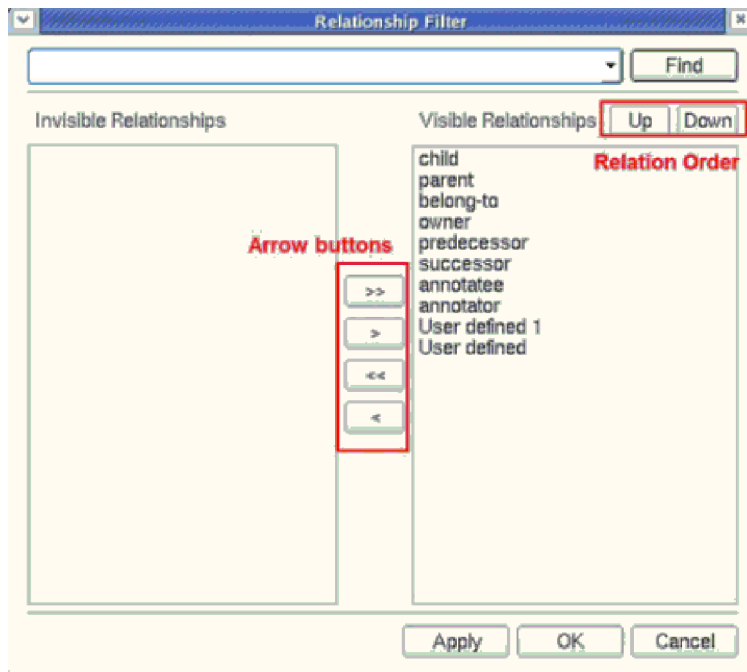
one level or use the **View -> Expand Tree by Level** command (as shown in figure [Expand by Level](#)) to expand multiple levels at one time. The **View -> Collapse All Transactions** command and the  icon are also provided to collapse the levels.

Figure 70 *Expand by Level*



- **Filter Relationships (Row Configuration):** By default, after creating the tabs, all the relationship types are visible. You can decide what types are shown using the **View -> Filter Relationships** command. As shown in [Figure: Relationship Filter](#), the *Relationship Filter* form displays all relationship types in the **Visible Relationships** field. Refer to the [Configuring Display](#) section for the usage of the arrow buttons and relation order.

Figure 71 *Relationship Filter*



- **Attributes Configuration:** After creating the tabs, there are some attributes listed by columns by default. You can use the **View -> Configure Columns** command to open the *Configure Columns* form and select the content. See the [Column \(Attribute\) Operations](#) section for details.
- **Display Other Relations:** The other relations group is not shown by default. You can display it by enabling the **View -> Display Other Relations** toggle command.
- **Display Class Definition:** If a transaction has the `class_name` attribute, you can use the **Display Class Definition** command in the right-click menu to trace the class definition (see the [Display Class Definition](#) section for details).

8

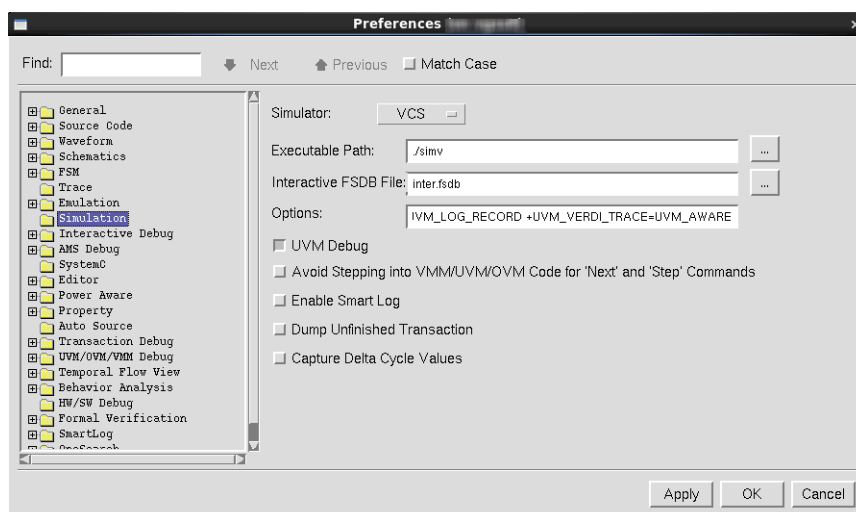
Performing Interactive Simulation Debug

You can also use the Interactive Simulation Debug solution (refer to *Verdi_SVTB_Interactive_Debug.pdf* for details) in the Verdi transaction debug platform to debug your design.

Use the following steps to enable Interactive Simulation Debug mode:

1. Invoke the *Preferences* form using the **Tools -> Preferences** command to set the simulation configurations. Select the **Simulation** page.

Figure 72 Simulation Preference



2. Select the simulator in the **Simulator** field and specify the executable path in the **Executable Path** field. Add the +UVM_VERDI_TRACE runtime option with +UVM_TR_RECORD or +UVM_LOG_RECORD runtime options in the **Options** field. Also add other options in demand. Click the **OK** button in the *Preferences* form.

Figure 73 Configure the Simulation

Simulator: VCS

Executable Path: simv ...

Interactive FSDB File: inter.fsdb ...

Options: pts uvm +UVM_LOG_RECORD +UVM_TR_RECORD +UVM_ ☐ No Append

☒ UVM Debug

☐ Avoid Stepping into VMM/UVM/QVM Code for 'Next' and 'Step' Commands

3. If you want to dump unfinished transaction attributes, select the **Dump Unfinished Transaction** preference. By default, the value of this option is *off*.

Figure 74 Dump Unfinished Transactions Option

Simulator: VCS

Executable Path: ./simv ...

Interactive FSDB File: inter.fsdb ...

Options: -HIER +verdi_recorder_debug +fsdb+event_dump_unfinished

☐ UVM Debug

☐ Avoid Stepping into VMM/UVM/QVM Code for 'Next' and 'Step' Commands

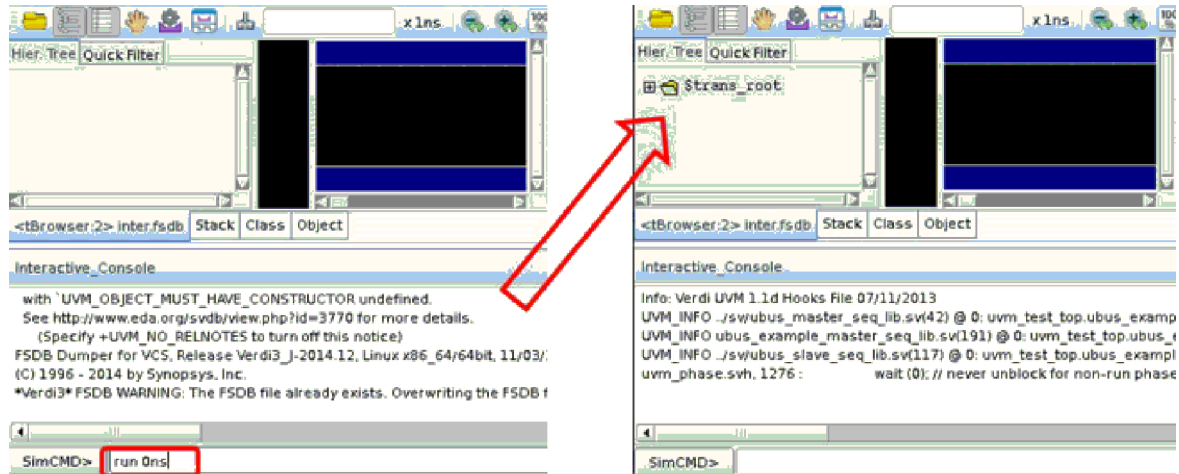
☐ Enable Smart Log

☒ Dump Unfinished Transaction

You can also use the command line option `-dumpUnfinishedTrans`, to dump unfinished transaction attributes. By default, the value of this option is *off*.

4. Use the **Tools -> Run Interactive Mode** command to enable the Interactive Simulation Debug mode. The *Hierarchy Tree* pane in the *Transaction and Protocol Analyzer* window is empty while Interactive mode is enabled.

5. To generate the hierarchy of the recorder module, you need to run the simulation a while to have the recording task been called. For example, execute the `run <duration>` command in the *Interactive Console* pane of the Verdi main window.

Figure 75 *Interactive Console*

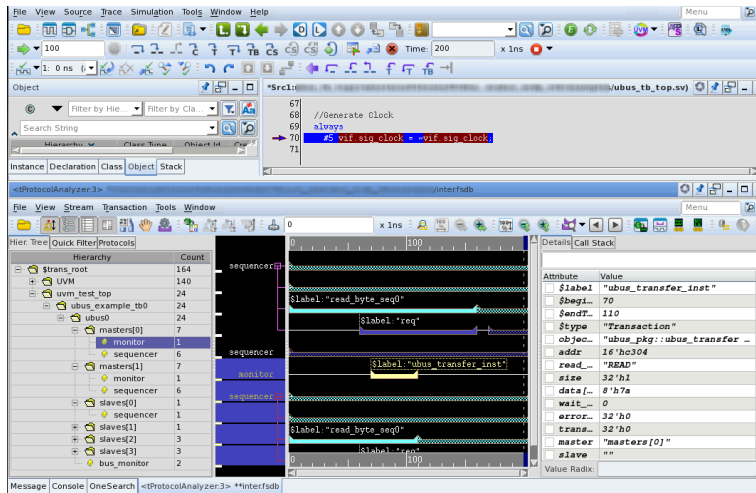
The streams are shown in the hierarchy tree when they are created completely.

The *Transaction and Protocol Analyzer* window automatically refreshes data when the simulator stops. The *Transaction* window can be auto refreshed using interactive mode functions: next, step, save/restore state, checkpoint join, and reverse-debug as shown in [Figure: Interactive Mode Functions](#).

Figure 76 *Interactive Mode Functions*

Transaction Debug features described in the previous sections are available. After adding streams into the *Waveform* pane of the *Transaction and Protocol Analyzer* window, the transactions are generated based on the advanced simulation time. The following figures show an example of the transaction generation based on the time advance in the *Transaction and Protocol Analyzer* window. If the hierarchy tree grows as the simulation advances, the hierarchy tree is updated accordingly.

Figure 77 Run 100ns



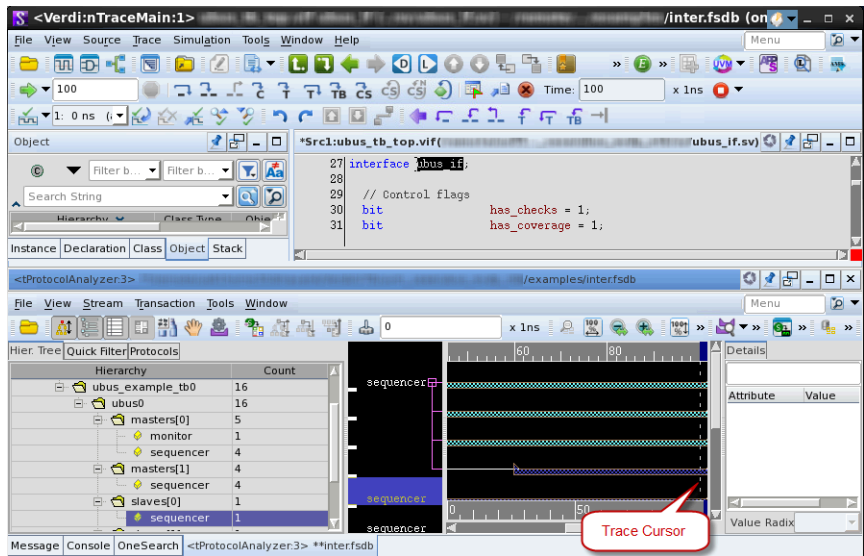
The information listed in the non-editable tabs of the *Transaction Table View* window is also updated accordingly. The hierarchy tree shown in the *Add Transactions* form is also refreshed. If the transaction is unfinished, the **<end time>** of the transaction is suffixed with **(unfinish)** in the **endTime** column.

Figure 78 Updated Transaction Status

	\$label	\$beginTime	\$endTime	\$type
1	slave_memor...	0	0(unfinish)	Transaction
2	slave_memor...	0	0	Message
3	req	80	110	Transaction

The visualization of an **unfinished** transaction is the same as **finished** transaction; however, it is displayed in a different pattern as shown in [Figure: Run 100ns](#). The attributes for **unfinished** transaction are also displayed in the **Detail** pane. The *Transaction and Protocol Analyzer* window also provides a trace cursor which shows that the trace has stopped for the current simulation as shown in [Figure: Trace Cursor](#).

Figure 79 Trace Cursor



When an unfinished FSDB is opened in the *Transaction and Protocol Analyzer*, its window title FSDB name is prefixed by "***", for example, *<tProtocolAnalyzer:2> **inter.fsdb* as shown in [Figure: Trace Cursor](#).

If the relationship of the selected transaction exists, you can use the **Show Transaction Relation Navigator** right-click option in *Transaction and Protocol Analyzer* and *Transaction Table View* windows to show the current relations in the *Transaction Relation Navigator* window. If the transaction is unfinished, the *<end time>* of the transaction is prefixed with *(unfinish)* in the *endTime* column.

If the relationship changes during the simulation, the information in the *Transaction Relation Navigator* is updated accordingly.

Figure 80 Transaction Relationship Update

transaction8				
Relationship	\$label	\$beginTime	\$endTime	
-	read_byte_seq0	0	110	Tr
child	req	60	110	Tr
Other Relations				
parent	read_modify_...	0	0(unfinish)	Tr

Note:

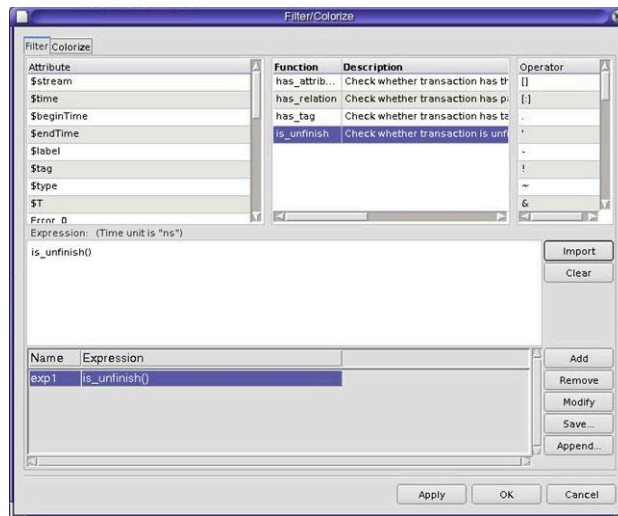
When the simulation is restarted, the added streams and transactions in all transaction windows are removed.

Filtering Unfinished Transactions

Transaction and Protocol Analyzer provides an option to filter unfinished transactions. Using this capability, you can search and filter unfinished transactions in cases where a simulation abruptly ends or hangs. To do this, select the **Stream -> Filter/Colorize** command. The *Filter/Colorize* form is displayed as shown in the following figure.

In the *Filter/Colorize* form, you can enter the `is_unfinish()` function in the **Expression** field to filter unfinished transactions.

Figure 81 Filter Unfinished Transactions



Linking Transaction and Protocol Analyzer With Testbench Debug Browsers

Transaction and Protocol Analyzer also provides the capability to view UVM recorded transactions in testbench debug browsers, like, Class Browser, Object Browser, and Sequence View during interactive debug.

In interactive mode, load the UVM streams with transactions into *Transaction and Protocol Analyzer*, select a transaction in the *Transaction* pane, and click the following right-click options:

- **Show In Class Browser** to view the transaction in *Class Browser*.
- **Show In Object Browser** to view the transaction in *Object Browser*.

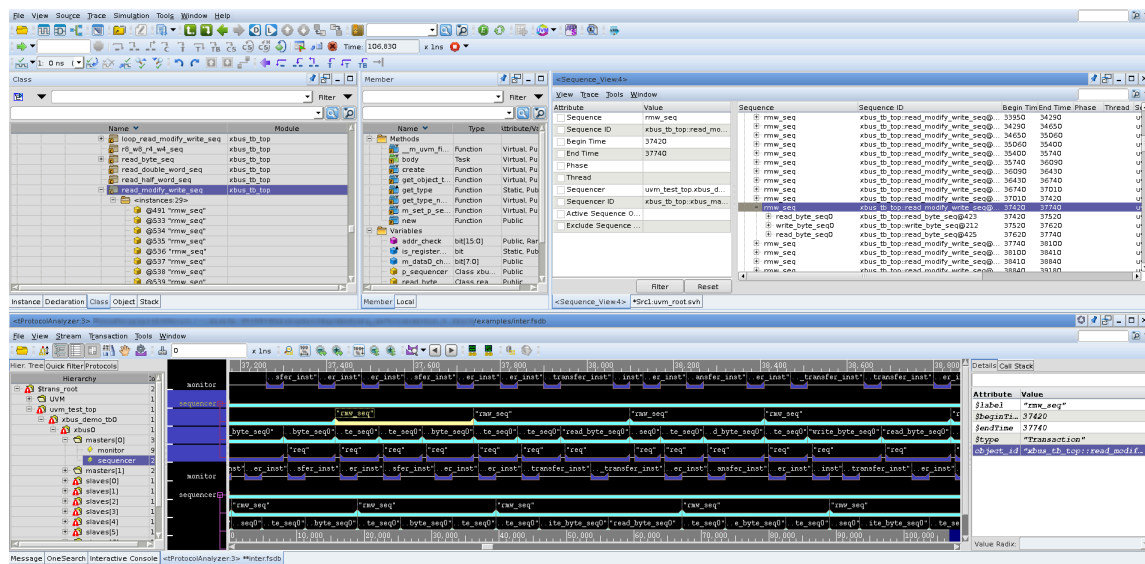
- **Show In Sequence View** to view the transaction in *Sequence View*. This option is disabled for monitor type transactions.

Note:

This functionality is available only in interactive mode and for non-VIP streams.

The following figure illustrates the transaction highlighted in *Transaction and Protocol Analyzer*, *Class Browser*, and in *UVM Sequence View*:

Figure 82 Linking Transaction and Protocol Analyzer with Class Browser and UVM Sequence View



Reverse Debug Capability in Transaction and Protocol Analyzer

Transaction and Protocol Analyzer also provides reverse debug capabilities to rewind and go back to the simulation point from where the transaction started/ended/from where the field value is assigned and use trace and simulation debug actions. You can also enable tracing of value assignments from the *Details* pane in *Transaction and Protocol Analyzer*.

These capabilities are helpful to debug transactions that have issues or if a transaction field value is not correct or needs to be traced to check its assigned value.

Tracing the Simulation to the Beginning or End Point

To trace the simulation back to where the transaction begins or ends, you must use the `-debug_access+reverse` argument as a part of VCS compile switches to dump the transaction step counter details.

By default, transaction begin step counter is recorded. You can control the required details by using the following simulation time switches:

Beginning of Transaction:

```
+fsdb+event_begin_step_count=on      /  
+fsdb+event_begin_step_count=off
```

Transaction end:

```
+fsdb+event_end_step_count=on        /  
+fsdb+event_end_step_count=off
```

In interactive mode, load the UVM streams having transactions into *Transaction and Protocol Analyzer* and

- Click the **Go To Transaction Start** right-click option to move the simulation backward to the source from where the transaction is started (not by time).
- Click the **Go To Transaction End** right-click option to move the simulation backward to the source from where the transaction is ended (not by time).

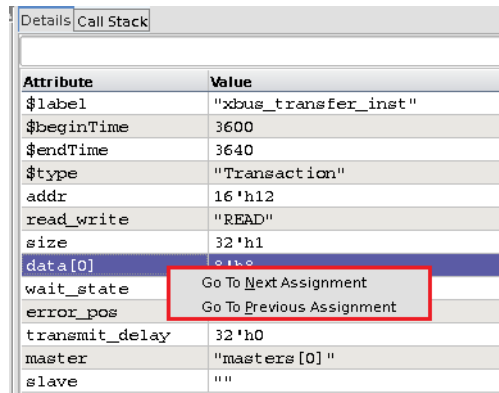
Trace Field Value Assignment

In interactive mode, load the UVM streams with transactions into *Transaction and Protocol Analyzer*. The values for the selected transaction fields are displayed in the *Details* tab.

Select the required field in the *Details* tab and click the **Go To Next Assignment** right-click option to trace the field value assignment. On clicking this option for the first time, simulation goes forward to the point where the value assignment happened for the first time. On clicking this option for the second time for the same transaction, the simulation is forwarded to the next assignment. On clicking this option every time, the simulation is forwarded to the next assignment.

Select the required field in the *Details* tab and click the **Go To Previous Assignment** right-click option to trace the field value assignment. On clicking the **Go To Previous Assignment** right-click option for the first time on the selected field, simulation goes backward to the point where the value assignment happened for the last time for that particular field. On clicking this option for the second time for the same transaction, the simulation goes backward to the previous assignment. On clicking this option every time, the simulation goes backward to the previous assignment.

These options are illustrated in the following figure:



The screenshot shows the 'Details' tab of the Transaction and Protocol Analyzer. It displays a table of transaction attributes. The 'data[0]' row is selected, and a context menu is open over it, showing two options: 'Go To Next Assignment' and 'Go To Previous Assignment', both of which are highlighted with a red border.

Attribute	Value
\$label	"xbus_transfer_inst"
\$beginTime	3600
\$endTime	3640
\$type	"Transaction"
addr	16'h12
read_write	"READ"
size	32'h1
data[0]	00000000000000000000000000000000
wait_state	Go To Next Assignment
error_pos	Go To Previous Assignment
transmit_delay	32'h0
master	"masters [0]"
slave	" "

Object ID Field Value in Details Tab

The `object_id` field value dumped in the FSDB file for all UVM transactions can now be viewed in the *Details* tab of the *Transaction and Protocol Analyzer*. This field value is used for all reverse debug activities and to show the selected transaction in the browsers.

Note:

For information on how to view OVM/UVM component waveforms in *nWave*, see the *Viewing OVM/UVM Component Waveforms in nWave* section in the *Verdi UVM Debug User Guide*.

9

Practice

This chapter demonstrates how to compile a design with a UVM environment, automatically record transaction data during simulation, and use the applications to observe and debug the transaction data in the Verdi platform.

This practice uses the UVM `test_read_modify_write` example with the Verdi modified libraries. Perform the following steps to complete this practice for different stages.

Compiling and Simulating Design With UVM Environment

1. Change directory to the example at the following location:

```
%> cd
$VERDI_HOME/share/vmlib/uvm/uvm/lib/examples/integrated/ubus_evd_n
tb/examples
```

2. Set the environment variables as follows:

```
# Set VCS environment
%> setenv FSDB_ENV_DUMP_SEQ_NUM 1
%> setenv FSDB_ENV_MAX_GLITCH_NUM 0
un
# Set FSDB dumping environment
%> setenv VERDI_HOME <NOVAS_INST_DIR>
%> setenv LD_LIBRARY_PATH
$VERDI_HOME/share/PLI/VCS/LINUXAMD64:$LD_LIBRARY_PATH
```

3. Specify the `VCS_UVM_HOME` environment variable before compilation to identify the path to the UVM 1.1d library for the VCS simulator.

```
%> setenv VCS_UVM_HOME $VERDI_HOME/share/vmlib/uvm/uvm-1.1d/lib/src
```

4. Use the following command to compile the design:

```
%> vcs -full64 -debug_access+all -sverilog -timescale=1ns/1ns
-ntb_opts uvm +incdir+../sv ubus_tb_top.sv
```

5. Use the following command to run the simulation:

```
%> ./simv +UVM_TESTNAME=test_read_modify_write +UVM_VERDI_TRACE
+UVM_LOG_RECORD +UVM_TR_RECORD +UVM_VERBOSITY=UVM_LOW
```

Use the `Makefile.vcs` file in the UVM-1.1d example directory with the following command:

```
%> make -f Makefile.vcs BITS=64
```

The content of the `Makefile.vcs` file is:

```
UVM_HOME = ../../../../..

include ../../../../Makefile.vcs_ntb

all: comp run

comp:
    $(VCS) +incdir+../sv \
        ubus_tb_top.sv

run:
    $(SIMV) +UVM_TESTNAME=test_read_modify_write +UVM_TR_RECORD
    +UVM_LOG_RECORD +UVM_VERDI_TRACE
    $(CHECK)
```

You can review other VCS options in the `Makefile.vcs_ntb` file of the `$VERDI_HOME/share/vmlib/uvm/uvm-1.1d/lib/examples` path.

After the simulation, the `novas.fsdb` FSDB file, including UVM transactions and messages is generated.

6. Invoke the Verdi platform with the `-workMode transactionDebug` option:

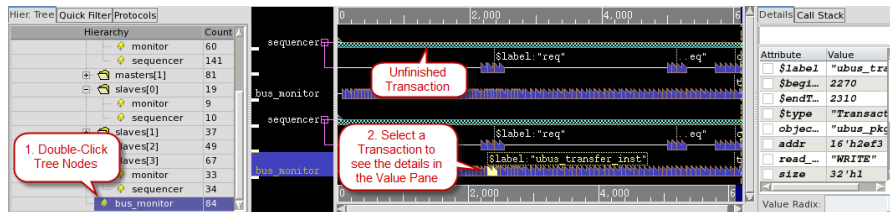
```
%> verdi -ssf novas.fsdb -workMode transactionDebug &
```

The Verdi platform is invoked with three Transaction Debug windows. The `novas.fsdb` file is loaded into *nWave*, *Transaction and Protocol Analyzer*, and *Transaction Table View* windows.

Viewing Transactions in Transaction and Protocol Analyzer Window

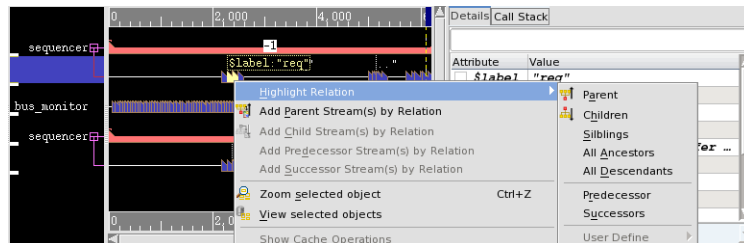
1. Expand the `$event_root -> uvm_test_top -> ubus_example_tb0 -> ubus0 -> masters[0]` and `slaves[0]` directories in the *Hier. Tree* tab. Double-click the tree nodes to add streams to the *Transaction* pane. As shown in [Figure: Add Streams](#), the sequencers of `masters[0]`, `slaves[0]`, and `bus_monitor` streams are added. You can check transaction details in the *Value* pane by selecting a transaction in the *Waveform* pane.

Figure 83 Add Streams



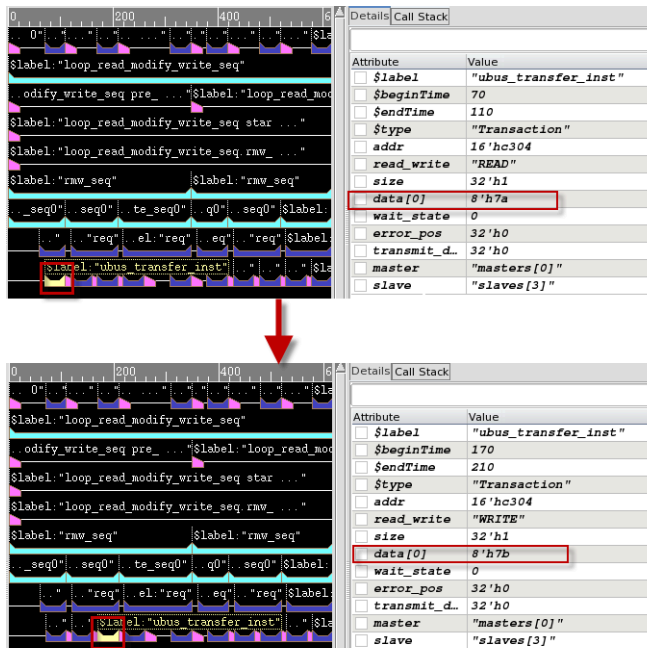
2. Select a transaction and use the **Highlight Relation -> Parent** command in the right-click command menu. As shown in [Figure: Highlight Relation](#), you can see the parent transaction of the selected `req` transaction is highlighted in red.

Figure 84 Highlight Relation



3. Observe the `bus_monitor` stream.
 - a. Select the first transaction of the `bus_monitor` stream. It is a *READ* transaction with address `16'hc304` and read data `8'h7a`.
 - b. Select the second transaction of the `bus_monitor` stream. It is a *READ* transaction with address `16'hb5fc` and read data `8'h1f`.
 - c. Select the third transaction of the `bus_monitor` stream. It is a *WRITE* transaction with address `16'hc304`. Now, the data is `8'h7b`.

Figure 85 Write/Read Transactions



4. Select the second `req` transaction of the first sequencer stream. Switch to the *Quick Filter* tab and you can see all attributes of the selected transaction are automatically shown in the *Quick Filter* tab.

Figure 86 Using Quick Filter



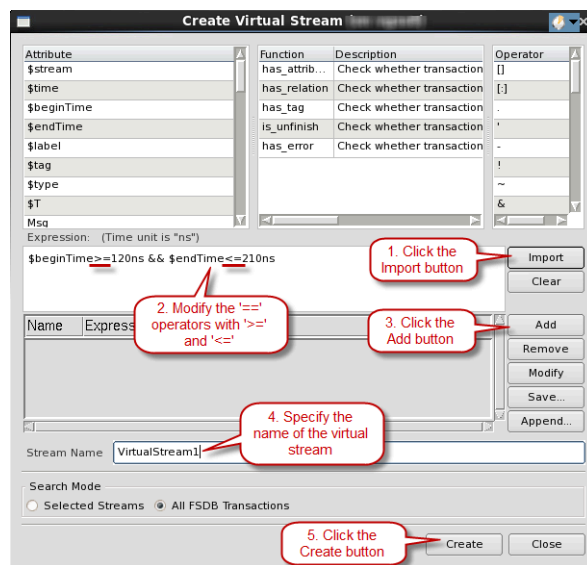
5. Select `beginTime` and `endTime` attributes in the *Quick Filter* tab. The expression of these two attributes is automatically completed. You can then use the **Filter** button to

Viewing Transactions in Transaction and Protocol Analyzer Window

filter out the transactions in the *Waveform* pane. You can also use this expression to create a virtual stream, as described in the next step.

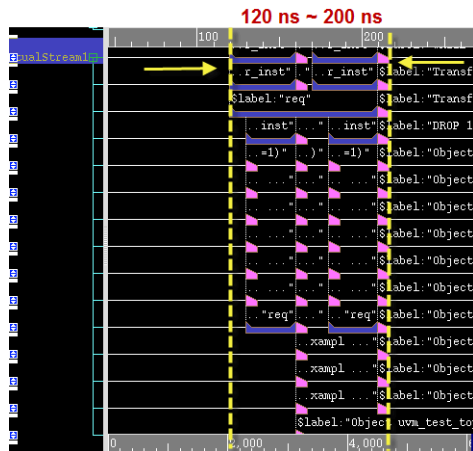
6. Use the **Stream -> Create Virtual Stream** command to open the *Create Virtual Stream* form (as shown in [Figure: Create Virtual Stream](#)).
 - a. Use the **Import** button to import the expression from the *Quick Filter* tab. The `$beginTime==120ns && $endTime==210ns` expression is shown in the **Expression** field.
 - b. Modify the expression as shown in [Figure: Create Virtual Stream](#).
 - c. Click the **Add** button to add the expression to the expression history.
 - d. In the **Stream Name** field, type `VS_TIME1` to specify the name of the new virtual stream.
 - e. Finally, click the **Create** button to create the `VS_TIME1` virtual stream

Figure 87 Create Virtual Stream



As shown in [Figure: VirtualStream1-Virtual Stream](#), the `VirtualStream1` stream is created with the transactions between 120ns to 210ns.

Figure 88 VirtualStream1-Virtual Stream

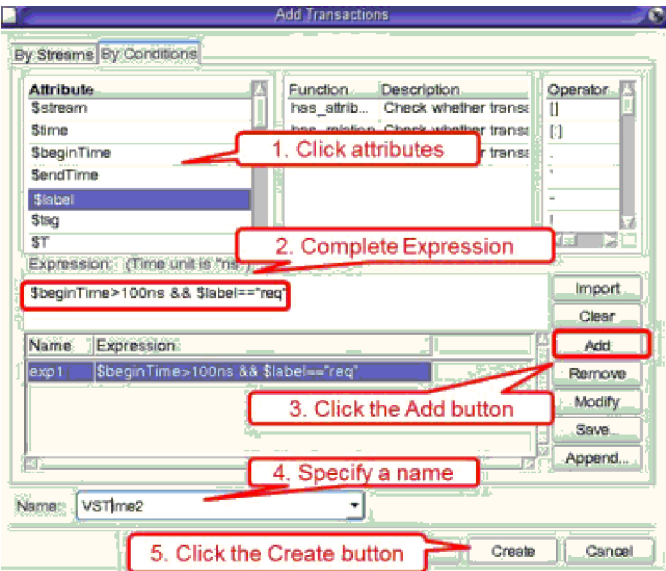


Viewing Transactions in the Transaction Table View Window

To observe transactions whose begin times are larger than 100ns and with the `req` label name, do the following:

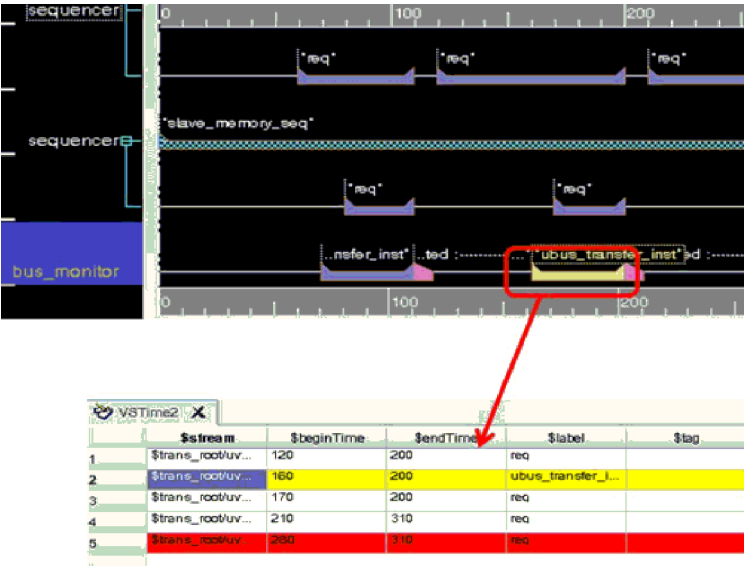
1. Click the  icon and then select the **By Conditions** tab in the *Add Transaction* form to add transaction by condition. Double-click `$beginTime` and `$label` attributes to add them into the **Expression** field.
2. Complete the expression with `$beginTime>100ns&&$label=="req"` condition.
3. Click the **Add** button to add the expression to the expression history.
4. In the **Stream Name** field, type **VSTIME2** to specify the name of the editable tab.
5. Finally, click the **Create** button to create the **VSTime2** tab in the *Transaction Table View* window (refer to [Figure: Add Transactions by Condition](#)).

Figure 89 Add Transactions by Condition



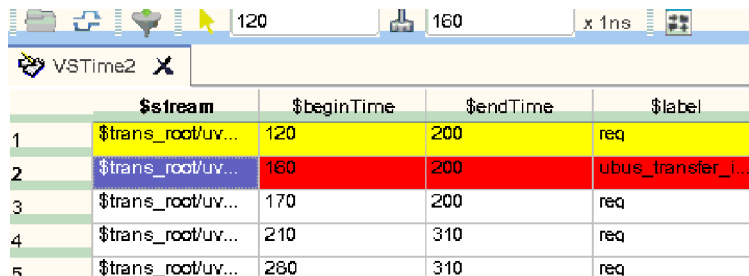
6. Transactions that match the expression are added in the **VSTime2** tab in the *Transaction Table View* window. Drag the second transaction of the **bus_monitor** from the *Transaction and Protocol Analyzer* window and drop to the *Transaction Table View* window.

Figure 90 Add a Transaction from Transaction and Protocol Analyzer



- Click the first transaction and middle-click the second transaction to check the relative time between two transaction recordings from *master* and *bus_monitor*. The cursor time shows the time of the first transaction that was selected by left-clicking and the row is highlighted with yellow. The marker time shows the time of the second transaction that was selected by middle-clicking and the row is highlighted with red.

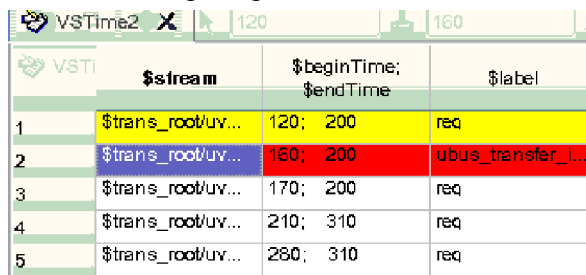
Figure 91 Relative Time between Transactions



	\$stream	\$beginTime	\$endTime	\$label
1	\$trans_root/uv...	120	200	req
2	\$trans_root/uv...	160	200	ubus_transfer_l...
3	\$trans_root/uv...	170	200	req
4	\$trans_root/uv...	210	310	req
5	\$trans_root/uv...	280	310	req

- Use the **View -> Configure Columns** command to combine *\$beginTime* and *\$endTime* columns for easier observation. Select *\$beginTime* and *\$endTime* attributes and click the **Pack Selected** button in the *Configure Columns* form. The names of packed attributes are added with a (Pack1) suffix. Click the **Apply** button and the columns are combined to one column with the *\$beginTime;\$endTime* name.

Figure 92 Configuring Columns

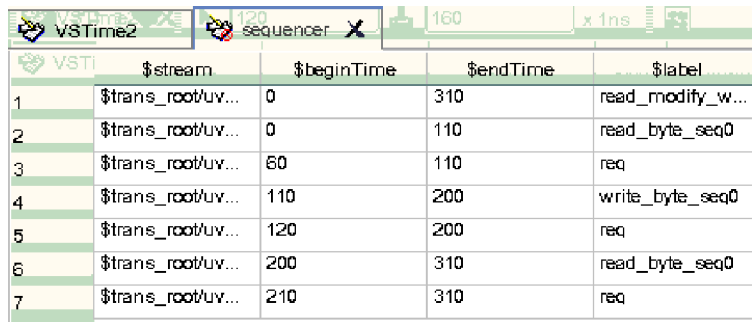


	\$stream	\$beginTime; \$endTime	\$label
1	\$trans_root/uv...	120; 200	req
2	\$trans_root/uv...	160; 200	ubus_transfer_l...
3	\$trans_root/uv...	170; 200	req
4	\$trans_root/uv...	210; 310	req
5	\$trans_root/uv...	280; 310	req

You can also unpack the packed attributes as follows:

- Invoke the **View -> Configure Columns** command.
 - Select the packed attribute.
 - Click the **Unpack Selected** button.
 - Click the **OK** button to apply the setting.
- To observe the transactions in the sequencer stream of *Master*, add the sequencer stream by dragging and dropping from the *Transaction and Protocol Analyzer* window. Drag the first sequencer from the *Transaction and Protocol Analyzer* window and drop to the *Transaction Table View* window to create a non-editable tab.

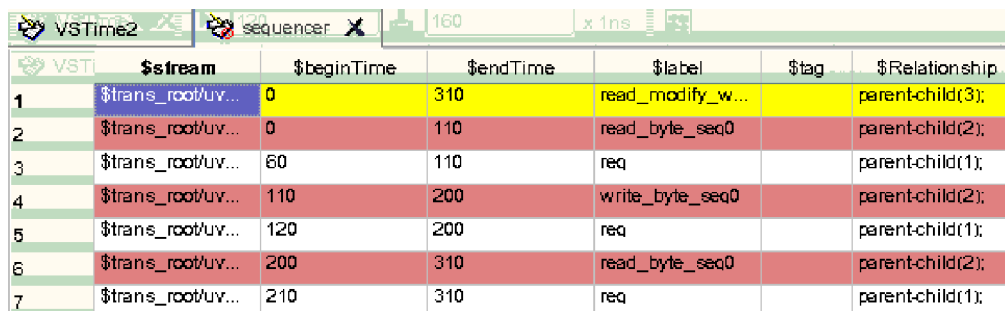
Figure 93 Adding Sequencer Stream



	\$stream	\$beginTime	\$endTime	\$label
1	\$trans_root/uv...	0	310	read_modify_w...
2	\$trans_root/uv...	0	110	read_byte_seq0
3	\$trans_root/uv...	60	110	req
4	\$trans_root/uv...	110	200	write_byte_seq0
5	\$trans_root/uv...	120	200	req
6	\$trans_root/uv...	200	310	read_byte_seq0
7	\$trans_root/uv...	210	310	req

- Use the **View -> Highlight Relation -> Child** command and select the first transaction. You can see the children of the selected transaction are highlighted. Also use the **View -> Show Relationship** command to show the *\$Relationship* column in the *Transaction Table View* window.

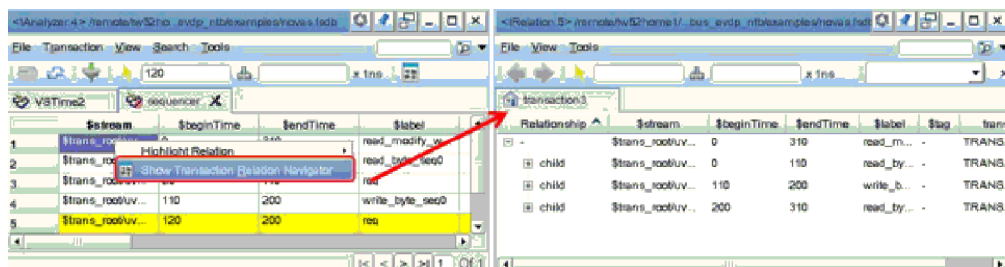
Figure 94 Viewing Children of Transactions



	\$stream	\$beginTime	\$endTime	\$label	\$tag	\$Relationship
1	\$trans_root/uv...	0	310	read_modify_w...		parent-child(3);
2	\$trans_root/uv...	0	110	read_byte_seq0		parent-child(2);
3	\$trans_root/uv...	60	110	req		parent-child(1);
4	\$trans_root/uv...	110	200	write_byte_seq0		parent-child(2);
5	\$trans_root/uv...	120	200	req		parent-child(1);
6	\$trans_root/uv...	200	310	read_byte_seq0		parent-child(2);
7	\$trans_root/uv...	210	310	req		parent-child(1);

- The relationship is easier to see in the *Transaction Relation Navigator*. To invoke the *Transaction Relation Navigator* window through the *Transaction Table View* window, right-click the first transaction and choose the *Show Transaction Relation Navigator* command in the right-click command menu.

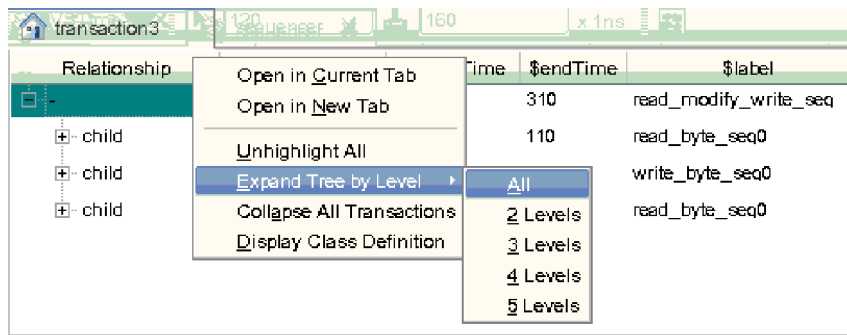
Figure 95 Transaction Relation Navigator



Viewing Transactions in Transaction Relation Navigator Window

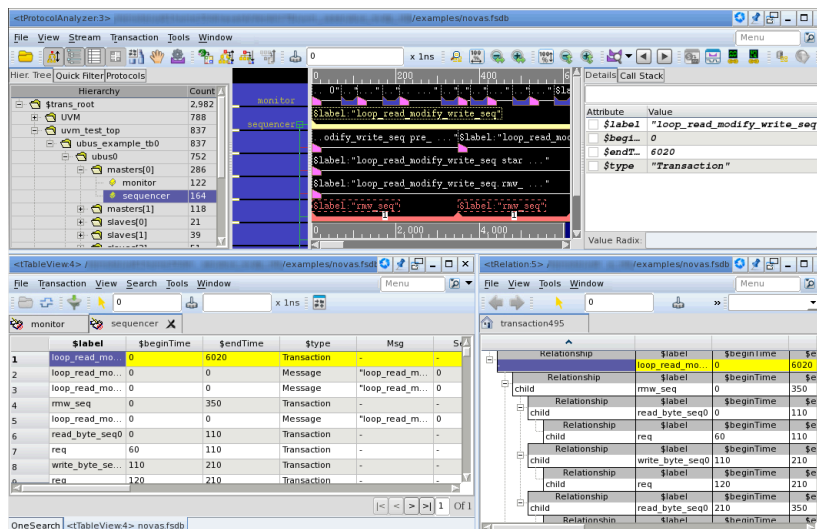
1. To observe all levels of the relationship, use the **Expand Tree by Level -> All** command in the right-click command menu to expand tree by all levels

Figure 96 Expand Tree by Level



2. To synchronize a selected transaction between the three windows, click the  icons in the right upper position of these windows. Then, select the fourth transaction in the *Transaction Relation Navigator* window, and you can see the same transactions are also highlighted in *Transaction and Protocol Analyzer* and *Transaction Table View* windows.

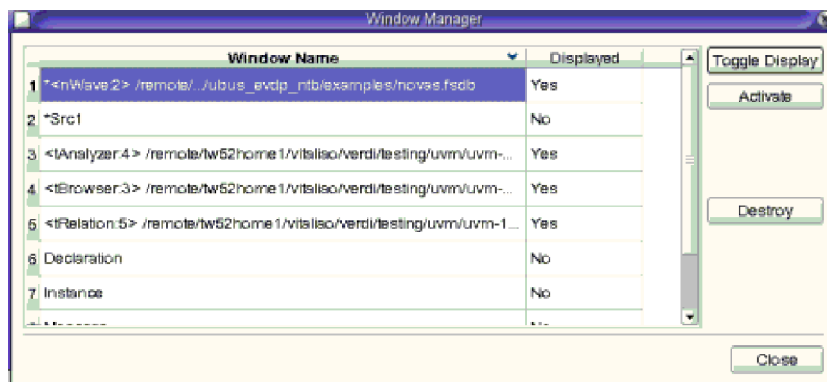
Figure 97 Synchronize a Selected Transaction



The content of the *Detail* tab is also automatically changed based on the selected transaction in the *Transaction and Protocol Analyzer* window and the corresponding child transaction is highlighted by the selected parent transaction.

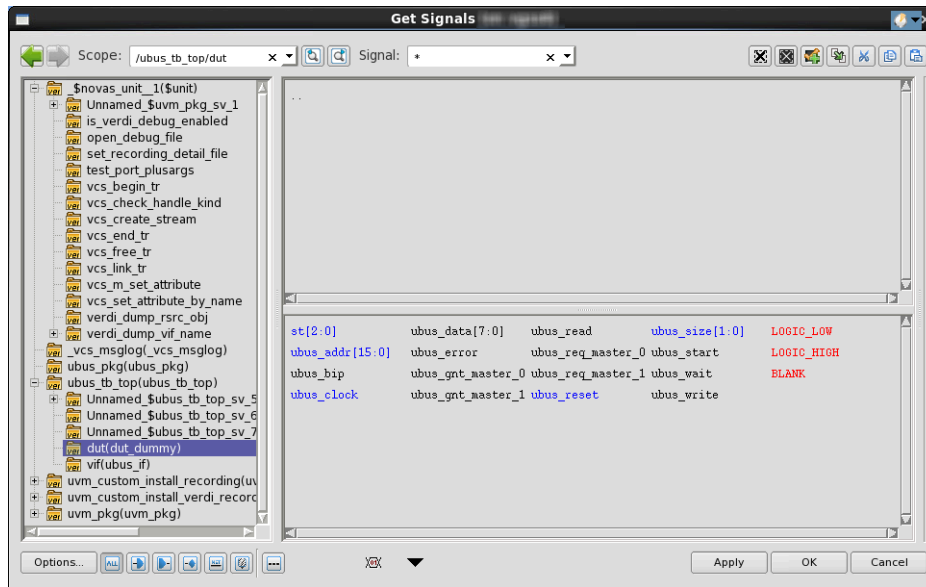
3. A transaction can be synchronized between these three windows and the *nWave* window.
 - a. Choose the **Window -> Window Manager** command to open the *Window Manager* form.
 - b. Select the opened *nWave* window in the first row.
 - c. Click the **Activate** button to activate the *nWave* window in the main window of the Verdi platform.
 - d. Click the **Close** button to close the *Window Manager* form.

Figure 98 Window Manager



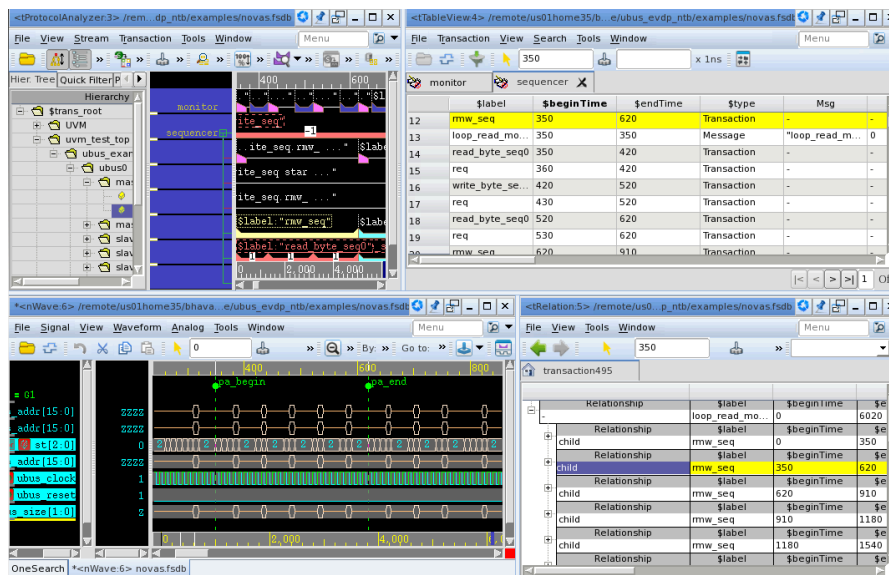
Now, you can see the *nWave* window is displayed in the Verdi main window. Add signals to the *nWave* window.

Figure 99 Adding Signals to nWave



4. Enable the **View -> Sync. with nWave -> Mark Transaction** toggle command in the *Transaction and Protocol Analyzer* window and adjust the location and size of these windows to fit the screen. Select the fourth transaction in the *Transaction Table View* window.

Figure 100 Adjusting Location and Size



You can see the begin time and the end time of the selected transaction in the *nWave* window with green dashed lines to indicate with *paBegin* and *paEnd* marks. You can clearly observe the signal changes during this time period.

The following sections show methods to configure a VIP to generate Transaction FSDB file and use it for protocol debug and performance analysis:

- [Installing VIP Examples](#)
- [Configuring VIP to Generate Transaction FSDB File](#)
- [Viewing VIP Transactions in Transaction and Protocol Analyzer Window](#)
- [Viewing VIP Transactions in Transaction Table View Window](#)
- [Viewing VIP Transactions in Transaction Relation Navigator Window](#)

Installing VIP Examples

You can install any of the VIP public examples and the instructions on how to install an example is present in the `README` file located at `$DESIGNWARE_HOME/vip/svt/<suite_name>/latest/examples/sverilog/<test_bench>/README`

Configuring VIP to Generate Transaction FSDB File

Most of the Synopsys VIPs have port level configuration control for recording the transaction information into an FSDB file. After installing the examples, you can search for `enable_xml_gen` and `pa_format_type` configuration attributes which are probably inside the `*cust_*configuration.sv` file. The fields need to be set with the following values:

```
enable_*_xml_gen=1;  
pa_format_type = svt_xml_writer::FSDB;
```

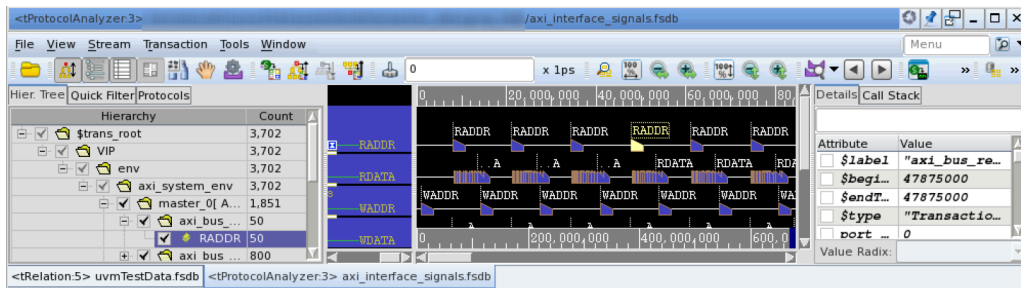
Along with the configuration, use the following additional compilation switch to enable the transaction recording:

```
+define+SVT_FSDB_ENABLE
```

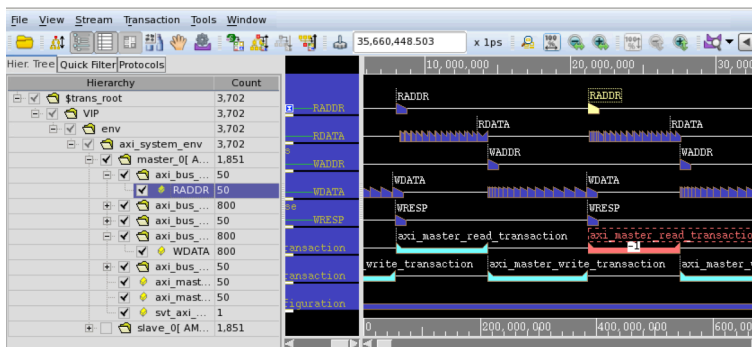
Viewing VIP Transactions in Transaction and Protocol Analyzer Window

1. Expand the hierarchy in the *Hier. Tree* tab and check the check-box for tree nodes to add streams to the *Transaction* pane. As shown in the below figure, the sequencers of *master[0]* stream are added. You can check transaction details in the *Value* pane by selecting a transaction in the *Transaction* pane.

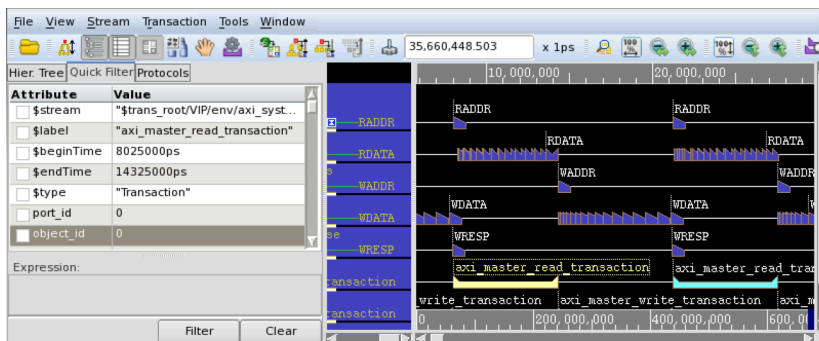
Viewing VIP Transactions in Transaction and Protocol Analyzer Window



2. Select a transaction and use the **Highlight Relation -> Parent** command in the right-click command menu. As shown in figure below, you can see the parent transaction of the selected RADDR transaction is highlighted in red.

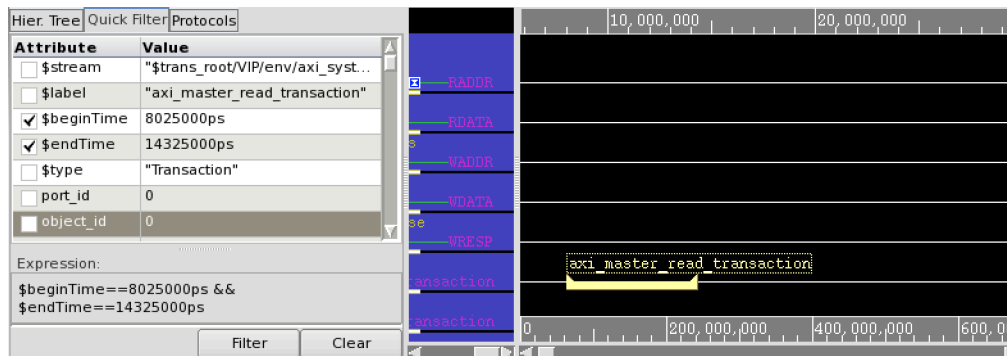


3. Select a transaction and switch to the *Quick Filter* tab and you can see all attributes of the selected transaction are automatically shown in the *Quick Filter* tab.



4. Select any attributes (for example, *beginTime* and *endTime*) in the *Quick Filter* tab. The expression of these two attributes is automatically completed and gets displayed in the

Expression field. You can then use the **Filter** button to filter out the transactions in the *Transaction* pane.



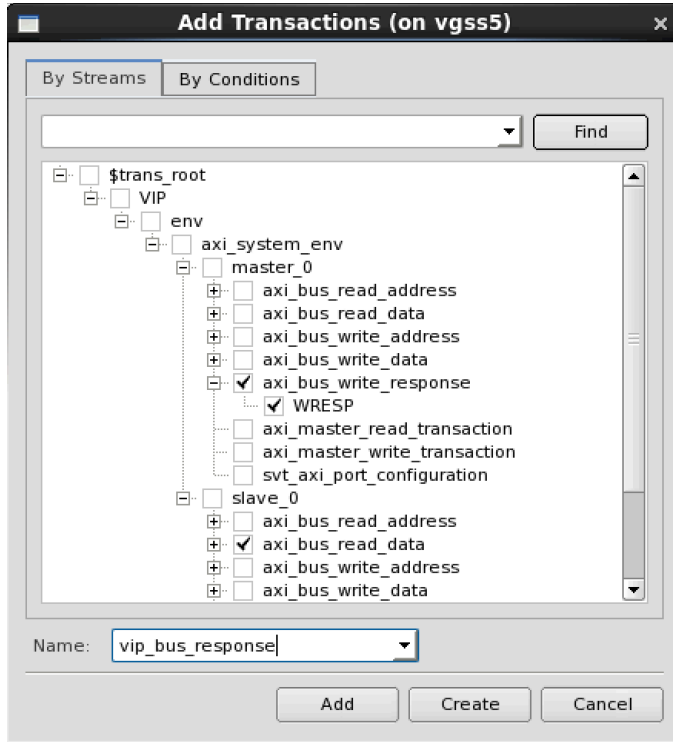
Viewing VIP Transactions in Transaction Table View Window

To add transactions to the *Transaction Table View* window, you can drag and drop the transactions from the *Transaction and Protocol Analyzer* window.

Alternatively, you can also click a transaction in the *Hierarchy Tree* tab and use the **Show In Transaction Table View** right-click option to add the transactions to the preferred *Transaction Table View* window.

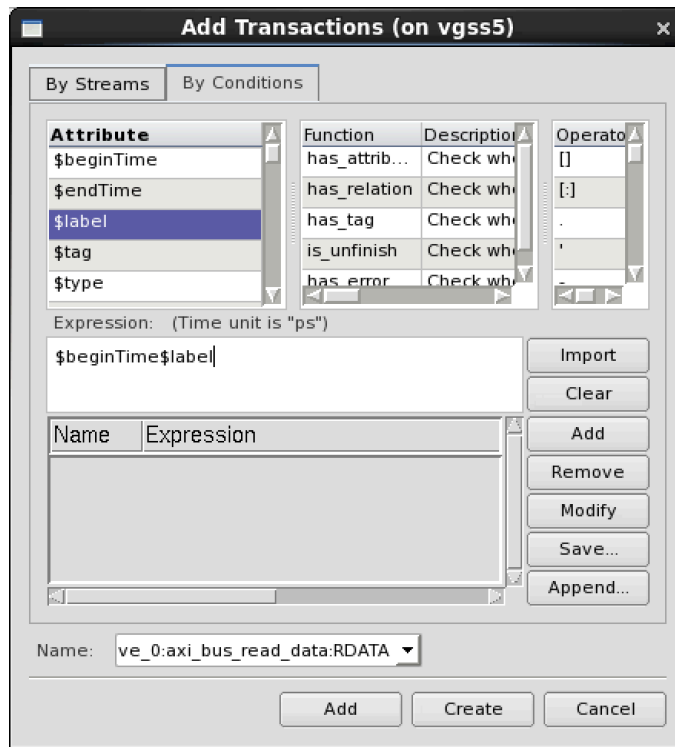
You can also add the transactions by clicking the  icon and then selecting *By Streams* or *By Conditions* tab in the *Add Transactions* form.

To add transactions by streams, click the *By Streams* tab, select the checkbox for the transactions you want to add, specify a name in the *Name* field, and click the **Create** button, as illustrated in the following figure:



To add transactions by condition, double-click the *By Conditions* tab and select the attributes from the *Attribute* column. The attributes get added to the *Expression* field automatically. Complete the expression and click the **Add** button. To add transactions to an already existing stream, select the stream name in the *Name* field and click the **Add**

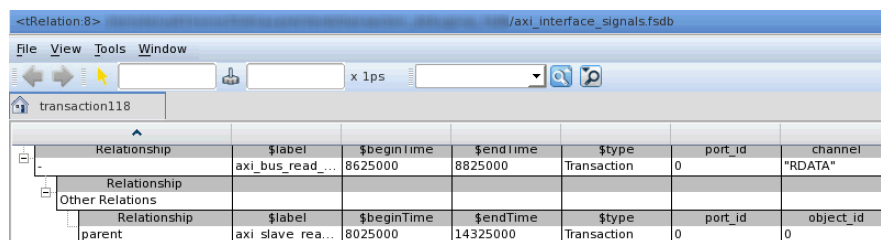
button. To add transactions to a new stream, specify the name in the *Name* field and click the **Create** button. This is illustrated in the following figure:



Viewing VIP Transactions in Transaction Relation Navigator Window

You can view the relationship of a selected transaction easily in the *Transaction Relation Navigator*. To invoke the *Transaction Relation Navigator* window through the *Transaction Table View* window, select a transaction and choose the *Show Transaction Relation Navigator* right-click option.

To observe all levels of the relationship for the selected transaction, click the **View -> Expand Tree by Level -> All** right-click option. The tree expands by all levels as illustrated in the following figure:



A

Appendix A: Using Transaction and Protocol Analyzer (Protocol Debug Mode)

The *Transaction Debug* platform in Verdi is designed from the ground-up for capture, display, debug, and analysis of abstract data, such as, transactions, messages, or any other information that users want to access at high-level.

Transaction and Protocol Analyzer is integrated into Verdi as an extension to Verdi's Transaction Debug platform. The *Verdi Transaction and Protocol Analyzer* is a protocol-aware transaction debug solution in Verdi that allows viewing and debugging high level protocol and transaction/memory information along with signal-level waveform, using one FSDB file for all the mentioned data. To enhance usability, this single-process solution provides a user-friendly platform to debug waveforms, transaction data, logs, and source code, with different views that can be synchronized. The protocol data in the FSDB file is dumped directly from Synopsys VIP. VC APPs APIs are also provided for users to enable their own custom VIP to dump the protocol data into the FSDB file.

This Appendix discusses the *VIP Protocol Analyzer* mode (*Protocol Debug* mode) in the following sections:

- [Invoking Protocol Debug](#)
- [Invoking Transaction and Protocol Analyzer](#)
- [Prerequisites](#)
- [Transaction and Protocol Analyzer Window Overview](#)

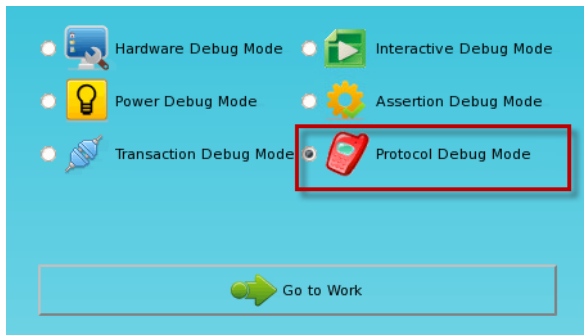
Invoking Protocol Debug

The recommended flow to browse the protocol data is as follows:

1. Invoke the Verdi platform with the whole system, including design, testbench, and FSDB file. For example,

```
verdi -ssf <file_name>.fsdb
```
2. Click the **Work Modes** icon in the *Welcome* Page, select the **Protocol Debug Mode** option, and then press the **Go to Work** button.

Figure 101 Selecting Protocol Debug Work Mode



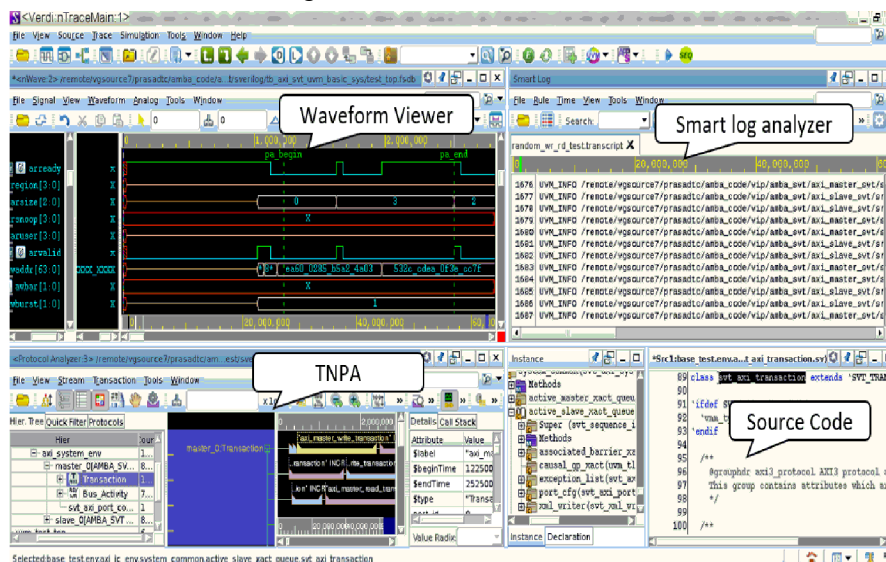
Or use the `-workMode protocolDebug` option directly to invoke Verdi in Protocol Debug mode. For example:

```
%> verdi -workMode protocolDebug <verdi options> &
```

Alternatively, you can invoke the *Transaction and Protocol Analyzer* pane manually using the **Tools -> Transaction Debug -> Transaction and Protocol Analyzer** command in the main window.

The *Protocol Debug* mode also opens the *Transaction and Protocol Analyzer* pane to debug the Verilog/SystemVerilog system at a higher level. As with other panes in the Verdi platform, the *Transaction and Protocol Analyzer* pane can be docked, undocked, or dragged and dropped to other parts of the main window. In the following figure, "TNPA" means Transaction and Protocol Analyzer.

Figure 102 Protocol Debug Mode



Invoking Transaction and Protocol Analyzer

You can invoke *Transaction and Protocol Analyzer* using the **Tools -> Transaction Debug -> Transaction and Protocol Analyzer** menu command in *nTrace*.

You can also invoke *Transaction and Protocol Analyzer* using the following menu commands:

- **Tools -> Transaction Debug -> Transaction and Protocol Analyzer** command in the Verdi main window.
- **Tools -> New Transaction and Protocol Analyzer Window** command in the *Transaction Table View* window.
- **Tools -> New Transaction and Protocol Analyzer Window** command in the *tRelation* Window.
- **Window -> Connect Transaction and Protocol Analyzer and nWave** command in the *nWave* Window.
 - The  icon opens *Transaction and Protocol Analyzer* along with *nWave*.

Prerequisites

Before using the Verdi *Transaction and Protocol Analyzer* to perform protocol-level debugging, use the VC VIP and execute the simulation to dump the protocol information into the FSDB file (refer to the VIP manual for the supported VIPs and the detailed usage). The HDL signals and the protocol data can be dumped into the same FSDB file.

Then, *Transaction and Protocol Analyzer* can be used to observe the VIP transaction data and protocol information in Verdi.

Also, the `extension.xml` file, that is, the protocol definition is required to show the layered structure for the protocol in the tree or the display.

You must also specify the following environment variables before invoking the Verdi platform to enable VIP-related *Transaction and Protocol Analyzer* features:

```
%> setenv DESIGNWARE_HOME <DesignWare install path>
%> setenv PA_CUSTOM_VIP_PATH <Protocol Definition Files path>
```

If proprietary protocol VIPs need to be analyzed, you can specify the location of proprietary protocol definitions using the `PA_CUSTOM_VIP_PATH` environment variable.

Transaction and Protocol Analyzer Window Overview

The *Transaction and Protocol Analyzer* pane provides a software-oriented waveform-like view (compared to the hardware hierarchical view in the *nTrace* window) for protocol/transaction level debug. The *Protocol Browser* window can be used to debug and analyze transactions and their activity by the time and transaction type.

The default pane layout of the *Transaction and Protocol Analyzer* window is shown in the following figure. The left region contains the *Hierarchy Tree* pane; the middle region contains the *Protocol Transaction* pane; the right region contains the *Details* pane.

Figure 103 Protocol Browser Default Layout



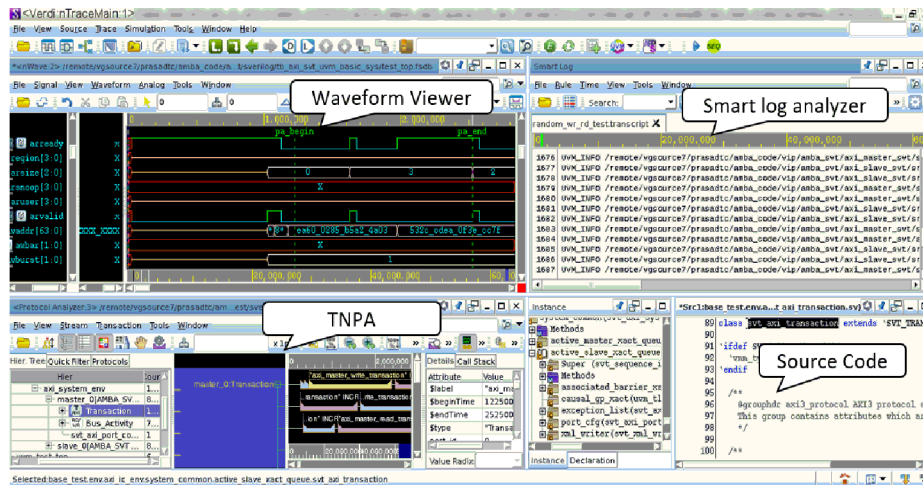
In the *Protocol Analyzer* pane, you can see transaction type, hierarchy, and attributes, including the selected transaction, parent-children relationships of the selected transaction, and detailed attributes for the selected transaction. Refer to the [Figure: Protocol Browser Default Layout](#).

If the FSDB file has not been loaded into Verdi, click the  button or select the **File -> Open** command to import the FSDB file. One *Transaction and Protocol Analyzer* pane can open only one FSDB file.

Transaction and Protocol Analyzer is integrated with *nWave*, *Smart Log*, and *Source Code* pane, leveraging the features and performance of Verdi. Protocol Analyzer, *nWave*, *Smart Log*, and the source code panes can be synchronized and docked to display the transaction data in various forms simultaneously.

Appendix A: Appendix A: Using Transaction and Protocol Analyzer (Protocol Debug Mode) Transaction and Protocol Analyzer Window Overview

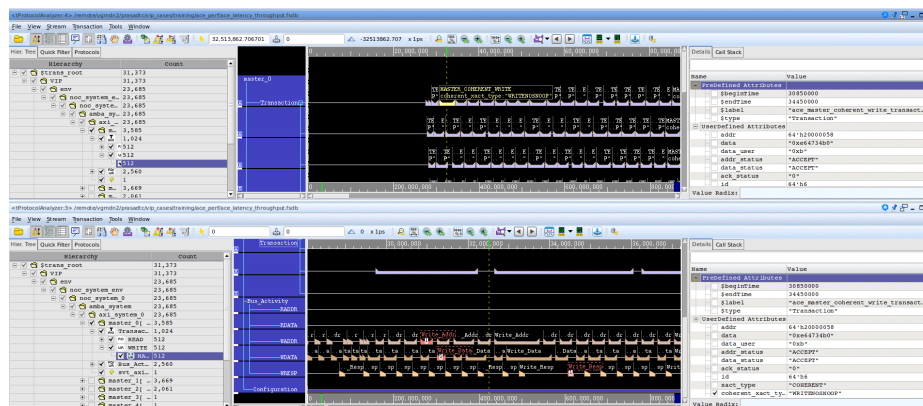
Figure 104 Protocol Analyzer Integrated with nWave, Smart Log, and Source Code



Two *Transaction and Protocol Analyzer* panes can be synchronized with the **Sync. Signal Selection** toolbar icon to display the selected object in one pane and the referenced objects in the other pane.

When debugging a complex protocol that has several layers (headings), it is difficult to visualize all in one *Transaction and Protocol Analyzer* pane. You can add some layers in one *Transaction and Protocol Analyzer* pane and other layers in another *Transaction and Protocol Analyzer* pane before synchronizing these two panes.

Figure 105 Synchronize Protocol Analyzer Panes



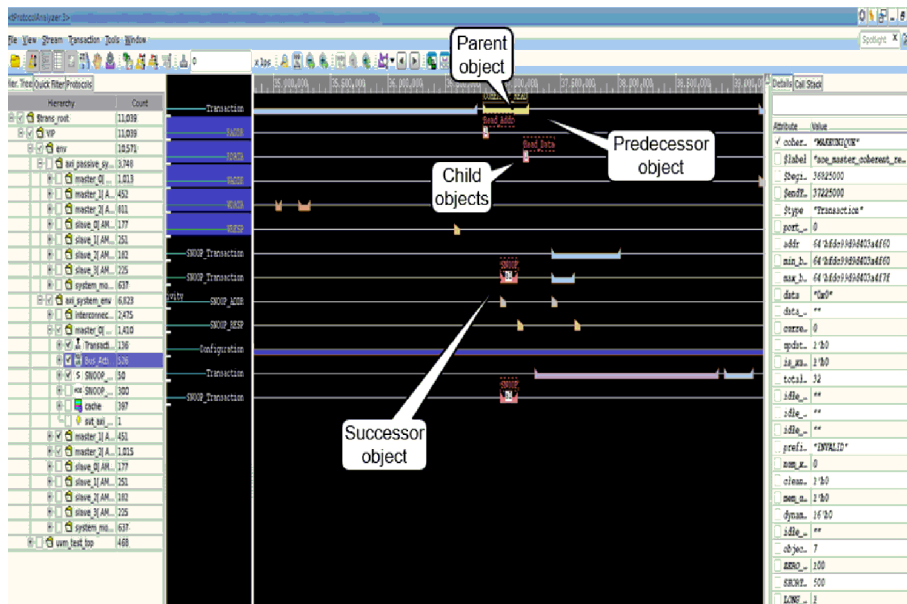
After importing an FSDB file, the protocol hierarchy tree recorded in the FSDB file is shown in the *Hier. Tree* pane. Select the objects listed in the *Hierarchy* tree to add the related protocol and transaction information to the *Transaction* pane as follows:

1. Expand the hierarchy tree by clicking the  icon.
2. Select the objects by clicking the  icon that belongs to the object node and switch to . When a node is selected, the parent and the children that are associated with the node are also selected.

Figure 106 Selecting the VIP Instance

☒		axi_system_0	5,646
☒		master_0[AMBA_SVT AXI_SVT]	813
☒		Transaction	90
☒		WR WRITE	90
☒		Master_WRITE	90
☒		Bus Activity	722
☒		svt_axi_port_configuration	1

The selected objects are added to the *Transaction* pane, as shown in the following figure:



3. After adding objects to the *Transaction* pane, add or remove other objects by checking or un-checking the  icons in the *Hierarchy Tree*.

Protocol Analyzer Toolbar or Menu Bar Overview

You can use commands in the menu on the top of the *Transaction and Protocol Analyzer* pane to invoke forms or execute operations. The categories of the command menu include: **File**, **View**, **Transaction**, **Tools**, and **Window**. Also, you can click the toolbar icons to execute the corresponding commands. If the command has corresponding toolbar icon, the figure of the icon displays to the left of the command name in the command menu.

For example, you can use the **File -> Save Streams / Restore Stream** commands to save or restore the protocol analyzer session; you can use the **View -> Configure Display** command to invoke the *Configure Display* form to set to hide or show attributes in the *Details* tab; you can use the commands under the **Transaction** menu to leverage the features of the *Transaction* pane.

Figure 107 Menu Bar Overview

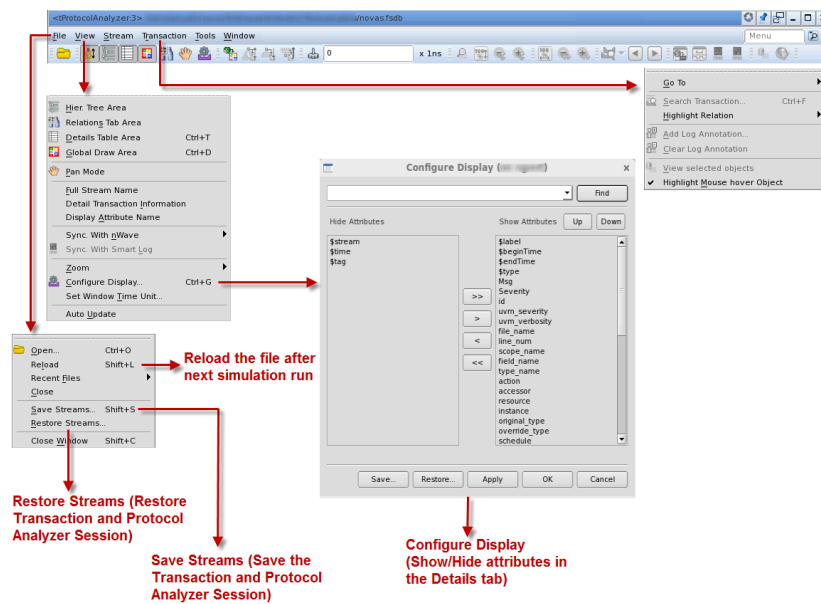
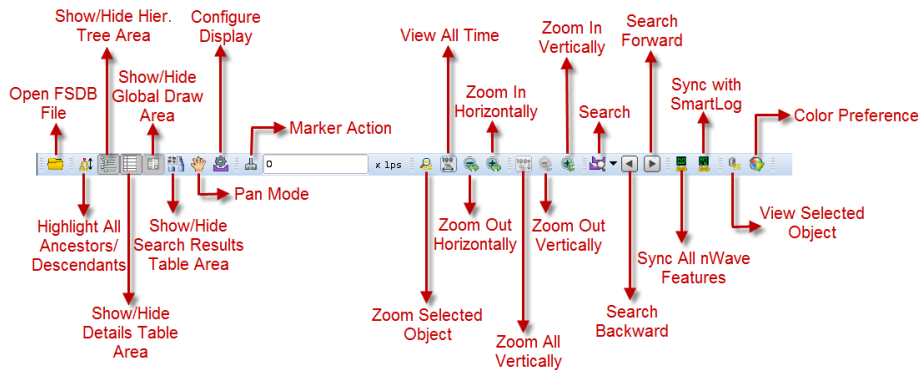


Figure 108 Toolbar Overview



The following command actions can be realized with icons and fields in the toolbar:

- **Show/hide objects:** Click the toggle icons to switch between showing and hiding panes or tabs, including *Hierarchy Tree* pane, *Details* tab, and global draw area.
- **Highlight all Ancestors/Descendants:** Click the toggle icon to highlight all the ancestors/descendants of the selected object.
- **Mark time and set time unit:** Click the **Marker Action** icon to mark the specified time in the *Transaction* pane. To specify the time unit, you can either enter it manually in the time unit field, or click the **Time Unit** icon to invoke the *Set Window Time Unit* form where you can select the time unit from the pull-down menu.
- **Adjust the view in the Transaction pane:** Click the icons to zoom in/out selected transaction vertically and horizontally, or show all vertical/horizontal transactions.
- **Synchronize with nWave and Smart Log:** Click the icons to synchronize the *Protocol Analyzer* pane with *nWave* pane and *Smart Log* pane.
- **Configure Display:** Click the icon to invoke the *Configure Display* form to specify the attributes to be shown or hidden in the *Details* tab.
- **Search Transaction:** Click the icon to invoke the *Set Search Attribute* form to specify the search criteria to search matching attributes.
- **View selected objects:** Click the **View selected objects** icon to show only the selected objects and check the related parent-children objects. Other objects are hidden.

Color Preferences

Menu Bar: Tools -> Preferences

Appendix A: Appendix A: Using Transaction and Protocol Analyzer (Protocol Debug Mode) Transaction and Protocol Analyzer Window Overview

This command opens the *Preferences* page where the color settings of the selected protocol can be configured.

Figure 109 Preferences Form

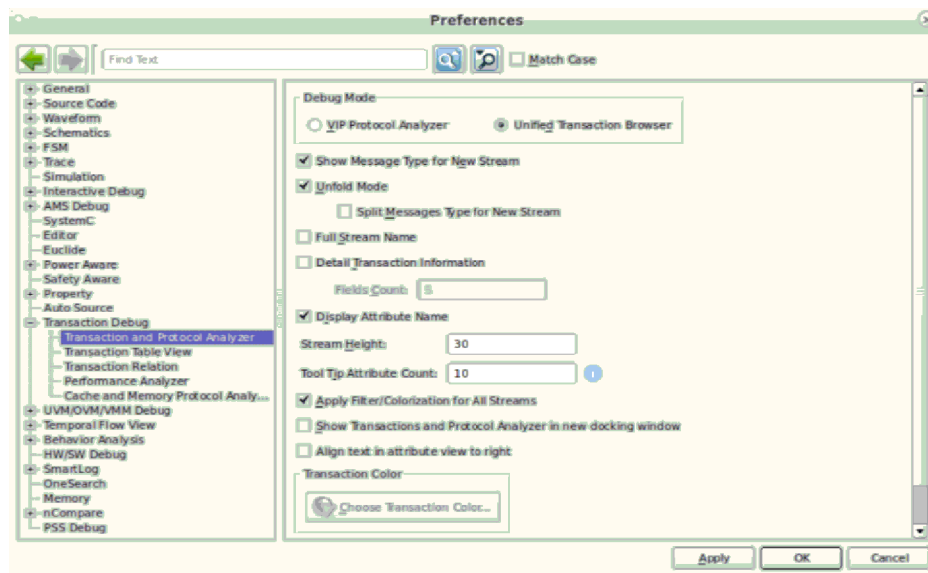


Figure 110 Color Preference Dialog

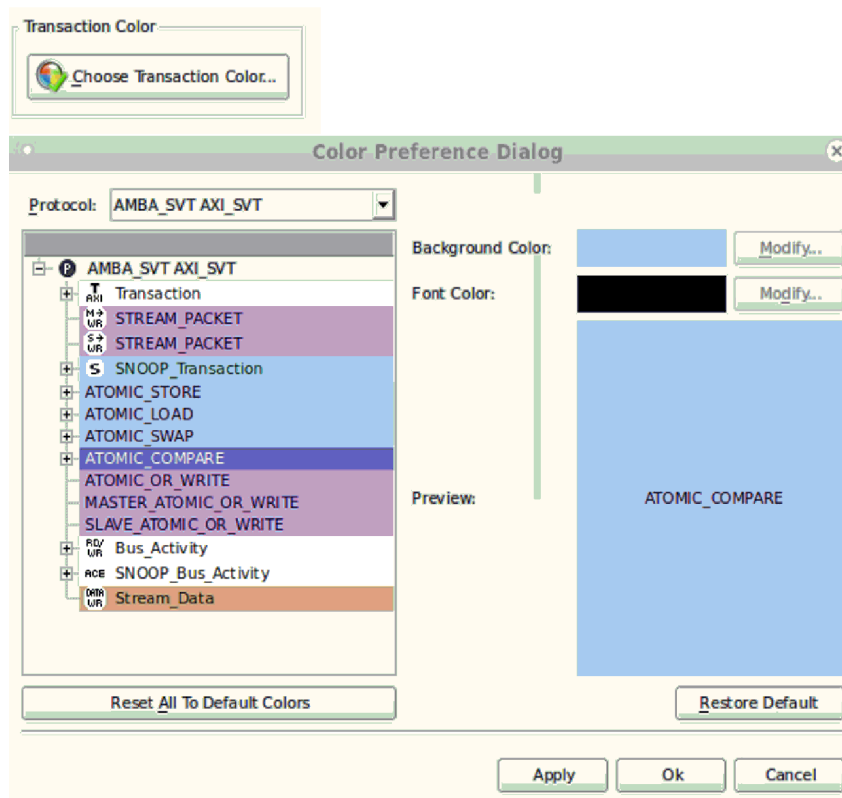
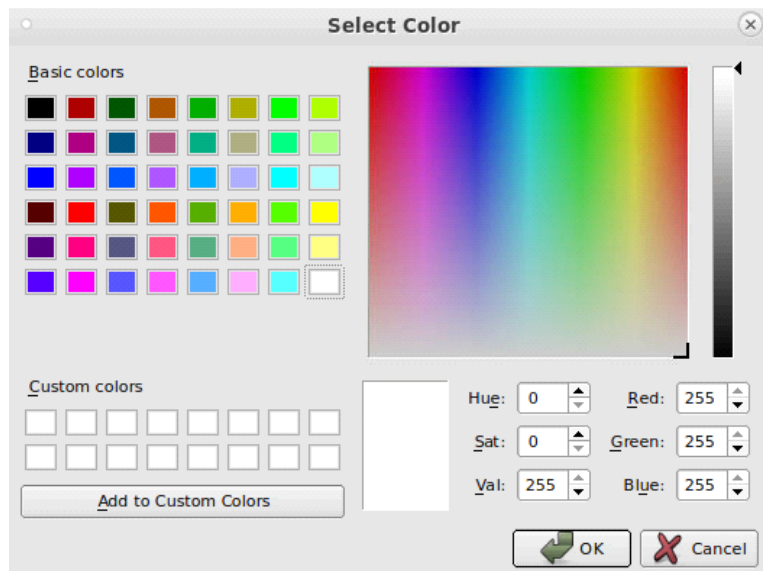


Figure 111 Select Color Form



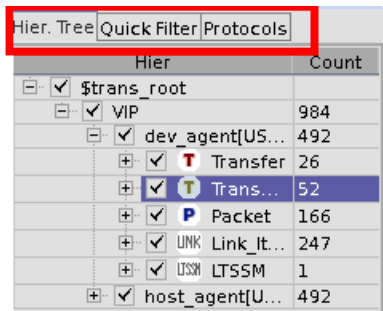
The following field and buttons are available:

- **Protocol:** Select the protocol from the **Protocol** selection field.
- **Hierarchy Tree:** Select the object to configure the color settings from the hierarchy tree section.
- **Back Ground Color:** This field displays the background color of the selected object. If other background color is specified from the *Select Color* form, the specified color is displayed in this field.
- **Font Color:** This field displays the font color of the selected object. If other font color is specified from the *Select Color* form, the specified color is displayed in this field.
- **Modify:** Click this button to open the *Select Color* form where the color of background or font can be specified for the selected object. The following fields and buttons are available:
 - **Basic colors:** Specify the desired color for the background or font of the object from this plate of basic colors.
 - **Custom colors:** Specify the customized colors from the **Basic colors** plate or the color spectrum.
 - **Add to Custom Colors:** Click this button to add the specified color to the **Custom colors** plate.
 - **Color Spectrum:** Specify the desired color for the background or font of the object by clicking the color in the color spectrum or specifying the color values in the selection fields below the color spectrum.
 - **OK:** Applies the settings and closes the *Color Preference* form.
 - **Cancel:** Cancels the settings and closes the *Color Preference* form.
- **Preview:** This field displays the specified colors of background and font of the object for preview.
- **Reset All To Default Colors:** Click this button to reset the color settings of all objects of the selected protocol to default.
- **Reset Default:** Click this button to reset the color settings of the selected objects to default.
- **Apply:** Click this button to apply all of the changes in the *Select Color* form and leave the *Color Preference* form open.

- **OK:** Click this button to save all of the changes in the *Select Color* form and close the *Color Preference* form.
- **Cancel:** Click this button to cancel all of the changes in the *Select Color* form and close the *Color Preference* form.

Tree pane

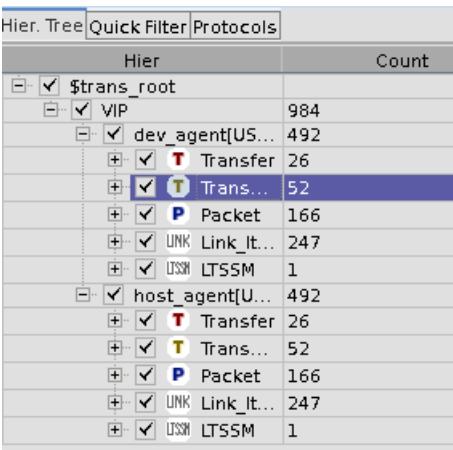
The *Tree* pane consists of three tabs: *Hierarchy Tree*, *Quick Filter*, and *Protocols*.



Hierarchy Tree

The *Hierarchy Tree* tab displays the protocol hierarchy that is present in the loaded FSDB file. In the *protocol-aware* pane, the tree displays groups of streams that are based on different layers that have been defined for the protocol.

For example, in the following figure, the tree for the USB protocol groups all streams into one of the following layers: "Transfer", "Transaction", "Packets", "Link Items", and "LTSSM". All these are different layers within the USB protocol.

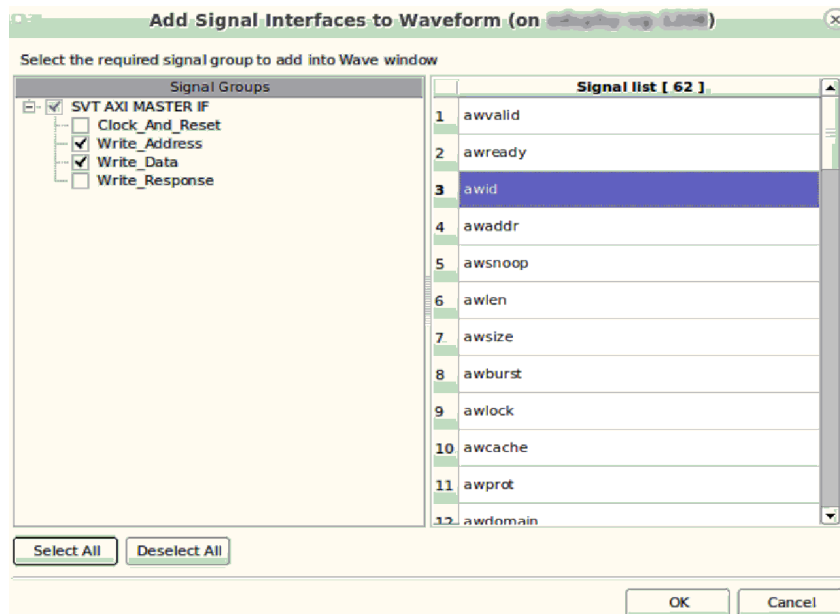


These groupings are helpful to decide which layer to open to debug. Double-click on each layer to open the content inside.

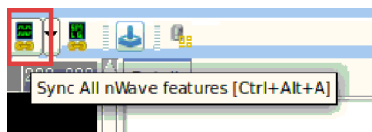
Add Signals to nWave

The **Add Signals To nWave** right-click command is provided to add signals from the loaded FSDB file to the *nWave* pane. Specify signal interfaces in the invoked *Add Signal Interfaces to Waveform* form and click the **OK** button.

Figure 112 Add Signal Interface to Waveform



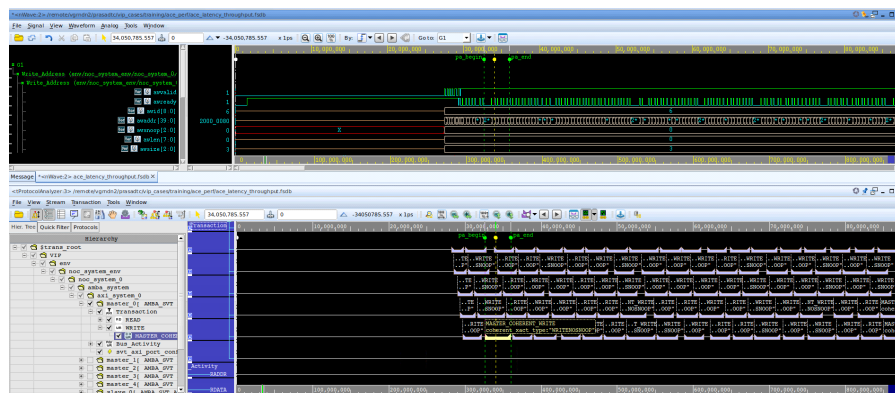
The **Sync Protocol Analyzer and nWave** option is automatically enabled, which syncs *Protocol Analyzer* and *nWave* as illustrated in the following figure:



The *Protocol Analyzer* synchronizes with *nWave* and marks the corresponding start time and end time of the selected object with cursors in the *nWave* pane. The *nWave* pane can then be used to analyze the signal activity between these two cursors.

Appendix A: Appendix A: Using Transaction and Protocol Analyzer (Protocol Debug Mode) Transaction and Protocol Analyzer Window Overview

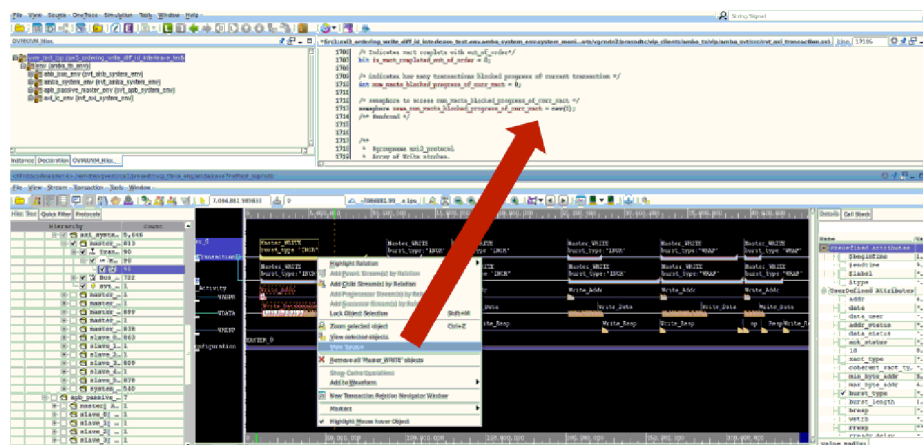
Figure 113 Protocol Analyzer Synchronized with nWave Pane



View Source

The **View Source** right-click command can be used to load the source code for the selected object into *nTrace*. Protocol Analyzer synchronizes with the source code from the hierarchy tree. You can see the corresponding class definition in the *Source Code* pane. If the source code has been loaded into Verdi, the corresponding class source code is displayed in the *Source Code* pane.

Figure 114 Protocol Analyzer Synchronized With Source Code Pane



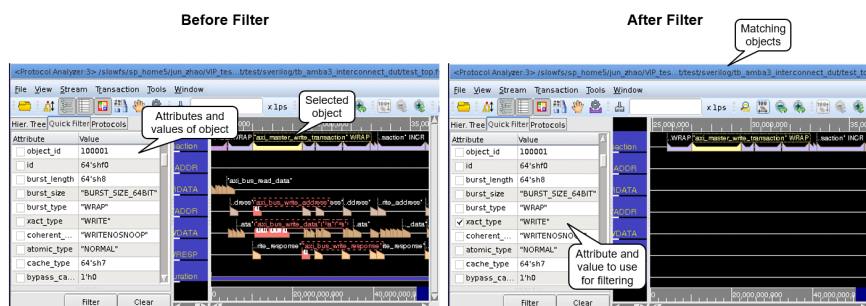
The *Count* column displays the object count instance, layer, or stream, adding up from bottom to top.

If a protocol definition is not available for a particular protocol, a warning message is reported and the hierarchy is shown without grouping.

Quick Filter

The *Quick Filter* tab provides the filtering feature for the protocol transaction content shown in the *Transaction* pane.

Select one transaction in the *Transaction* pane (for example, in the following figure, the selected transaction is a `Control` transfer); the attributes of this transaction are shown on the *Quick Filter* tab. These attributes and their corresponding values that can be modified can be used to perform quick filtering.

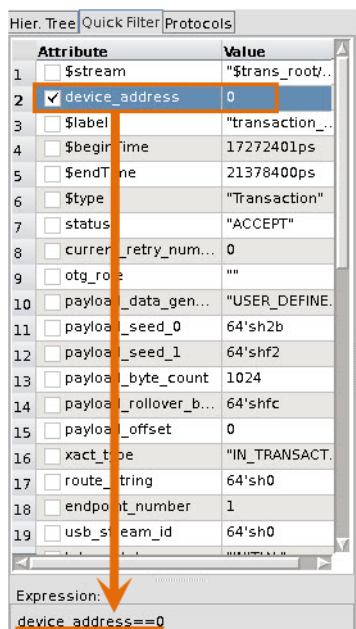


To set the filter criteria, select or de-select the attributes by checking or un-checking the corresponding boxes. Click the **Filter** button to hide the transactions that do not match the filter criteria. Only transactions that match the filter criteria are displayed in the *Transaction* pane. Filtering transactions based on attribute values simplify the debug process by narrowing down an issue to a smaller debug area.

For example, click the `device_address` field (refer to the figure below), the `device_address==0` expression is automatically displayed in the *Expression* field. Click the **Filter** button. The results of filtering are shown in the *Transaction* pane. Only transactions whose `device_address` attribute is equal to 0 is displayed.

Appendix A: Appendix A: Using Transaction and Protocol Analyzer (Protocol Debug Mode) Transaction and Protocol Analyzer Window Overview

Figure 115 Specify the Filter Condition

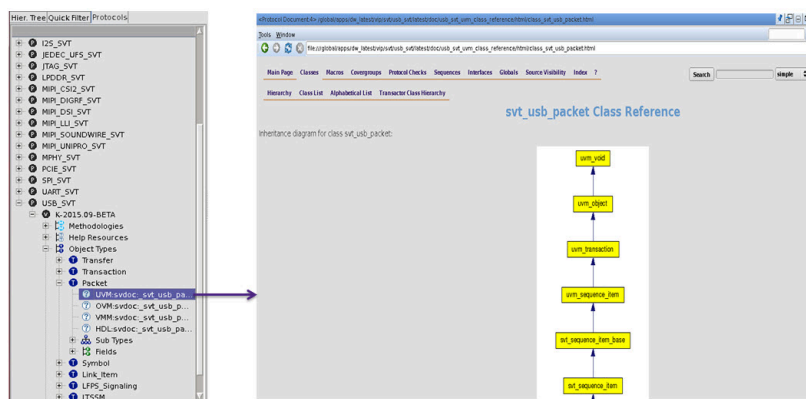


Protocols

The *protocols* tab displays all of the protocol definitions that have been read by the Protocol Analyzer. This tree is constructed hierarchy by reading the *protocol definitions* files from VIP installed in the path specified with the `DESIGNWARE_HOME` environment variable and *protocol definition* files for custom or proprietary protocols. The location of proprietary protocol definitions is specified using the `PA_CUSTOM_VIP_PATH` environment variable.

To view a document, double click on the document link in the *Protocols* tab. The document is opened in a new window.

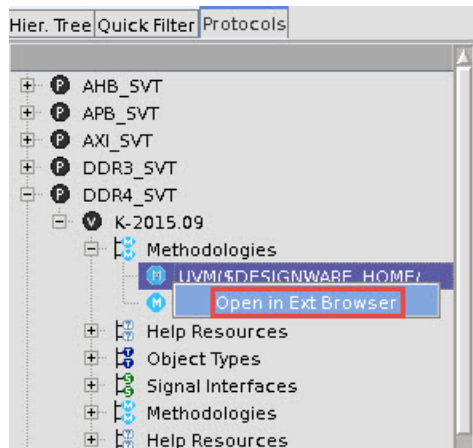
Figure 116 Protocol Tree and Document Tree



The class reference documents can be opened from the *protocols* tree in the embedded browser. The class reference document can be found for a methodology or accessed for a field from the *Details* tab.

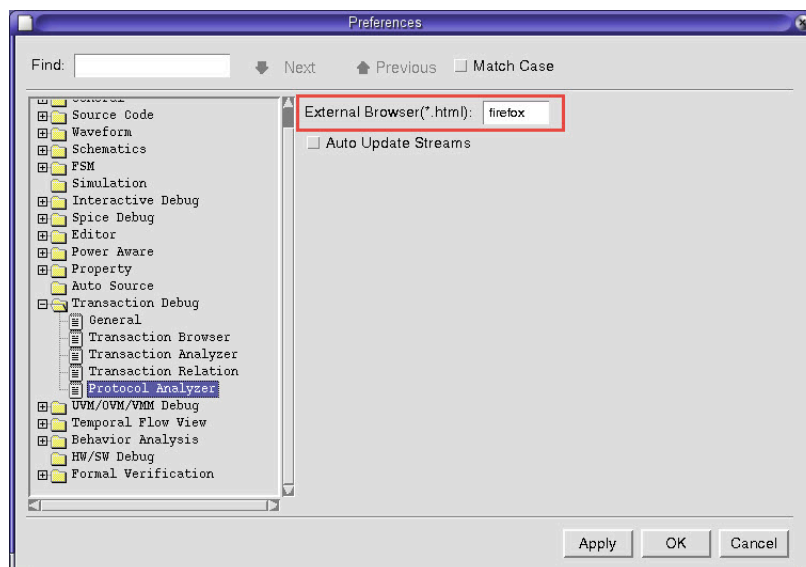
Open the document of the specified object with the **Open in Ext Browser** right-click command.

Figure 117 Open Document in External Browser



Specify the browser (Internet Explorer, Fire Fox, Chrome, and so on) in the *Transaction Debug -> Transaction and Protocol Analyzer* page of the *Preferences* form (invoked with the **Tools -> Preferences** command).

Figure 118 Protocol Analyzer External Browser Preference



Appendix A: Appendix A: Using Transaction and Protocol Analyzer (Protocol Debug Mode) Transaction and Protocol Analyzer Window Overview

The protocols tree also provides access to the VIP user guide PDF or other documents associated with the VIP. PDF documents are opened in Acroread as a separate process.

Log Annotation

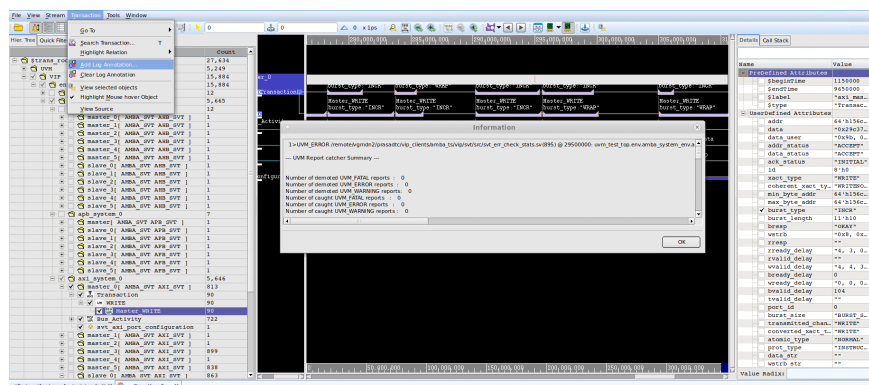
You can annotate the time and the severity of log messages in the *Transaction* pane with the **Transaction -> Add Log Annotation** command. When this command is enabled, the *Log Annotation* form is invoked. In this form, you can select the types of log messages to be annotated in the *Transaction* pane. One or more of the following message types can be selected: *Fatal*, *Error*, *Warning*, and *Info*.

For example, for the annotation of an *Info* message, a black line is drawn based on the time associated with the message. You can click on the line to invoke and view the log message. The message is displayed in a dialog that is drawn over the objects in the *Transaction* pane.

Note:

A log file must be loaded before invoking the **Add Log Annotation** command.

Figure 119 Log Annotation and Message Display in Transaction Pane

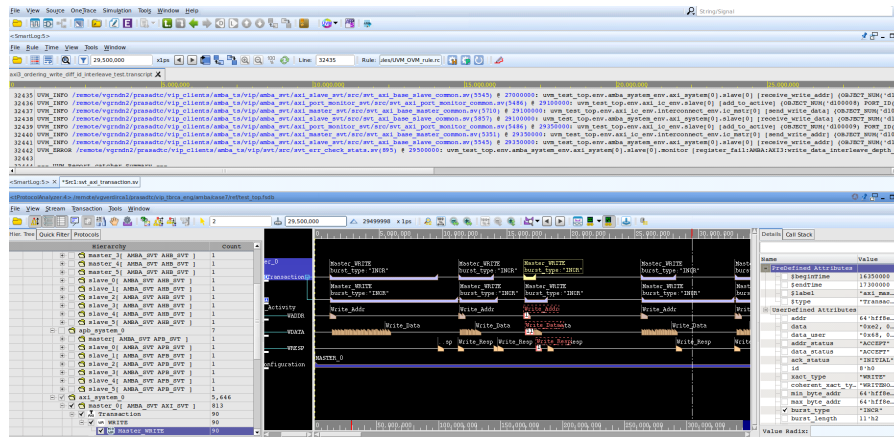


To view the complete simulation log, you can invoke the *Smart Log* pane by selecting the

Smart Log  icon in the *Verdi* pane. Click the  icon to synchronize the *Protocol Analyzer* pane with the *Smart Log* pane. Protocol analyzer marks the log position in the *Smart Log pane* for the selected transaction.

Appendix A: Appendix A: Using Transaction and Protocol Analyzer (Protocol Debug Mode) Transaction and Protocol Analyzer Window Overview

Figure 120 Synchronization with Smart Log Pane



Detail Pane

You can see *Details* and *Call Stack* tabs in the *Detail* pane.

Details

The *Details* tab displays the field and the value for the selected object.

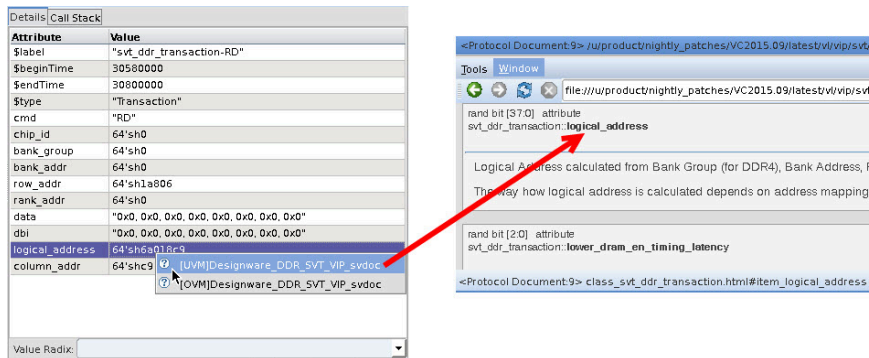
Figure 121 Details Tab

Attribute	Value
\$label	"svt_ddr_transaction-RD"
\$beginTime	30580000
\$endTime	30800000
\$type	"Transaction"
cmd	"RD"
chip_id	64'sh0
bank_group	64'sh0
bank_addr	64'sh0
row_addr	64'sh1a806
rank_addr	64'sh0
data	"0x0, 0x0, 0x0, 0x0, 0x0, 0x0, 0x0, 0x0"
dbi	"0x0, 0x0, 0x0, 0x0, 0x0, 0x0, 0x0, 0x0"
logical_addr...	64'sh6a018c9
column_addr	64'shc9

Value Radix: 16

You can select an object and use the right-click command to see the document for the particular field.

Figure 122 View Selected Field in Protocol Document Window

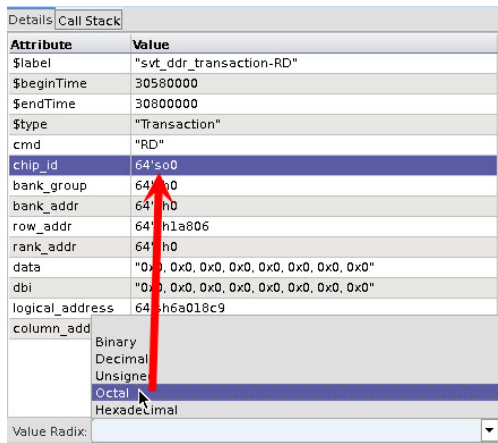


You can specify the format to display the value of the attribute from the **Value Radix** pull-down menu. The options include: **Binary**, **Decimal**, **Unsigned**, **Octal**, and **Hexadecimal**.

Note:

If the selected attribute is a string or time that is presented in a fixed way, the **Value Radix** pull-down menu is disabled.

Figure 123 Specify Radix Value



Call Stack

The complete call stack for the selected object is displayed in the *Call Stack* tab. You can check the call hierarchy. If the class design is loaded into *nTrace*, you can double-click a particular line in the call stack to view the corresponding code in the *Source Code* pane.

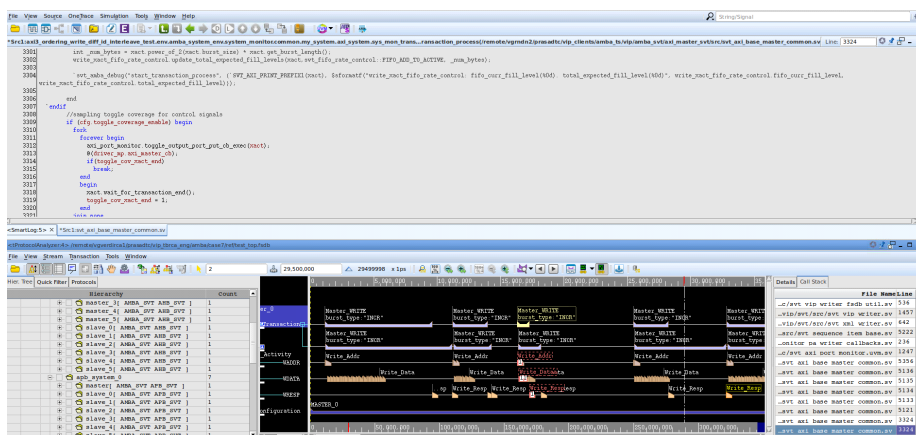
Appendix A: Appendix A: Using Transaction and Protocol Analyzer (Protocol Debug Mode) Transaction and Protocol Analyzer Window Overview

Figure 124 Call Stack Tab

Details	Call Stack
File Name	Line
.../vcs/svt_vip_writer.svp	1067
.../vcs/svt_xml_writer.svp	330
.../sequence_item_base.svp	2101
.../s/svt_usb_transfer.svp	5643
.../xml_gen_callbacks.svp	78
.../s/svt_usb_protocol.svp	7656
.../rotocol_processor.svp	10890
.../rotocol_processor.svp	10890
.../rotocol_processor.svp	10890
.../isoc_ep_processor.svp	12
.../isoc_ep_processor.svp	12
.../isoc_ep_processor.svp	12
isoc_ep_processor.svp	12

The *Call Stack* tab lists the path and the line number of all the source code files that were traversed up to the recording of the selected transaction. Double-click an item in the call stack to highlight the corresponding line in the *Source Code* pane.

Figure 125 Call Stack for Selected Transaction



Search Results

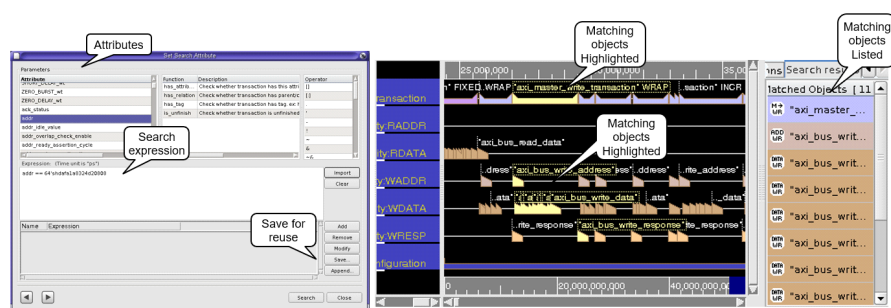
The *Search Results* tab displays all objects that match the search criteria that were specified using the Protocol Analyzer advanced search feature. You can traverse through the list of matching objects and select an object to view the object details. During interactive debug, the search results are updated with the objects that match the latest search criteria when the simulator stops.

Appendix A: Appendix A: Using Transaction and Protocol Analyzer (Protocol Debug Mode) Transaction and Protocol Analyzer Window Overview

To specify the transaction search criteria, use the **Transaction -> Search Transaction** command to invoke the *Set Search Attribute* form. If you plan to use the search criteria again, click the **Save** button to save the current search criteria as a *.list* file. Click the **Search** button to highlight transactions that match the search criteria in the *Transaction* pane. These transactions are displayed and stored in the **Search Results** tab simultaneously.

While the quick filter feature filters the object based on the AND condition, with the advanced search feature, you can define customized search criteria and save the criteria for future debug purposes.

Figure 126 Search Results Displayed in Search Results Tab



Note:

All the operations mentioned above can be used during interactive mode.

B

Appendix B: Automatically Record SCV Transactions

Verdi provides an FSDB transaction back-end for SystemC designs that uses SCV to capture transactions. This is useful for designs that have already been instrumented with SCV `scv_tr_` statements to record transactions. You just need to activate the FSDB transaction back-end provided by Verdi which is described in this appendix.

Note:

- The supported usage is available since Verdi 2014.12 in combination with VCS, VCS+Virtualizer, or IUS.
 - If your SystemC design has not yet been instrumented at all for transaction recording, then it is recommended that you use the SystemC transaction API described in the *VC APPS Native Programming Interface User Guide*.
-

Use Model

Use the following steps to enable this feature:

1. The `$VERDI_HOME/share/fsdbTrans_API/inc/scv_fsdbTrans_API.h` header file needs to be included and the `snps_scv::scv_fsdbTrans_init` function needs to be called in your SCV-code. For example:

```
...
#include scv_fsdbTrans_API.h
...
int sc_main(int argc, char* argv[])
{
    scv_tr_cwrdb_init();
    snps_scv::scv_fsdbTrans_init(argc, argv);
    ...
}
```

The following environment variables also need to be set to enable SystemC dumping feature in Verdi:

```
%> setenv VERDI3_SC_BETA 1
%> setenv FSDB_DUMP_RIDB 1
```

2. Include the directory of the header file:

```
%> g++ <g++ options> -I$VERDI_HOME/share/fsdbTrans_API/inc
```

3. Always link this Verdi library:

```
$VERDI_HOME/share/fsdbTrans_API/lib/$(FTAPI_PLATFORM)/libFTAPI.so
```

4. Also link the simulator-specific libraries:

```
$VERDI_HOME/share/PLI/<simulator>/<platform>/libsscore_<simulator_version>*_scv_fsdbTrans_*.so
```

For example,

VCS

```
$VERDI_HOME/share/PLI/VCS/LINUX/libsscore_vcs201412_sc230_scv_fsdbTrans_gcc47.so
```

VCS+Virtualizer

```
$VERDI_HOME/share/PLI/VCS/LINUX/libsscore_vcs201412_sc230_scv_fsdbTrans_gcc47_vir.so
```

IUS

```
$VERDI_HOME/share/PLI/IUS/LINUX/libsscore_ius141_scv_fsdbTrans_gcc44.so
```

5. Execution of the simulation. For example,

```
//VCS simulation
```

```
%> ./simv
```

```
//VCS+Virtualizer simulation
```

```
%> vpsession ./simv
```

```
//IUS simulation
```

```
%> ncsc_run <runtime options>
```

Limitations

The following are the limitations of this feature:

- This feature is not available for standalone Virtualizer. It is available only for Virtualizer with VCS co-simulation.
- Dumping of TLM ports is not supported.

- Interactive debug is not supported.
- Some internal Virtualizer modules are visible but should be hidden: `PAM_FACTORY`, `SCProfiler`, `SNPS_PAM`

C

Appendix C: Dumping Transaction Data Using SC-API

The functions of the SC-API have almost a one-to-one correspondence to the functions of the C-API. You can use this API in your SystemC-model to dump transactions conveniently to the FSDB file and observe the results in the Verdi platform.

The SC-API uses the C-API to implement its functionality and provides the following features by overloading some of the functions:

- All functions are prefixed with `sc_fsdbTrans`.
- SystemC time, SystemC scope, and the C call stack are automatically added as attributes to the recorded transactions and are displayed in the Verdi platform.

The following sections describe how you can use the SC-API and the environment settings for compilation and execution.

For more information on SC-API description, see the `$VERDI_HOME/share/fsdbTrans_API/inc/sc_fsdbTrans_API.h` header file.

Note:

- The SC-API can be used with the C-API. Both SC-API and C-API write the same FSDB file. All file, stream, and transaction handles are valid for both SC-API and C-API. This combined usage, although possible, is not recommended. It is recommended to use each API in its domain consistently, the SC-API within the SystemC code. This is easy because all C-API functions have a corresponding function in the SC-API. You can therefore use all C-API functions by adding the `sc_` prefix with the function name.
- When the Verdi dumper (using the `$fsdbDumpvars` command) and C-API or SC-API are used simultaneously to generate FSDB files, the FSDB file name for the Verdi dumper cannot be the same as the one for C-API and SC-API.

SC-API Descriptions

This section introduces the usage of the SC-API functions that are new or different from the C-API functions. Refer to the `$VERDI_HOME/share/fsdbTrans_API/inc/sc_fsdbTrans_API.h` header file for other one-to-one corresponding API functions.

fsdbTrans_file_id sc_fsdbTrans_file_open	
Syntax	<code>sc_fsdbTrans_file_open (const char* file_name);</code>
Description	Firstly, the <code>fsdbTrans_set_time_unit(...)</code> function is called to set the unit returned by the <code>sc_get_time_resolution()</code> SystemC function. Then the <code>fsdbTrans_file_open(filename)</code> function is called and the result is returned.
Arguments	
<code>file_name</code>	Name of the FSDB file to be opened.
Return Value	
<code>fsdbTrans_file_id</code>	Unique file id: If no error occurs. 0: If an error occurs while opening the FSDB file.

fsdbTrans_stream_id sc_fsdbTrans_create_stream_begin	
Syntax	<code>sc_fsdbTrans_create_stream_begin(fsdbTrans_file_id file_id [, const char* scope[, char delimiter]]);</code>
Description	If scope is omitted (which is the suggested usage), it is determined as the hierarchical path to transaction stream from the SystemC context. Then <code>fsdbTrans_create_stream_begin</code> is called to create a stream and return it's result as <code>stream_id</code> .
Arguments	
<code>file_id</code>	The return value from the call to <code>sc_fsdbTrans_file_open(...)</code>
<code>scope</code>	The hierarchical path to the transaction stream; per default determined automatically from the SystemC context if omitted.
<code>delimiter</code>	The hierarchical path delimiter, '.' per default if not specified otherwise.
Return Value	
<code>fsdbTrans_ret_value</code>	Unique stream id: If no error occurs. 0: If an error occurred while opening the transaction stream.

fsdbTrans_trans_id sc_fsdbTrans_begin	
Syntax	<code>sc_fsdbTrans_begin(fsdbTrans_stream_id sid, fsdbTrans_trans_type_e trans_type);</code>
Description	Firstly, the <code>fsdbTrans_set_time_stamp(...)</code> function is called with the time stamp from the <code>sc_core::sc_time_stamp()</code> SystemC function. Then, the <code>fsdbTrans_begin(sid, type)</code> function is called to create the transaction, and add the current “C” call stack as a <code>\$backtrace</code> attribute. Then, return the transaction ID. Use the <code>transaction_id</code> to create transaction details or add relationship between transactions. If the returned value is ‘0’, it indicates that something is wrong and the transaction cannot be created.
Arguments	
<code>sid</code>	Specifies the stream ID it belongs to.
<code>trans_type</code>	Specifies the transaction type. The Transaction Debug windows use different representations (GUI symbols) for each type. The <code>trans_type</code> attribute is one of the following: <code>fsdbTrans_MESSAGE</code> , <code>fsdbTrans_ACTION</code> , <code>fsdbTrans_TRANSACTION</code> or <code>fsdbTrans_GROUP</code> .
Return Value	
<code>fsdbTrans_trans_id</code>	Unique transaction ID: If no error occurs. 0: If an error occurs while opening the transaction.

fsdbTrans_ret_value_e sc_fsdbTrans_end	
Syntax	<code>sc_fsdbTrans_end (fsdbTrans_trans_id tid);</code>
Description	Firstly, the <code>fsdbTrans_set_time_stamp(...)</code> function is called with the time stamp from the SystemC function <code>sc_core::sc_time_stamp()</code>. Then, the <code>fsdbTrans_end(tid)</code> function is called to close the transaction and return its return value.
Arguments	
<code>tid</code>	Specifies the transaction ID to be closed.
Return Value	
<code>fsdbTrans_ret_value_e</code>	<code>fsdbTrans_RET_SUCCESS</code>: If no error occurs. <code>fsdbTrans_RET_WARNING</code>: If any violations of the above rules occur.

fsdbTrans_ret_value_e sc_fsdbTrans_add_attribute	
Syntax	<code>sc_fsdbTrans_add_attribute (fsdbTrans_trans_id tid, T attr)</code>
Description	Add an attribute of any data type into the <code>tid</code> transaction, which is the first argument. The second argument is an expression of an arbitrary type <code>T</code>, value <code>attr</code> and a readable representation <code>#attr</code>, that is, the variable name, or the literal expression (such as, <code>counter + 1</code>). This function creates an attribute with <code>attrname</code> equal to <code>#attr</code> and the value equal to <code>attr</code>. The type of the FSDB attribute depends on the type of the <code>attr</code> expression. The conversion rules are: • If the value is an integer type with 64 bits or less, an integer attribute of a matching width (8, 16, 32, or 64 bit) with decimal representation is created. • If the value is a pointer type, a matching 32 or 64bit integer attribute with hexadecimal representation is created. • If the value is a size limited SystemC integer type (<code>sc_int</code>, <code>sc_uint</code>, <code>sc_int_base</code>, <code>sc_uint_base</code>), a matching width (8, 16, 32, or 64 bit) integer attribute with decimal representation is created. • If the value is an unlimited SystemC integer type (<code>sc_signed</code>, <code>sc_unsigned</code>, <code>sc_bigint</code>, <code>sc_bignint</code>), and the length is > 64 bit, a string attribute is created; if the length is 64 bits or less, a matching width (8, 16, 32, or 64 bit) integer attribute with decimal representation is created. • If the value is of a floating or fixed point type that fits a float or a double, a float or double attribute is created. • For SystemC fast fixed point types (<code>sc_fixed_fast</code>, <code>sc_ufixed_fast</code>, <code>sc_fix_fast</code>, <code>sc_ufix_fast</code>, <code>sc_fxval_fast</code>), a double attribute is created. • All other data types are stored as string attributes, converted by the string stream operator of the data type (as required by all SystemC data types).
Arguments	
<code>tid</code>	Specifies the transaction. An error is issued if this is not a valid transaction id (obtained from the <code>sc_fsdbTrans_begin()</code> API). An error is issued if <code>sc_fsdbTrans_end()</code> is already called for <code>tid</code>, and this call has no effect.
<code>attrname</code>	Specifies the name of the attribute being added. A warning is issued if <code>attrname</code> exceeds the maximum length and <code>attrname</code> is truncated.
<code>attr</code>	Specifies the value of the attribute being added. The data type of this value determines the FSDB attribute type (refer to the conversion rules in the Description field).

Return Value	
<code>fsdbTrans_ret_value_e</code>	<code>fsdbTrans_RET_SUCCESS</code>: If no error is issued. <code>fsdbTrans_RET_ERROR</code>: If any error is issued for the above rules. <code>fsdbTrans_RET_WARNING</code>: If only warning (no error) is issued for the above rules.

fsdbTrans_ret_value_e sc_fsdbTrans_add_T_attribute	
Syntax	<code>sc_fsdbTrans_add_T_attribute (fsdbTrans_trans_id tid, const char* attrname, T attr)</code>
Description	Add an attribute of any data type into the <code>tid</code> transaction. This function implements the previous function <code>sc_fsdbTrans_add_attribute</code> and also offers a way to manually control the attribute name. Conversion rules of the previous function apply.
Arguments	
<code>tid</code>	Specifies the transaction. Error if this is not a valid transaction id (obtained from the <code>sc_fsdbTrans_begin()</code> API). Error if <code>sc_fsdbTrans_end()</code> is already called for <code>tid</code> , and this call has no effect.
<code>attrname</code>	Specifies the name of the attribute being added. Warning if <code>attrname</code> exceeds the maximum length and <code>attrname</code> is truncated.
<code>attr</code>	Specifies the value of the attribute being added. The data type of this value determines the FSDb attribute type (refer to the conversion rules in the Description field of <code>sc_fsdbTrans_add_attribute</code>).
Return Value	
<code>fsdbTrans_ret_value_e</code>	<code>fsdbTrans_RET_SUCCESS</code>: If no error is issued. <code>fsdbTrans_RET_ERROR</code>: If any error is issued for the above rules. <code>fsdbTrans_RET_WARNING</code>: If only warning (no error) is issued for the above rules.

Execution Flow

The following are the requirements to use the SC-API:

- The following header file must be included in the C-code:
`$VERDI_HOME/share/fsdbTrans_API/inc/sc_fsdbTrans_API.h`
- SystemC must be installed (refer to the SystemC documentation of the simulator vendor for reference).
- Other requirements are the same as for the C-API.

Example Using VCS

Use the following steps and the example to use SC-API for transaction dumping.

1. Change the current directory to the example path:

```
%> cd <Verdi install path>/demo/transaction/systemc_api
```

2. Use the source file to set the environment variables and VCS setting:

```
%> source sourceMeVCS
```

3. Use the following command to compile, link the `main.cpp` example file, and execute the simulation:

```
%> syscan -cflags -g main.cpp \  
-cflags -I$VERDI_HOME/share/fsdbTrans_API/inc  
  
%> vcs -full64 -sysc sc_main -debug_access+r -lca \  
-timescale=1ns/1ps \  
$VERDI_HOME/share/fsdbTrans_API/lib/LINUX64/libFTAPI.so \  
  
%> ./simv -ucli -do dump.ucli
```

The `default.ridb`, `novas.fsdb`, and `sc_trans.fsdb` files are generated in the current working directory. The SystemC signals are dumped into the `novas.fsdb` file and SystemC transactions are dumped into the `sc_trans.fsdb` file.

4. Invoke the Verdi platform with the `verdi_cmd.tcl` Tcl file and start debugging at both signal and transaction levels:

```
%> verdi -ridb default.ridb -ssf novas.fsdb -play verdi_cmd.tcl  
-nologo &
```

5. You can also use the `runMeVCS` script to go through step 3 to step 4:

```
%> source runMeVCS
```

Example Using IUS

Use the following steps and the example to use the SC-API for transaction dumping.

1. Change the current directory to the example path:

```
%> cd <Verdi install path>/demo/transaction/systemc_api
```

2. Use the source file to set the environment variables and IUS setting:

```
%> source sourceMeIUS
```

3. Use the following command to compile, link the main.cpp example file, and execute the simulation:

```
ncsc_run -64BIT main.cpp \  
-Wcxx,-g \  
-Wcxx,-DSC_INCLUDE_DYNAMIC_PROCESSES \  
-Wcxx,-I$VERDI_HOME/share/fsdbTrans_API/inc \  
$VERDI_HOME/share/fsdbTrans_API/lib/${IUS_PLATFORM}/libFTAPI.so \  
-lrt -sysc -top sc_main \  
-tcl -input dump.tcl -access +r
```

The `default.ridb`, `novas.fsdb`, and `sc_trans.fsdb` files are generated in the current working directory. The SystemC signals are dumped into the `novas.fsdb` file and SystemC transactions are dumped into the `sc_trans.fsdb` file.

4. Invoke the Verdi platform with the `verdi_cmd.tcl` Tcl file and start debugging at both signal and transaction levels:

```
%> verdi -ridb default.ridb -ssf novas.fsdb -play verdi_cmd.tcl  
-nologo &
```

5. You can also use the `runMeICS` script to go through step 3 to step 4:

```
%> source runMeIUS
```

D

Appendix D: Automatically Recording OVM Transactions and Messages

Use the following simple methods to enable automatic OVM transaction and message recording.

Compiling and Linking a Design

For the VCS simulator:

1. The FSDB dumper needs to be specified at the beginning. For example:

```
%> setenv tab $VERDI_HOME/share/PLI/VCS/LINUXAMD64/novas.tab
%> setenv pli $VERDI_HOME/share/PLI/VCS/LINUXAMD64/pli.a
```

2. Specify the following environment variable before compilation:

```
%> setenv VCS_OVM_HOME $VERDI_HOME/share/vmlib/ovm/ovm/lib/src
```

3. Add the `-ntb_opts ovm` compile option for OVM library. For example:

```
%> vcs -full64 -debug_access+all -sverilog -P $tab $pli \
-l compile.log -ntb_opts ovm \
-assert dve -assert api +memcbk +vcs+lic+wait ./xbus_tb_top.sv
+incdir+. +incdir+../sv +incdir+../../src \
```

4. Add the `+OVM_TR_RECORD` run time option to dump transactions. For example:

```
%> ./simv +OVM_TESTNAME=test_read_modify_xbus_tb_top
+fsdb+trans_begin_callstack +OVM_TR_RECORD
```

For other simulators:

1. Compile the design with the Verdi OVM library. For example:

```
%> vlog +incdir+../sv
+incdir+$OVM_HOME/src $OVM_HOME/src/ovm_pkg.sv xbus_tb_top.sv
```

2. Add the `+OVM_TR_RECORD` run time option to dump transactions. For example:

```
%> vsim +OVM_TR_RECORD +OVM_TESTNAME=test_read_modify_write -pli
novas_fli.so -do vsim.do -c -suppress 3829 xbus_tb_top
```

Support for UVM Aware Debug

Two runtime options are available to dump the information for UVM Aware Debug and subsequently debug the OVM design with UVM Aware Debug application.

The runtime options `+UVM_VERDI_TRACE` or `+OVM_VERDI_TRACE` enable specific Verdi features such as component level transaction recording, port level transaction recording, factory dumping, and dynamic hierarchy tree dumping. Both the options dump the same information for UVM Aware Debug and display identical behavior.

The following table lists the format and details.

Table 4 *UVM_VERDI_TRACE/OVM_VERDI_TRACE Arguments*

+UVM_VERDI_TRACE	
Usage	<code>+UVM_VERDI_TRACE = UVM_AWARE FAC TLM HIER</code> <code>+OVM_VERDI_TRACE = UVM_AWARE FAC TLM HIER</code>
Arguments	
UVM_AWARE	If this argument is specified, similar to <code>UVM_CONFIG_DB_TRACE</code> , configuration information is dumped. Factory override trace messages are also enabled. Dynamic hierarchy tree dumping are enabled.
TLM	Port level transaction recording is enabled. The <code>+OVM_TR_RECORD</code> option is automatically enabled to record transactions into the FSDB file.
TLM+IMP	Both port and implementation port level transaction recording is enabled. Note that IMP must be used with TLM. The <code>+OVM_TR_RECORD</code> option is automatically enabled to record transactions into the FSDB file.
HIER	Dynamic hierarchy tree dumping is enabled.
FAC	Factory dumping is enabled.

- **+define+OVM_VERDI_NO_FACTORY_RECORDING**: Add this compile option to disable the compilation of instrumented codes for factory recording.
- **+OVM_VERDI_TRACE=FAC**: Add this runtime option to enable factory recording.

Multiple features can be enabled with the “|” operator.

For example:

You can enable port level transaction recording and message dumping with the `+OVM_VERDI_TRACE=TLM|HIER` compound argument:

```
>./simv +OVM_VERBOSITY=OVM_LOW +OVM_TESTNAME=test_read_modify_write  
+OVM_VERDI_TRACE="TLM|HIER"
```

Note:

You need to use the Transaction Debug features described in previous chapters to recognize transactions and messages in the `novas.fsdb` transaction-based FSDB file.

Recording Transactions for Non-Sequencer Components

If automatic recording for non-sequencer components (for example, drivers or monitors) is required, you need to modify the code of the components to explicitly call `void'(this.begin_tr())` and `void'(this.end_tr())` to trigger the recording hooks.

Note:

You need to make sure `begin_tr()` and `end_tr()` are called in pairs.

Here is an example to explicitly call `void'(this.begin_tr())` and `void'(this.end_tr())` in a wishbone monitor:

```
class uart_base_monitor extends ovm_monitor;  
    function void detect_bus_event(uart_bus_transaction bus_trans);  
        m_bus_port.write(bus_trans);  
        this.end_tr(bus_trans); // By Synopsys  
    endfunction : detect_bus_event  
endclass : uart_base_monitor  
  
class uart_wishbone_monitor extends uart_base_monitor;  
    //@ run related  
    task run();  
        //@ detect and put transaction  
        fork  
            monitor_addr();  
            monitor_sel();  
            monitor_dat_i();  
            monitor_dat_o();  
            monitor_we();  
            monitor_stb();  
            monitor_cyc();  
            monitor_ack();  
            monitor_int();  
            monitor_baud();  
        join_none  
    endtask : run
```

Appendix D: Automatically Recording OVM Transactions and Messages Recording Transactions for Non-Sequencer Components

```

task monitor_addr();
    uart_bus_transaction    bus_trans;
    forever begin
        @m_bus_if.wb_addr_i;
        //@ bus event
        $cast(bus_trans , m_bus_trans.clone());
        void'(this.begin_tr(bus_trans)); // By Verdi
        bus_trans.m_signal_type = uart_bus_transaction::WB_ADR_I;
        bus_trans.m_new_value = { 26'b0, m_bus_if.wb_addr_i };
        detect_bus_event(bus_trans);
    end
endtask : monitor_addr

task monitor_sel();
    uart_bus_transaction    bus_trans;
    forever begin
        @m_bus_if.wb_sel_i;
        //@ bus event
        $cast(bus_trans , m_bus_trans.clone());
        void'(this.begin_tr(bus_trans)); // By Verdi
        bus_trans.m_signal_type = uart_bus_transaction::WB_SEL_I;
        bus_trans.m_new_value = { 27'b0, m_bus_if.wb_sel_i };
        detect_bus_event(bus_trans);
    end
endtask : monitor_sel
endclass : uart_wishbone_monitor

```

E

Appendix E: Using the Old Transaction Table View

Note:

The content in this chapter is the previous behavior of the tool, and it is being planned to phase out. Refer to the *Verdi Migration Guide* or contact Synopsys Support Center to roll back to the old function.

The *Transaction Table View* window displays streams in the FSDB file as a spreadsheet view of transaction types. The contents are based on data-mining (during loading) or other operations (for example, drag and drop streams from other windows or created virtual streams). You can read and analyze the streams by the tabular highlight form. The relationship between transactions can also be shown in the *Transaction Table View* window.

The following sections describe the usage of *Transaction Table View*:

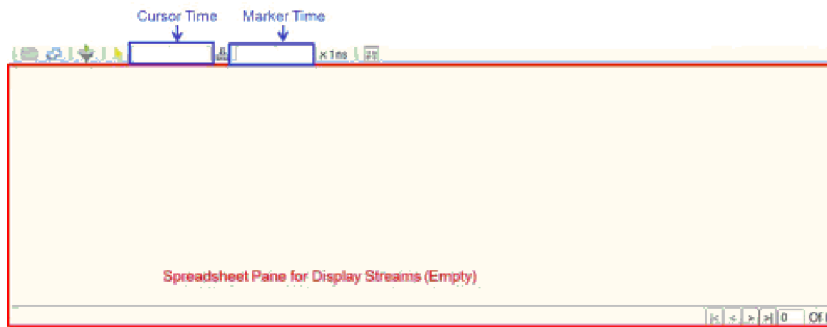
- [Importing FSDB to the Transaction Table View Window](#)
- [Transaction Table View Window](#)
- [Reading and Analyzing Transactions in Transaction Table View](#)
- [Showing Transaction Relation Navigator](#)
- [Auto-Switching During Sync Selection](#)

Importing FSDB to the Transaction Table View Window

Use the **Tools -> New Transaction Table View** command in the *Transaction and Protocol Analyzer* window after invoking the Verdi platform or directly use the default window that was opened when invoking *Transaction Debug Mode*.

As shown in [Figure: Empty Transaction Table View Window](#), the *Transaction Table View* window displays the empty stream tabs pane, Cursor Time, and Marker Time fields by default.

Figure 127 Empty Transaction Table View Window



Click the  button directly or select the **File -> Open** command to import the FSDB file. If the *Transaction Table View* window is invoked from the *Transaction and Protocol Analyzer* window (by selecting the **Tools -> New Transaction Table View Window** command), the FSDB file that was opened in the *Transaction and Protocol Analyzer* window is automatically imported into the *Transaction Table View* window. One *Transaction Table View* window only can import one FSDB file; however, you can open multiple *Transaction Table View* windows as needed to observe multiple FSDB files. The virtual FSDB file is not supported currently.

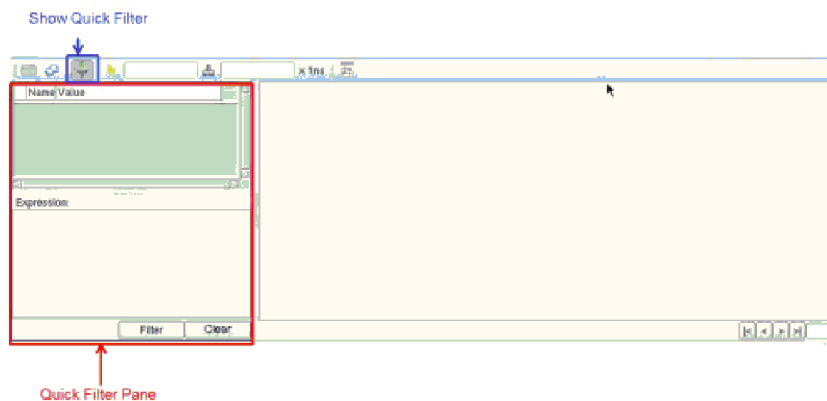
Note:

The *Transaction Table View* window cleans all data in the window and closes the FSDB file if the FSDB file is reloaded in other windows, that is, the *Transaction and Protocol Analyzer* window.

Transaction Table View Window

After importing an FSDB file, invoke the **Show Quick Filter** command by clicking the  button or selecting the **View -> Show Quick Filter** command. The *Transaction Table View* window is illustrated in [Figure: Invoke the Hierarchy Tree/Quick Filter Pane](#), with the *Quick Filter* pane added to the left side. The *Quick Filter* pane shows the selected transaction information inline from the spreadsheet pane. See the next sections for details about the *Transaction Table View* window functions.

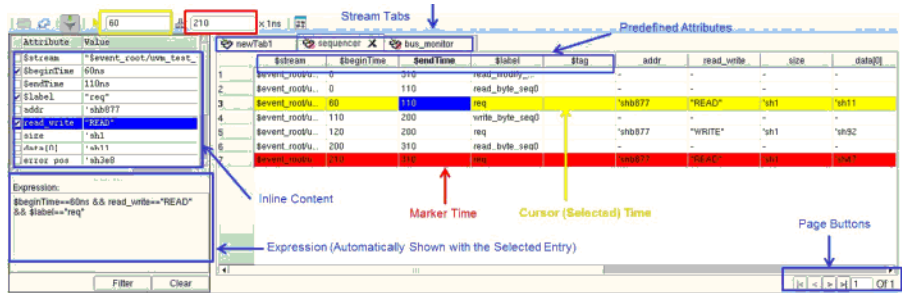
Figure 128 Invoke the Hierarchy Tree/Quick Filter Pane



As shown in [Figure: Transaction Table View Window Overview](#), the spreadsheet pane displays all imported streams in each tab and all transactions of the stream per tab. Each tab contains one stream. A row shows a transaction and the columns show all normal attributes and visible predefined attributes prefixed with the dollar (\$) character. When the number of rows reaches a pre-defined number, the contents of this tab are shown by page and the **page** buttons are provided to change pages easily. The predefined number is 30000 currently and you can change the default value in the *Transaction Table View* page under the *Transaction Debug* folder in the *Preferences* form opened by the **Tools -> Preferences** command. The selected transaction is displayed in yellow color and the marked row is displayed in red color. The transaction time is also shown in the **Cursor Time** and **Marker Time** fields. The *Quick Filter* pane shows the attributes of the selected transaction inline (see [Figure: Transaction Table View Window Overview](#)). Tabs are divided into two categories: non-editable and editable:

- **Non-editable tab (with the  icon before the tag name):** These tabs cannot be edited after adding the streams. A stream is the same as the stream displayed in the *Transaction and Protocol Analyzer* window and is shown by a tabular view in the tab.
- **Editable tab (with the  icon before the tag name):** A container for data mining. Transactions can be added from different streams, can be deleted, and then can be referenced for creating a virtual stream.

Figure 129 Transaction Table View Window Overview



When the attributes of the transaction have expected value, the row of the transaction is highlighted in magenta. Additionally, the attributes are shown with the (*expected value*) suffix in the column.

Figure 130 Attributes with Expected Values

test_top-root0 Packets:RX												test_top-root0 Packets:TX											
	ep	length	payload	status	completer_id	completion_status	rite_count	modifi	byte_count	lower_address	nak_seq	ecrc											
10		-	-	-	-	-	-	-	-	-	-	-											
11	h0	4096'h10	"8C72471c893..."	"GOOD"	4096'h100	4096'h0	1'h0		4096'h289	4096'h0	-	-											
12	h0	4096'h20	"fcd66f23038..."	"GOOD"	4096'h100	4096'h0	1'h0		4096'h249	4096'h40	-	-											
13	h0	4096'h10	"51cca0b161d..."	"GOOD"	4096'h100	4096'h0	1'h0		4096'h1c9	4096'h40	-	4096'h4feb62											
14	h0	4096'h10	"eba34418e2..."	"GOOD"	4096'h100	4096'h0	1'h0		4096'h189	4096'h0	-	-											
15	h0	4096'h10	"fa3669b2b24..."	"GOOD"	4096'h100	4096'h0	1'h0		4096'h14	4096'h4feb628c(4096'h0)	-	-											
16	h0	4096'h20	"074f2494948..."	"GOOD"	4096'h100	4096'h0	1'h0		4096'h109	4096'h0	-	4096'h996319											
17	h0	4096'h20	"73bcae48235..."	"GOOD"	4096'h100	4096'h0	1'h0		4096'h89	4096'h0	-	4096'ha57eb7											
18	h0	4096'h3	"20d00ef1db0..."	"GOOD"	4096'h100	4096'h0	1'h0		4096'h9	4096'h0	-	-											
19	h0	4096'ha	"00008c72c2..."	"GOOD"	4096'h100	4096'h0	1'h0		4096'h18b	4096'ja	-	-											

Attribute with expected value suffix in brackets

Attributes with Expected Value

Reading and Analyzing Transactions in Transaction Table View

This section contains the following sub-sections:

- [Importing Streams into Transaction Table View](#)
- [Column \(Attribute\) Operations](#)
- [Row \(Transaction\) Operations](#)

Importing Streams into Transaction Table View

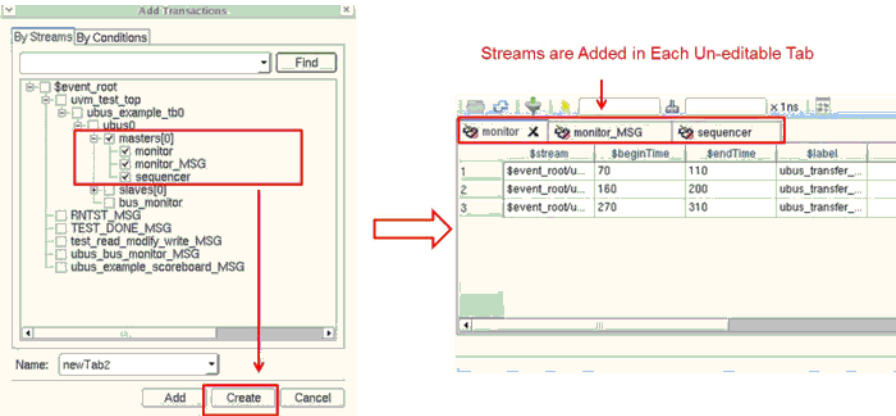
After importing an FSDB file into the *Transaction and Protocol Analyzer* window, import the streams with different situations:

When the spreadsheet pane is empty (see [Figure: Invoke the Hierarchy Tree/Quick Filter Pane](#) and [Figure: Transaction Table View Window Overview](#)), use the following methods:

Method	Usage	Stream Added Result
--------	-------	---------------------

1	Display the Add Transaction form by using the Transaction -> Add Transaction command or clicking the  icon. The <i>Add Transaction</i> form shows full hierarchy of streams and shows all streams for selection by user. Select the streams and click the Create button. The string entered in the Find field filters and leaves the partial matching scopes and streams.	The streams are added in each non-editable tab. See figure Add Streams in Each Tab . If the streams exist in the tab bar, the creation of these streams is skipped.
2	Drag and drop streams or transactions from the <i>Transaction and Protocol Analyzer</i> window.	All the dropped streams are added in an editable tab. See figure Add Streams into One Tab .
	Right-click on the empty spreadsheet pane and select the New Tab command from the menu. Add data to this tab by: a) Dragging and dropping streams or transactions from the <i>Transaction and Protocol Analyzer</i> window to the newTab pane. b) Clicking the  icon, selecting the streams and clicking the Add button (See figure Add Streams into New Tab). c) Clicking the  icon, selecting the streams and clicking the Create button.	a) The same as Method 2. b) The same as Method 2. All the selected streams are added in this new editable tab. c) The same as Method 1.

Figure 131 Add Streams in Each Tab



Appendix E: Appendix E: Using the Old Transaction Table View Reading and Analyzing Transactions in Transaction Table View

Figure 132 Add Streams into One Tab

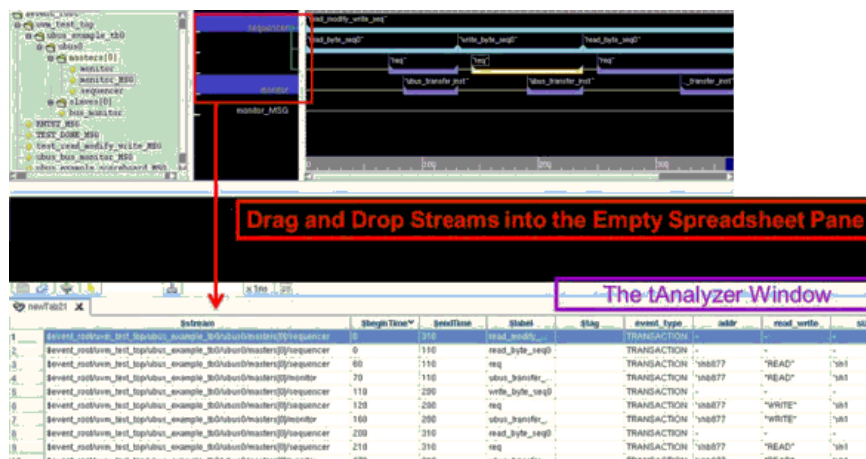
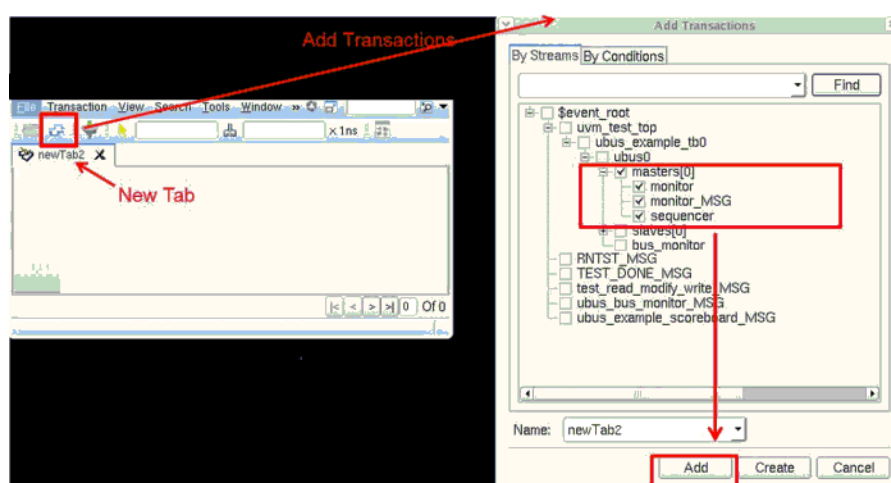


Figure 133 Add Streams into New Tab



When the spreadsheet pane is a non-editable stream tab:

Method	Usage	Stream Added Result
1	Click the  icon. Select the streams and click the <i>Create</i> button. See Figure: Add Streams with Non-editable Tab , the already opened stream is colored blue.	The streams are added in each non-editable tab. See Figure: Add Streams in Each Tab .
2	Drag and drop streams to the tab bar on the spreadsheet pane from the <i>Transaction and Protocol Analyzer</i> window.	The streams are added in each non-editable tab. The tab is switched from the current tab to the dropped stream tab if the stream already exists in the spreadsheet pane.

Appendix E: Appendix E: Using the Old Transaction Table View Reading and Analyzing Transactions in Transaction Table View

3	Right-click on the tab bar and select the New Tab command from the menu (see Figure: Invoke the New Tab Command). Add data to this tab by: a) Dragging and dropping streams or transactions from the <i>Transaction and Protocol Analyzer</i> window to the <i>newTab</i> pane. b) Clicking the  icon, selecting the streams, and clicking the Add button. c) Clicking the  icon, selecting the streams, and clicking the Create button.	a) All the dropped streams are added in this new editable tab. b) All the selected streams are added in this new editable tab. c) The same as Method 1. If the streams exist in the tab bar, the creation of these streams is skipped.
---	---	---

Figure 134 Add Streams with Non-editable Tab

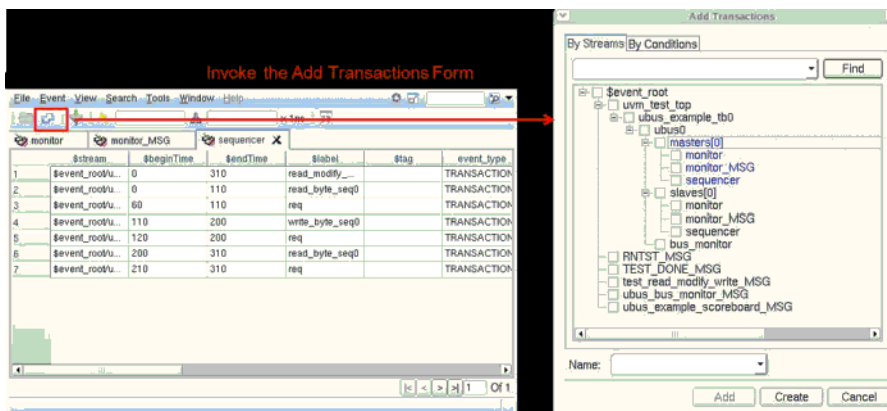
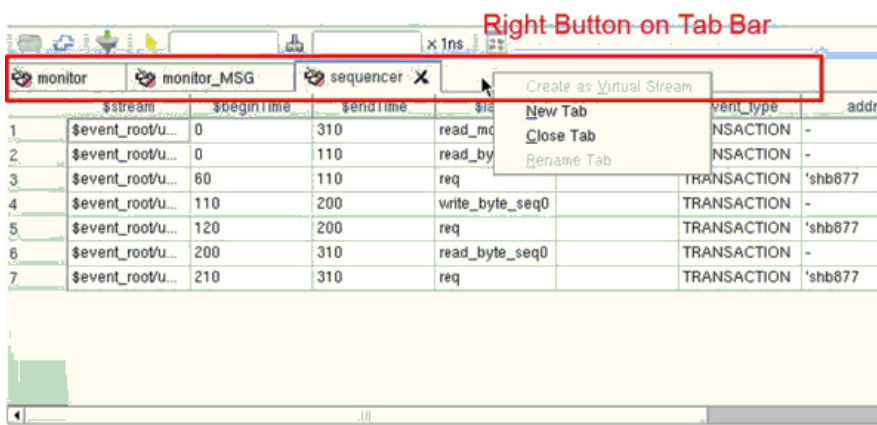


Figure 135 Invoke the New Tab Command



When the spreadsheet pane is an editable stream tab:

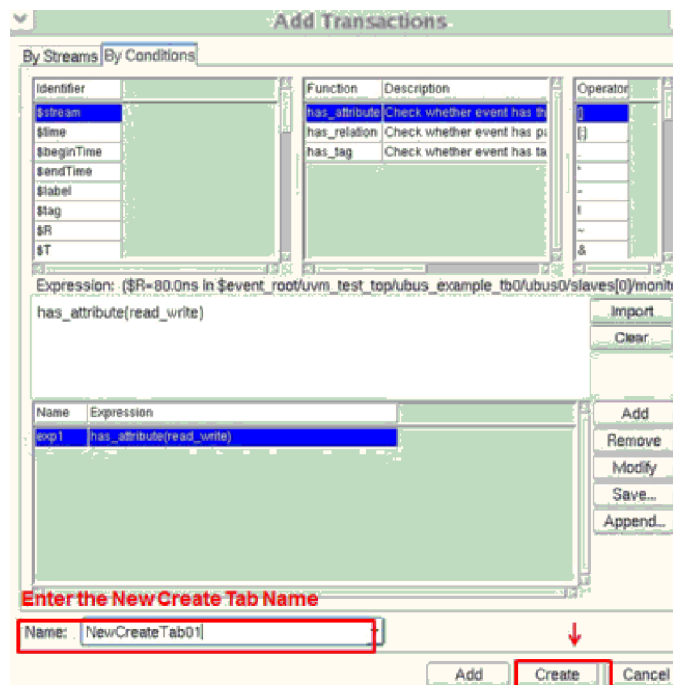
Method	Usage	Stream Added Result
1	Click the  icon. Select the streams and click the Create button. See Figure: Add Streams with Non-editable Tab , the already opened stream is colored blue.	The streams are added in each non-editable tab. See Figure: Add Streams in Each Tab .
2	Click the  icon. Select the streams and click the Add button. The already opened stream is colored blue.	The streams are added in this editable tab. If the streams or transactions exist in this tab, the rows are selected and highlighted in yellow.
3	Drag and drop streams to the tab bar on the spreadsheet pane from the <i>Transaction and Protocol Analyzer</i> window.	The streams are added in each non-editable tab. The tab is switched from the current tab to the dropped stream tab if the stream already exists in the spreadsheet pane.
4	Drag and drop streams or transactions from the <i>Transaction and Protocol Analyzer</i> window to this editable tab.	All the dropped streams are added in this editable tab. If the streams or transactions exist in this tab, the rows are selected and highlighted in yellow.
5	Right-click on the tab bar and select the New Tab command from the menu (see Figure: Invoke the New Tab Command) Add data to this tab by: a) Dragging and dropping streams or transactions from the <i>Transaction and Protocol Analyzer</i> window to the <i>new Tab</i> pane. b) Clicking the  icon, selecting the streams, and clicking the Add button (See Figure: Add Streams into New Tab). c) Clicking the  icon, selecting the streams, and clicking the Create button.	a) All the dropped streams are added in this new editable tab. b) All the selected streams are added in this new editable tab. c) The same as Method 1.

Add Transactions by Conditions

Click the  icon and click the **By Conditions** label in the *Add Transaction* form to switch to the **By Conditions** tab. All of the predefined attributes and normal attributes are listed in the *Identifier* column. The usage is the same as the description of creating a virtual stream in the *Transaction and Protocol Analyzer* window in the [Creating Virtual Streams](#) section. After completing the expression input, select the editable tabs that are listed in the *Name* column and click the lower **Add** button to add the transactions that match the

expression to the existing editable tab. Alternatively, you can enter a new name in the **Name** column and click the **Create** button to add these transactions to a newly created tab (see [Figure: Add Transactions by Conditions](#)). This operation of adding transactions by conditions is only supported in an editable tab.

Figure 136 Add Transactions by Conditions



You can save or append the added conditions to a list file by clicking **Save** or **Append** buttons.

After creating the stream tabs, you can create the virtual stream containing current transactions in the current tab using the **Transaction -> Create as Virtual Stream** command or right-click to select the **Create as Virtual Stream** command (see [Figure: Create as Virtual Stream](#)) and input the virtual stream name. This command cannot be invoked when the tab is empty or non-editable. See [Figure: Virtual Stream Root](#), the added virtual streams are listed in the *VirtualStreamRoot* directory and can be added to the spreadsheet pane. The virtual stream tab is non-editable.

Figure 137 Create as Virtual Stream

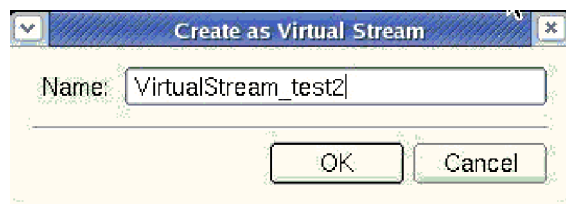
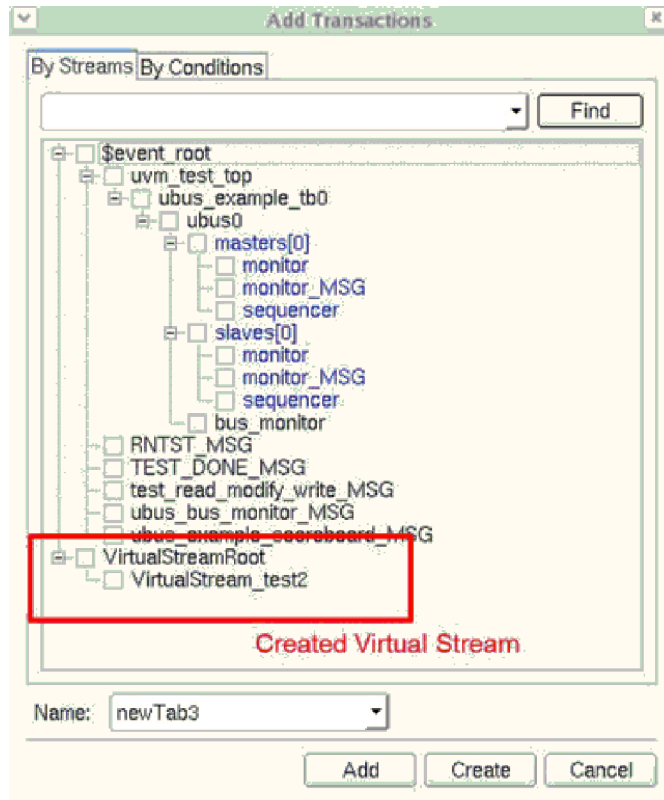


Figure 138 Virtual Stream Root



The virtual stream root is also refreshed if the virtual stream was created or deleted by the *Transaction and Protocol Analyzer* window.

Column (Attribute) Operations

By default, after creating the tabs, the visible predefined attributes, including streams, `begin_Time`, `end_Time`, `label`, and `tag`, and all the normal attributes are listed by columns in one tab.

The same type of VIP transactions (from the same top-level channel) are grouped into one tab with the tab naming convention: *instance_name:column_name*. If the transactions come from different channels, these are grouped into different tabs with the tab naming convention: *instance_name:column_name:channel_name*.

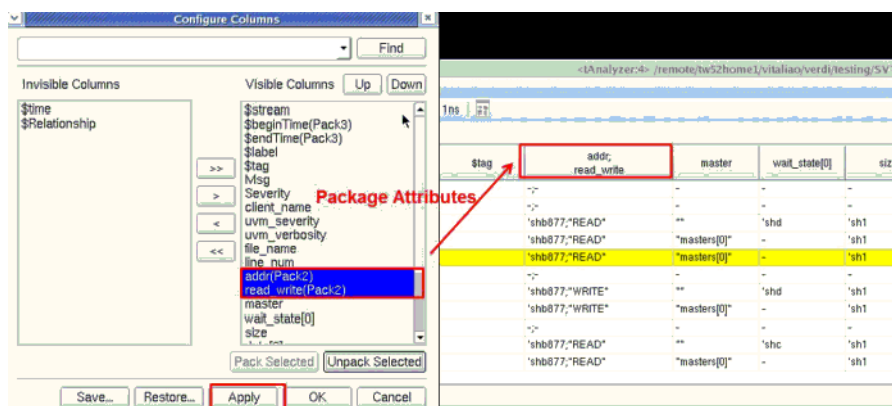
For VIP transactions, all the attributes are displayed based on the pre-defined order decided by the VIP. The ordering of the VIP attributes displayed in the tab is as follows:

- Firstly, the predefined attributes in the XML file with the `field_order` keyword are displayed.
- Other attributes' order follows the current column configuration defined using the **View -> Configure Columns** command (as described below).

You can use the **View -> Configure Columns** command to design the tab format in the *Configure Columns* form, that is, hide/show some attributes or pack some attributes into one column. The string entered in the **Find** field finds the partial matched attributes in both lists and the results are highlighted. The *Configure Columns* form displays attributes in the **Invisible Column** and **Visible Column** fields so you can easily understand which attributes are visible or not. The following paragraphs describe the operations applied in columns.

- **Visible/Invisible Columns:** Use the arrow buttons in the *Configure Columns* form to decide the visibility of columns in the spreadsheet pane.
- **Pack/Unpack Attributes:** Attributes can be combined to one column. Select the attributes and click the **Pack Selected** button in the *Configure Columns* form (see [Figure: Pack the Attributes](#)). The names of packed attributes are added with the (PackNumber) suffix. After clicking the **Apply** button, the columns are combined to one column with the concatenated names (for example (addr;read_write)). Click the **Unpack Selected** button to unpack these attributes.

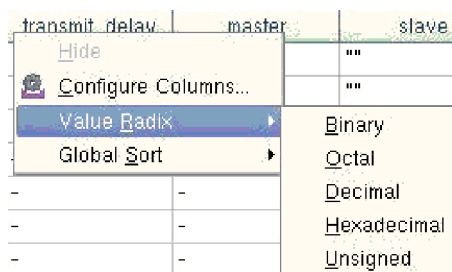
Figure 139 Pack the Attributes



- **Column Order:** The column order of the spreadsheet pane can be changed using the **Up** and the **Down** buttons (see [Figure: Pack the Attributes](#)) in the *Configure Columns* form.

- **Column Value Radix:** The attribute value radix can be changed by right-clicking on the attribute name and selecting the radix from the menu. This is disabled when used in the predefined attributes and packed columns.

Figure 140 Change Value Radix



You can save or restore the configuration by clicking **Save** or **Restore** buttons.

Row (Transaction) Operations

The sorting of the rows (transactions) is based on increasing `begin_time` by default. When moving the cursor to the row and left-clicking on the row, the row is highlighted in yellow. Multiple row selection is also provided. The row is marked in red by middle-clicking. Only one marked transaction is supported in a tab. The following paragraphs describe the operations applied in rows.

- **Sort:** The transactions (rows) can be sorted based on the selected attribute (column). The sorting is applied in this page when you directly click on the attribute name ([Figure: Sorting in a Page](#)). Click on the attribute name again to switch the ascending and descending sorting direction.

Figure 141 Sorting in a Page

	\$stream	\$beginTime	\$endTime	\$label
7	\$event_root/u...	0	110	read_byte_seq0
5	\$event_root/u...	60	110	req
4	\$event_root/u...	110	200	write_byte_seq0
3	\$event_root/u...	120	200	req
6	\$event_root/u...	0	310	read_modify_...
2	\$event_root/u...	200	310	read_byte_seq0
1	\$event_root/u...	210	310	req

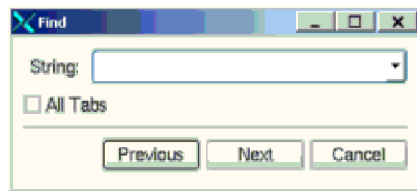
The global sorting action is also provided to sort all the transactions of all pages. Right-click on the attribute name and select the **Global Sort** command (refer to [Figure: Sorting all Transactions in All Pages](#)) to sort the transactions in ascending or descending direction.

Figure 142 Sorting all Transactions in All Pages

\$endTime	\$label	\$tag
110	Hide	TR.
110	Configure Columns...	TR.
200	Value Badix	TR.
200	Global Sort	TR.
310	read_modify_...	TR.

- **Search:** The transactions (attributes) can be searched using the **Search** commands for analyzing the stream. The **Search -> Go To** commands can be used to search the first transaction, last transaction, or marker transaction in this tab. The **Search -> Go To -> Time** command can be used to find the previous and closest transaction that has the `begin_time` attribute as a time value. You can use the **Search -> Find** command to find the entry of the first transaction that contains partial matching of the string value. If the **All Tabs** option is enabled, the searching is through all the tabs (see [Figure: Find String](#)).

Figure 143 Find String



The **Search -> Search by Attributes** command is used to search in the current tab using the rule displayed in the **Expression** field. The usage is the same as the description in the [Creating Virtual Streams](#) section. If the **All Tabs** option is enabled, the searching is through all tabs. You can save or append the searching conditions to a list file by clicking **Save** or **Append** buttons.

- **Editable/Non-editable Tab:** The included transactions are allowed to come from different streams and can be added or deleted (use the **Delete Transactions** command in the right-click command menu) in editable tabs. The transaction cannot be added or deleted in non-editable tabs.
- **Highlight Relation:** The relationship between transactions can be highlighted by right-clicking on the spreadsheet pane or selecting the **View -> Highlight Relation** command. For example, invoke the **View -> Highlight Relation -> Child** command. The child/children of the selected transaction are highlighted in light red (see [Figure: Highlight Relation Command](#)) and the *Status Message* is updated with the highlighted relationship of the transaction. If there are other related transactions in other tab(s), the related tab(s) text changes color too and you can see them after switching to that tab (see [Figure: Highlight Relation with Related Tab Text Color Changing](#)).

Figure 144 Highlight Relation Command

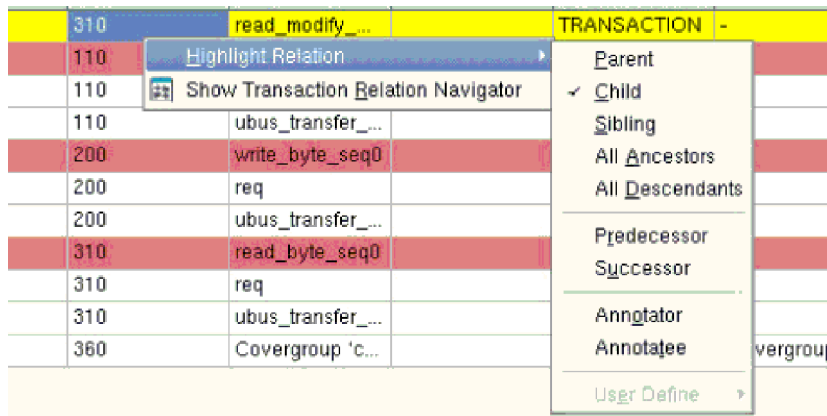
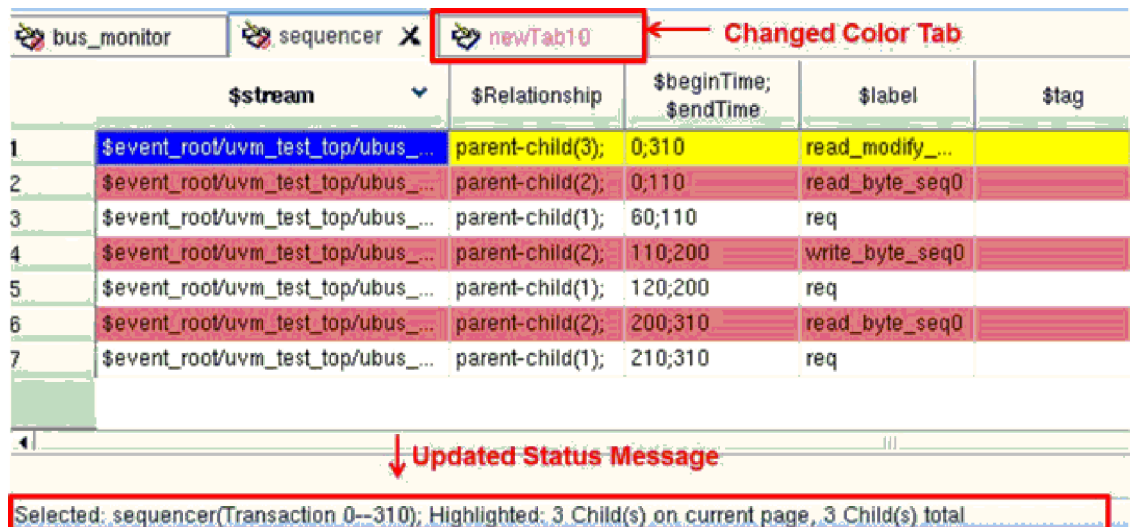
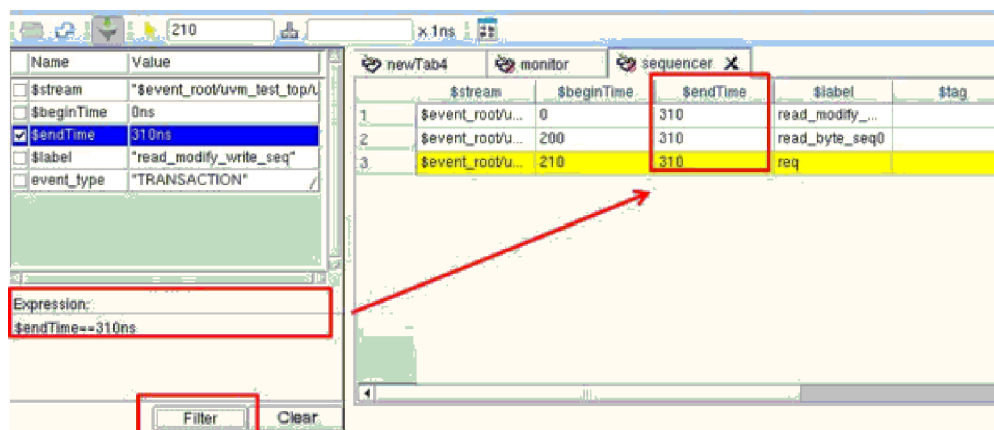


Figure 145 Highlight Relation with Related Tab Text Color Changing



- **Quick Filter:** You can quickly filter the transactions in the current stream using the *Quick Filter* form. The usage is the same as the description in the [Quick Filter Tab](#) section for the *Transaction and Protocol Analyzer* window. [Figure: Quick Filter in Transaction Table View](#) shows the result of filtering transactions quickly with the `$endTime==310ns` expression as an example. The same expression can be applied to other streams by switching to the tab and clicking the **Filter** button again.

Figure 146 Quick Filter in Transaction Table View



- **Filter/Colorize:** You can filter and color the transactions by selecting the **View -> Filter/Colorize** command. Refer to the [Filter/Colorize the Transactions](#) section in the *Transaction and Protocol Analyzer* chapter for details. The operation can apply to all tabs if the **All Tabs** option is enabled (see [Figure: Filter/Colorize Form](#)).

Figure 147 Filter/Colorize Form

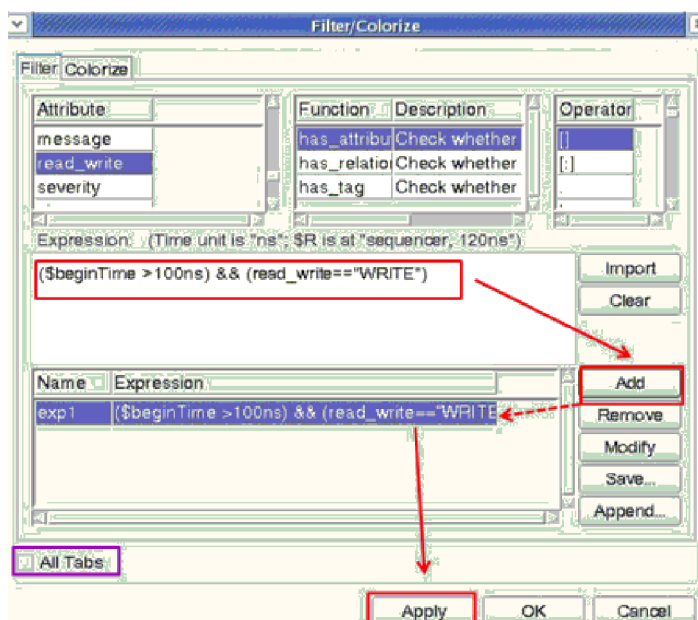


Figure: Filtered and Colorized Results in Transaction Table View shows the result of filtering and colorizing in green for the sequence stream with the `$beginTime > 100ns && read_write == "write"` expression.

Figure 148 Filtered and Colorized Results in Transaction Table View

Original Stream

	\$stream	\$beginTime	\$endTime	\$label	\$tag	event_type	addr	read_write	size
1	\$event_root/...	0	360(unfinish)	slave_memory...		TRANSACTION	-	-	-
2	\$event_root/...	80	110	req		TRANSACTION	'shb877	"READ"	'sh1
3	\$event_root/...	170	200	req		TRANSACTION	'shb877	"WRITE"	'sh1
4	\$event_root/...	280	310	req		TRANSACTION	'shb877	"READ"	'sh1

Stream with the Filtered Results

	\$stream	\$beginTime	\$endTime	\$label	\$tag	event_type	addr	read_write	size
1	\$event_root/...	170	200	req		TRANSACTION	'shb877	"WRITE"	'sh1

Stream with the Colorized Results

	\$stream	\$beginTime	\$endTime	\$label	\$tag	event_type	addr	read_write	size
1	\$event_root/...	0	360(unfinish)	slave_memory...		TRANSACTION	-	-	-
2	\$event_root/...	80	110	req		TRANSACTION	'shb877	"READ"	'sh1
3	\$event_root/...	170	200	req		TRANSACTION	'shb877	"WRITE"	'sh1
4	\$event_root/...	280	310	req		TRANSACTION	'shb877	"READ"	'sh1

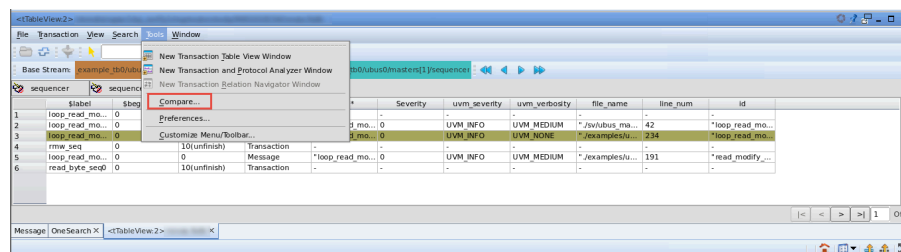
You can use the **View -> Clear Filtering Results** or **View -> Clear Colorization Results** commands to clear the filtered or the colorized result.

- **Display Class Definition:** If a transaction has the `class_name` attribute, you can use the **Display Class Definition** command in the right-click menu to trace the class definition (see section [Display Class Definition](#) for details).

Support for Transaction Comparison

The comparing feature is enabled in the transaction window. The *tTableView* is the transaction analysis window and you can provide comparison in this window. Choose the **Tools -> Compare** Menu/Toolbar command from the main window as shown in the following figure. The **Compare** command is enabled when FSDB is opened in *tTableView*.

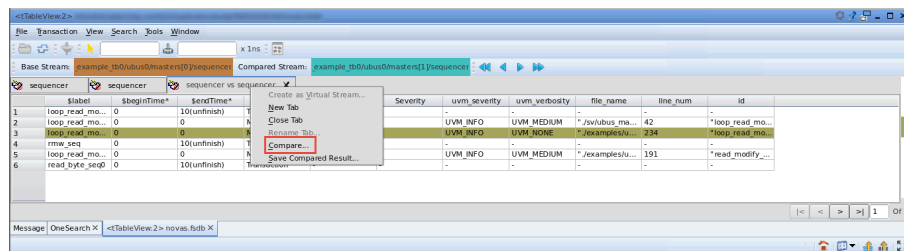
Figure 149 tTableView Compare Command



Appendix E: Appendix E: Using the Old Transaction Table View Reading and Analyzing Transactions in Transaction Table View

To invoke **Compare** command from the stream tab, click the right mouse button as shown in the following figure. The stream is specified as the base stream by default. If you invoke **Compare** command from the virtual stream or editable tab, the base stream needs to be specified.

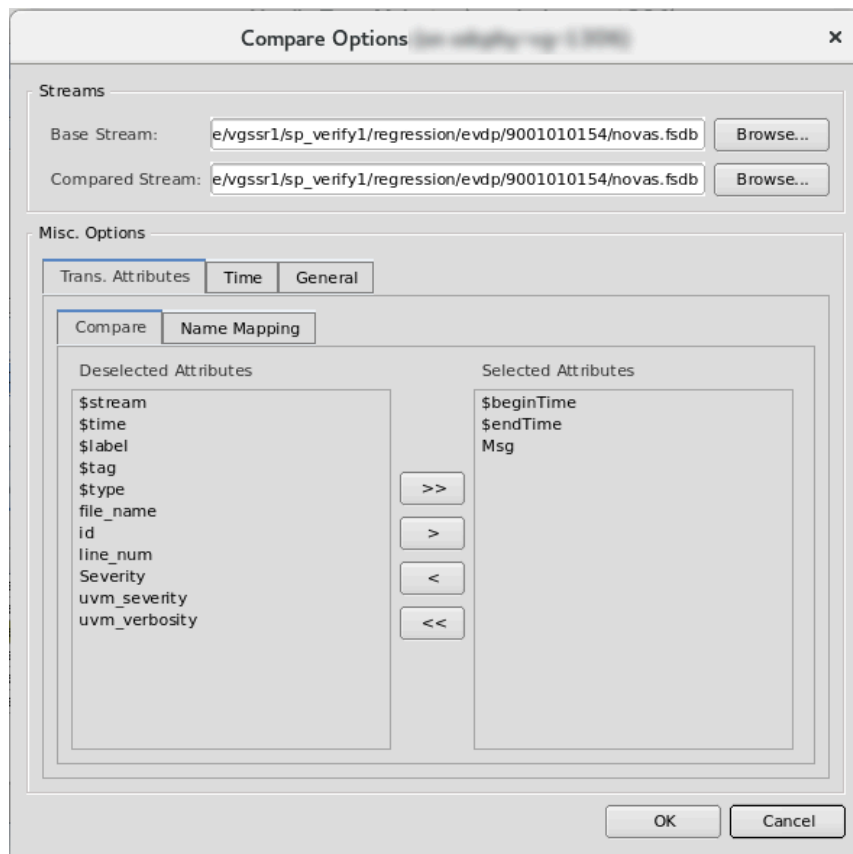
Figure 150 Compare Command From Stream Tab



Compare Options

After invoking the **Compare** command, *Compare Options* window opens as shown in the following figure:

Figure 151 *Compare Options Window*



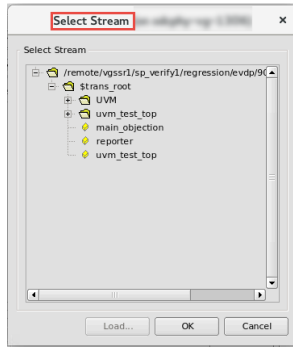
The *Streams* option includes the following fields:

- **Base Stream:** This text field specifies the base stream name in the FSDB file. The base stream name can be specified by either typing directly in the text field or by clicking the **Browse** button and selecting the stream name in the **Select Stream** form. This feature is not supported in the virtual stream.
- **Compared Stream:** This text field specifies the stream name to be compared in the FSDB file. The stream name can be specified by either typing directly in the text field or clicking the **Browse** button and selecting the stream name in the **Select Stream** form. This feature is not supported in the virtual stream.

When you browse the **Compared Stream** field, it shows streams in all tTableView/tBrowser FSDBs. It also allows you to specify the stream from new FSDB by **Load** button.

When you browse the **Base Stream** field, it shows streams only in the current tTableView opened FSDB and the **Load** button is disabled. The following figure shows the *Select Stream* window:

Figure 152 *Select Stream*



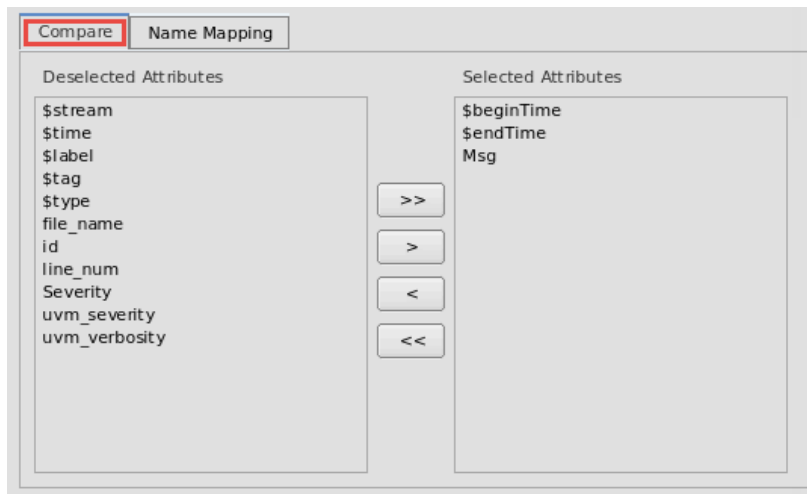
Trans. Attributes Tab

The following fields and buttons are available in the **Trans. Attributes** tab.

Compare

This tab specifies which attributes in the base stream is compared.

Figure 153 *Compare Tab*



- **Deselected Attributes:** This column lists the attributes that are not compared.
- **Selected Attributes:** This column lists the attributes that are compared. The `$beginTime` and `$endTime` attributes are listed by default. Use the  and  buttons to move the selected attributes to compare from **Deselected Attributes** to

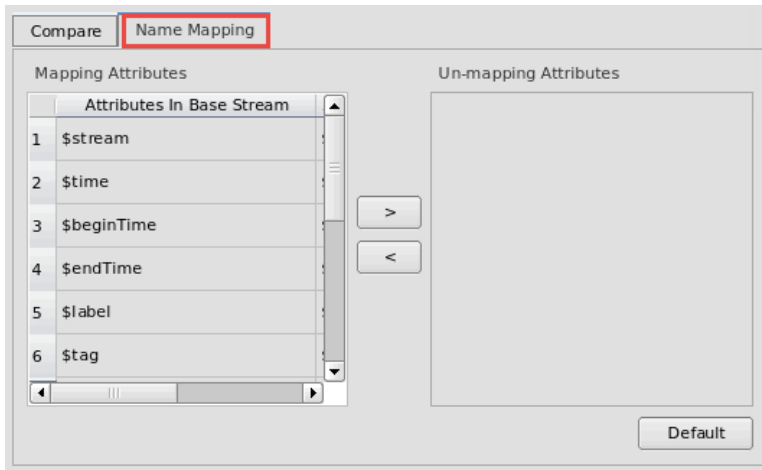
Selected Attributes, and vice versa. Only one attribute can be selected at a time.

Use the  and  buttons to move all the selected attributes to compare from **Deselected Attributes** to **Selected Attributes**, and vice versa.

Name Mapping

This tab is used to rearrange name mapping of each attribute in the compared stream to the attribute in the base stream.

Figure 154 Name Mapping Tab



- **Mapping Attributes:** This column lists the attributes that are mapped. Select the attribute in the white cells and use the  and  buttons to move the selected attributes from **Mapping Attributes** to **Unmapping Attributes**, and vice versa. The gray cells cannot be selected.
- **Unmapping Attributes:** This column lists the attributes without mapping. You can use the  and  buttons to move the selected attributes from **Unmapping Attributes** to **Mapping Attributes**, and vice versa. Click the **Default** button to reset all attributes back to the default name mapping. If a base attribute is selected to compare, but there is no compared attribute to map it, a warning pops up.

Time Tab

The Time Range section includes the **Time Range** and **Time Tolerance** options as shown in the following figure.

Figure 155 Time Tab

The screenshot shows the 'Time Tab' in a software interface. It has three tabs: 'Trans. Attributes', 'Time' (which is selected and highlighted with a red box), and 'General'. The 'Time Range' section contains two radio buttons: 'Full Range' (selected) and 'Special Range'. Below 'Special Range' are two sets of text fields: 'Base Stream: From' and 'To' (with a '1ns' unit indicator), and 'Compared Stream' (with a checkbox) and 'From' and 'To' (with a '1ns' unit indicator). The 'Time Tolerance' section contains two text fields: 'Tolerance of Begin Time' and 'Tolerance of End Time', both set to '0' with a '1ns' unit indicator.

Time Range

Only one of the following options can be selected at a time.

- **Full Range:** When this option is turned *on*, the full transaction time range is used for comparison. The default is *on*.
- **Specific Range:** When this option is turned *on*, the partial transaction time range in the stream is used for comparison. The default is *off*.
- **Base Stream:** Specifies the base stream time range used for comparison in the **From** and **To** text fields. The time unit is the file time unit of the base stream.
- **Compare Stream:** When this option is turned *on*, it specifies a different time range to use for the stream that is compared in the **From** and **To** text fields. When this option is turned *off*, the compared stream uses the same time range as the base stream. The time unit is the file time unit of the base stream.

Note:

An information pops-up if the base and compared stream have different time units.

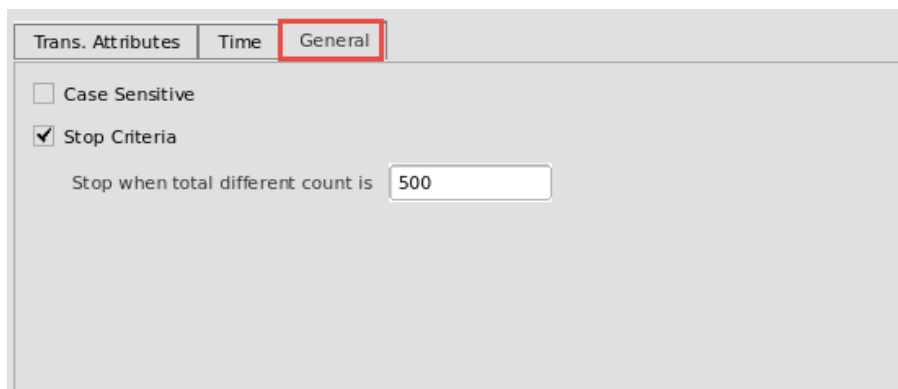
Time Tolerance

- **Tolerance of Begin Time:** Specifies the time tolerance for the comparison begin time. The default value is 0. This text field is available only when the **BeginTime** attribute is selected to be compared. The time unit is the file time unit of the base stream.
- **Tolerance of End Time:** Specifies the time tolerance for the comparison end time. The default value is 0. This text field is available only when the **EndTime** attribute is selected to be compared.

General Tab

The **General** tab is shown in the following figure:

Figure 156 General Tab



The **General** tab includes the following options:

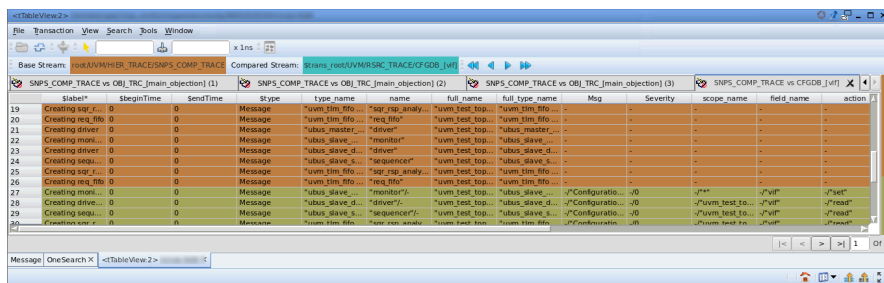
- **Case Sensitive:** When this option is turned *on*, attributes are compared with case-sensitivity. When this option is turned *off*, the attributes are compared without case-sensitivity. The default value is *off*.
- **Stop Criteria:** When this option is turned *on*, the maximum number of errors allowed before stopping the comparison are specified. The default value is *on*, and the default difference count is *500*.

After specifying the compare options, click **Ok** to save the settings and close the *Compare Options* window, then perform the comparison, or click the **Cancel** button to close the *Compare Options* window without making any changes.

Comparison Results

The comparison result is shown in the *Comparison* tab, which is none-editable. There is a toolbar to indicate Base and Compared Stream, and provide buttons to find first/previous/next/last different row in the current page.

Figure 157 Comparison Results



In the *Comparison* tab, the column headings marked with * indicate the attributes selected for comparison in the *Compare* tab of the **Compare Options** window.

Differences where only the base stream has the transaction are marked in dark orange . The dark yellow  rows indicates the base stream and compared stream have different attribute values, different values are separated by /. The rows having no color indicates the base stream and compared stream have same attribute values. Differences where only the compared stream has the transaction are marked in cyan .

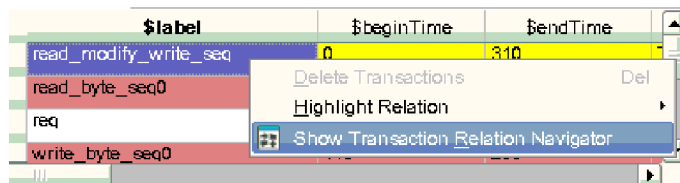
The **Difference Map** option on the right side displays the summary of different mismatch areas. Each of the difference areas are marked in a different color.

Click right mouse button on the *Comparison* tab to save the comparison result into text file using the **Save Compared Result** command.

Showing Transaction Relation Navigator

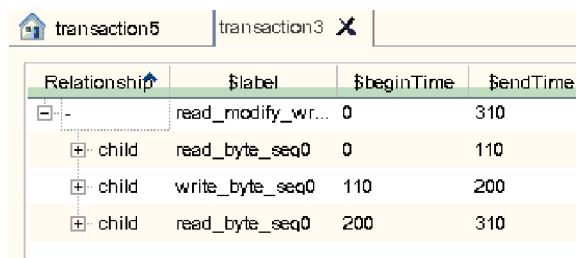
The transaction relationship can be displayed by right-clicking on a transaction and selecting the **Show Transaction Relation Navigator** command or clicking the  button on the toolbar to invoke the *Transaction Relation Navigator* window (*tRelation*).

Figure 158 Invoke the Transaction Relation Navigator Window from Transaction Table View



As shown in [Figure: Show the Relation in the Transaction Relation Navigator Window](#), the relationship of selected transaction is displayed on the *Transaction Relation Navigator* window. The [Using Transaction Relation Navigator](#) section describes how to use the *Transaction Relation Navigator* for observing the relationships.

Figure 159 Show the Relation in the Transaction Relation Navigator Window

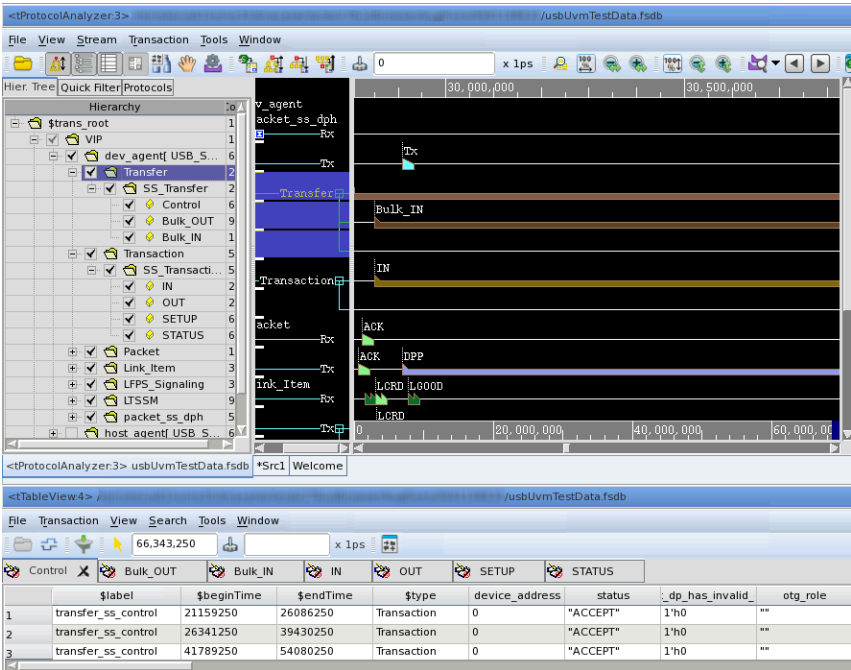


Auto-Switching During Sync Selection

To explain this feature, consider an example where you enable sync selection in three *Protocol Debug* windows. Consider that the active tab on *Transaction Table View* is stream Control. If you select a transaction of stream SS_Transfer on *Transaction and Protocol Analyzer*; *Transaction Table View* auto-switches to the BULK_IN tab.

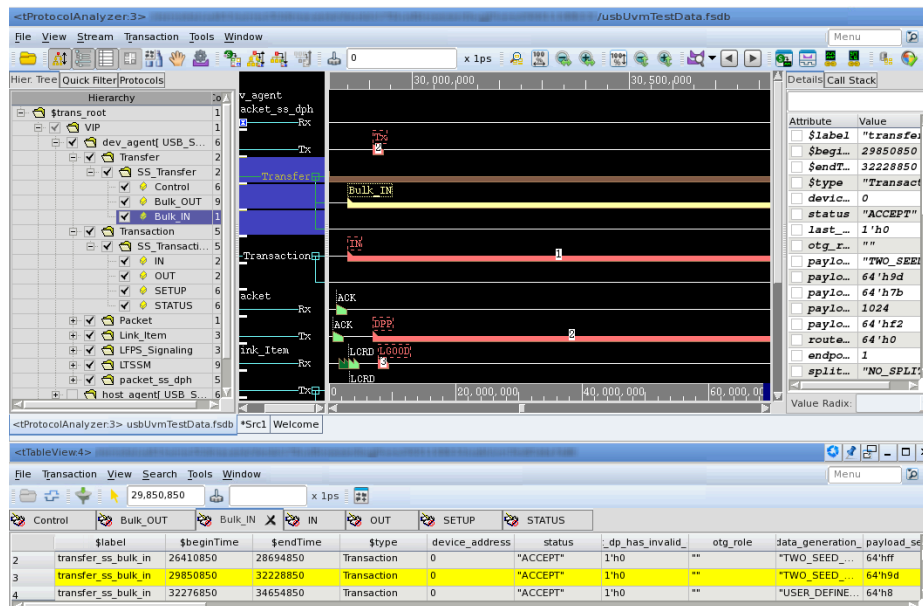
Figure: Before Selecting Transaction in Transaction and Protocol Analyzer and Figure: Transaction Table View Auto-Switches to Corresponding Tab show an example of before and after selecting a transaction in *Transaction and Protocol Analyzer* respectively. After selecting BULK_IN in *Transaction and Protocol Analyzer*, *Transaction Table View* auto-switches to the corresponding tab along with the transaction.

Figure 160 Before Selecting Transaction in Transaction and Protocol Analyzer



Appendix E: Appendix E: Using the Old Transaction Table View Auto-Switching During Sync Selection

Figure 161 Transaction Table View Auto-Switches to Corresponding Tab



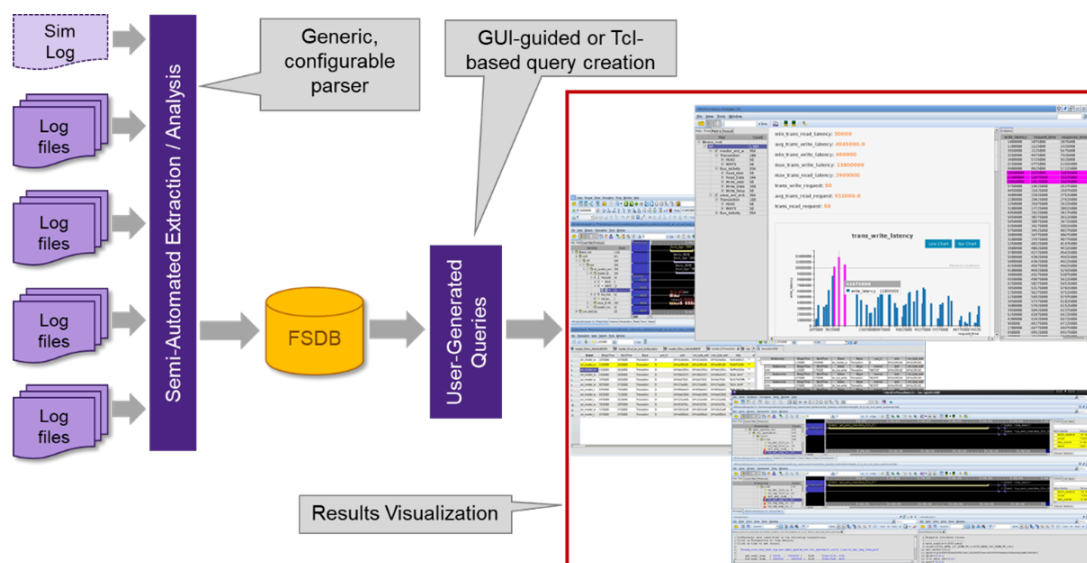
F

Appendix F: Log Conversion Using the log2transdb Utility

The information about transactions is written directly to one or more log files in textual format using the capabilities provided by UVM/OVM or plain SystemVerilog.

The Log2FSDB feature enables the visualization and debug of transactions by processing the textual descriptions of the transactions and converting them into a transaction FSDB database. The information about transactions is written to the simulation log file or to one or more additional external log files.

Log analysis includes log conversion and data visualization. Log conversion is supported using the `log2transdb` utility and log visualization is supported using Protocol Analyzer and Performance Analyzer.



For log conversion, you need not have to provide any rule file. However, to refine the details of the extracted information, you can pass the rules to understand the messages better.

Log Messages Supported by Log Analyzer

The Log Analyzer supports the following types of table formats:

- Single root tree-based table format
- Multiple root tree-based table format
- Predefined UVM/OVM information format
- User-defined messages
- Multiple-line table format

Logging and Debugging

Logging and debugging is done using the `log2transdb` utility.

Example

In the following example,

`chi_non_coherent_exclusive_access_random_target_hn_i_test.transcript` is the input file and the output is generated to the `test.fsdb` file.

```
Monitoring Toggle Coverage on Only Ports of a Design
$VERDI_HOME/edpProd/bin/log2transdb -db "test.fsdb" -scale "1.0ps"
  -protocol "chi" -rule
"$VERDI_HOME/share/log2transdb/default.rules" -treatUvmAsVIP -input
chi_non_coherent_exclusive_access_random_target_hn_i_test.transcript
```